

# **System Design Details**

## Table of Contents

|                                                                                         |     |
|-----------------------------------------------------------------------------------------|-----|
| Design 1 → Payment Gateway System (Stripe / PayPal-like) .....                          | 3   |
| Design 2 → WhatsApp / Chat System (Billions of Messages) .....                          | 16  |
| Design 3 → E-Commerce (Amazon-scale).....                                               | 31  |
| Design 4 → Banking Ledger System (Millions / Billions of Records).....                  | 46  |
| Design 5 → Social Media Feed (Facebook / X-like).....                                   | 61  |
| Design 6 → URL Shortener (TinyURL / Bitly).....                                         | 72  |
| Design 7 → Global API Gateway .....                                                     | 86  |
| Design 8 → Netflix / Video Streaming Platform (Azure + AWS).....                        | 100 |
| Design 9 → Notification System (Email / SMS / Push) .....                               | 116 |
| Design 10 → Uber / Ride Matching System .....                                           | 134 |
| Design 11 → Search Engine (Google-like basics) .....                                    | 151 |
| Design 12 → Distributed Rate Limiter .....                                              | 168 |
| Design 13 → Multi-Tenant SaaS Platform.....                                             | 186 |
| Design 14 → File Storage System (Dropbox / Google Drive).....                           | 206 |
| Design 15 → IoT Event Platform (10M–1B events/day) .....                                | 223 |
| Design 16 → Global Distributed Cache Platform (Redis-like) .....                        | 241 |
| Design 17 → Searchable Logging Platform (ELK / Splunk-like) .....                       | 259 |
| Design 18 → Order Management System (Enterprise Workflow + Saga) .....                  | 276 |
| Design 19 → YouTube-like Video Platform .....                                           | 295 |
| Design 20 → 10B+ Record Database Design (Sharding + Partitioning + Hot Partitions)..... | 314 |

# Design 1 → Payment Gateway System (Stripe / PayPal-like)

This is one of the strongest FAANG + Banking + Accenture Architect questions.

**Q1. How do you design a Payment Gateway handling millions of secure payments?**

**Answer:**

A Payment Gateway must safely handle:

- payment authorization
- retries
- fraud validation
- idempotency
- strong consistency
- settlement
- reconciliation
- audit trail
- refunds
- webhooks
- failure recovery
- high availability

This is a **financially critical distributed system**.

---

## 1. Functional Requirements

System should support:

- create payment
- authorize payment
- capture payment
- refund payment
- cancel payment
- retry failed payment

- payment status tracking
  - webhooks
  - transaction history
  - settlement
  - reconciliation
  - fraud checks
  - multi-currency
  - duplicate prevention
- 

## 2. Non-Functional Requirements

Need:

- zero double-charge
  - very high reliability
  - high consistency
  - auditability
  - PCI compliance
  - encryption
  - HA
  - low latency
  - fault tolerance
  - scalability
- 

## 3. Capacity Estimation (Example Large Scale)

Assume:

- 20M payments/day
- Peak = 50K TPS
- Avg txn = 2KB metadata

Impacts:

- DB sharding



- queue scaling
- idempotency store
- audit storage
- async processing

#### 4. High-Level Architecture Diagram



This is usually **sync + async hybrid architecture**.

#### 5. Core Components

##### Q2. What is API Gateway role?

**Answer:**

Handles:

- TLS termination
- auth

- rate limiting
  - WAF
  - request filtering
  - DDoS protection
  - logging
- 

### **Q3. What is Payment Orchestrator?**

#### **Answer:**

Main workflow coordinator.

Manages:

- validation
- fraud flow
- idempotency
- PSP routing
- retries
- state transition

Very important.

---

### **Q4. Why Orchestrator pattern?**

#### **Answer:**

Payment flows are state-heavy and business-critical.

Central orchestration improves:

- consistency
  - auditing
  - rollback
  - visibility
- 

### **Q5. Fraud Service role?**

#### **Answer:**

Checks:

- abnormal amount
  - geolocation mismatch
  - velocity attacks
  - suspicious behavior
  - stolen card patterns
- 

#### **Q6. Ledger Service role?**

**Answer:**

Maintains immutable financial records.

Huge banking concept.

Never trust mutable payment state only.

---

### **6. Payment Flow**

#### **Q7. End-to-end payment flow?**

**Answer:**

Step 1 → User clicks pay  
Step 2 → API Gateway validates request  
Step 3 → Idempotency check  
Step 4 → Fraud check  
Step 5 → Orchestrator creates payment  
Step 6 → PSP/bank authorization  
Step 7 → Success/Fail  
Step 8 → Ledger write  
Step 9 → DB update  
Step 10 → Event published  
Step 11 → Settlement  
Step 12 → Notification/Webhook

---

### **7. DB Design**

#### **Q8. Which DB?**

**Answer:**

Usually mix:

## Relational DB

- transactions
- payment state
- settlement

## Event Store / Ledger

- immutable history

## Redis

- idempotency
  - hot lookup
- 

### Q9. Why relational DB?

**Answer:**

Payments need:

- ACID
  - consistency
  - constraints
  - rollback
  - auditing
- 

### Q10. Why immutable ledger?

**Answer:**

Financial systems must never “overwrite truth.”

Use append-only records.

Very important.

---

## 8. Idempotency (Critical)

### Q11. Why idempotency critical?

**Answer:**

Prevent double charge.

Huge interview favorite.

Example:

User retries after timeout.

Only one payment should happen.

---

**Q12. Common idempotency implementation?**

**Answer:**

Use:

- Idempotency-Key
- unique request hash
- transaction token
- dedupe table
- Redis short-lived key

---

**9. Consistency / Distributed Transactions****Q13. 2PC or Saga?**

**Answer:**

Usually:

- Core financial DB → strong ACID
- Cross-service async → Saga / eventual consistency

Hybrid model.

---

**Q14. Why not full distributed 2PC?**

**Answer:**

Too expensive:

- blocking
- fragile
- slow
- hard recovery

---

## 10. Scaling Strategy

### Q15. How scale to millions of payments?

**Answer:**

Use:

- stateless orchestrators
- load balancers
- DB sharding
- async queues
- PSP routing pools
- caching
- partitioned Kafka
- multi-region HA

---

### Q16. What gets sharded?

**Answer:**

Usually:

- transaction table
- audit data
- ledger partitions
- webhook events

---

## 11. Retry Handling

### Q17. What if PSP timeout?

**Answer:**

Never blindly retry.

Check:

- payment status lookup
- idempotency state

- reconciliation
- safe retry path

Huge banking trap.

---

#### **Q18. Retry strategy?**

**Answer:**

Use:

- exponential backoff
  - DLQ
  - retry queue
  - bounded retries
- 

#### **12. Settlement + Reconciliation**

##### **Q19. What is Settlement?**

**Answer:**

Moving funds between institutions/accounts.

---

##### **Q20. What is Reconciliation?**

**Answer:**

Comparing internal records vs bank/PSP records.

Critical.

---

##### **Q21. Why reconciliation mandatory?**

**Answer:**

Detect:

- missing transactions
- duplicates
- mismatches
- settlement failures

---

### 13. Security (Very Important)

#### Q22. Security controls?

**Answer:**

Need:

- PCI DSS
- TLS
- tokenization
- encryption
- RBAC
- audit trail
- HSM / key vault
- mTLS
- WAF
- rate limiting

---

#### Q23. Why tokenization?

**Answer:**

Avoid storing raw card data.

Huge compliance topic.

---

### 14. Failure Handling

#### Q24. If DB write succeeds but event publish fails?

**Answer:**

Use **Outbox Pattern**.

Store event in DB transaction.

Publisher sends later.

Critical architect answer.

---



**Q25. If payment approved but client times out?**

**Answer:**

Client should call payment-status API.

Never re-charge blindly.

Huge trap.

---

**Q26. If duplicate webhook received?**

**Answer:**

Idempotent consumer.

---

**15. HA / DR**

**Q27. HA design?**

**Answer:**

Use:

- active-active services
  - multi-AZ DB
  - failover
  - queue replication
  - health checks
- 

**Q28. DR strategy?**

**Answer:**

- backup
  - multi-region
  - replication
  - tested restore
  - RPO/RTO
- 

**16. Bottlenecks**

## Q29. Hotspots?

Answer:

- idempotency store
  - ledger writes
  - fraud engine
  - PSP bottleneck
  - reconciliation
  - webhook spikes
- 

## 17. Trade-offs

### Q30. Strong consistency vs eventual consistency?

Answer:

**Strong Consistency    Eventual Consistency**

Ledger / money state notifications / analytics

Must know.

---

## 18. Anti-Patterns

### Q31. Dangerous anti-patterns?

Answer:

- retry without idempotency
  - mutable ledger
  - no reconciliation
  - no audit trail
  - shared DB across payment domains
  - blind duplicate webhook processing
- 

## 19. FAANG Trap Questions

### Q32. Can payment be eventually consistent?

**Answer:**

Core money state → no

Notifications/reporting → yes

---

**Q33. Can Redis alone store payment truth?**

**Answer:**

No.

---

**Q34. Can we update ledger rows?**

**Answer:**

Usually no. Append-only preferred.

---

**Final Architect Summary**

For **large-scale payment gateway**, best design is:

**Stateless Orchestrator + ACID DB + Immutable Ledger + Idempotency + Fraud Engine + Async Event Bus + Reconciliation + Strong Security + HA + Outbox + Multi-region DR**

## Design 2 → WhatsApp / Chat System (Billions of Messages)

This answers:

**“How do you design a real-time chat platform like WhatsApp handling billions of messages?”**

---

**Q1. What are core requirements of WhatsApp-like system?**

**Answer:**

### **Functional Requirements**

- 1:1 chat
- group chat
- online/offline messaging
- delivery acknowledgment
- read receipts
- typing indicators
- media/file sharing
- unread counts
- message history
- push notifications
- presence (online/offline/last seen)
- retries
- multi-device sync

### **Non-Functional Requirements**

- low latency (<100–300ms)
- massive concurrency
- high availability
- eventual consistency (acceptable in some areas)
- message durability
- ordered delivery
- fault tolerance

- horizontal scaling
  - encryption
  - geo-distribution
- 

## 1. Capacity Estimation (Large Scale)

Assume:

- 2B users
- 300M active users/day
- 50M concurrent connections
- 100B messages/day
- avg message = 1KB

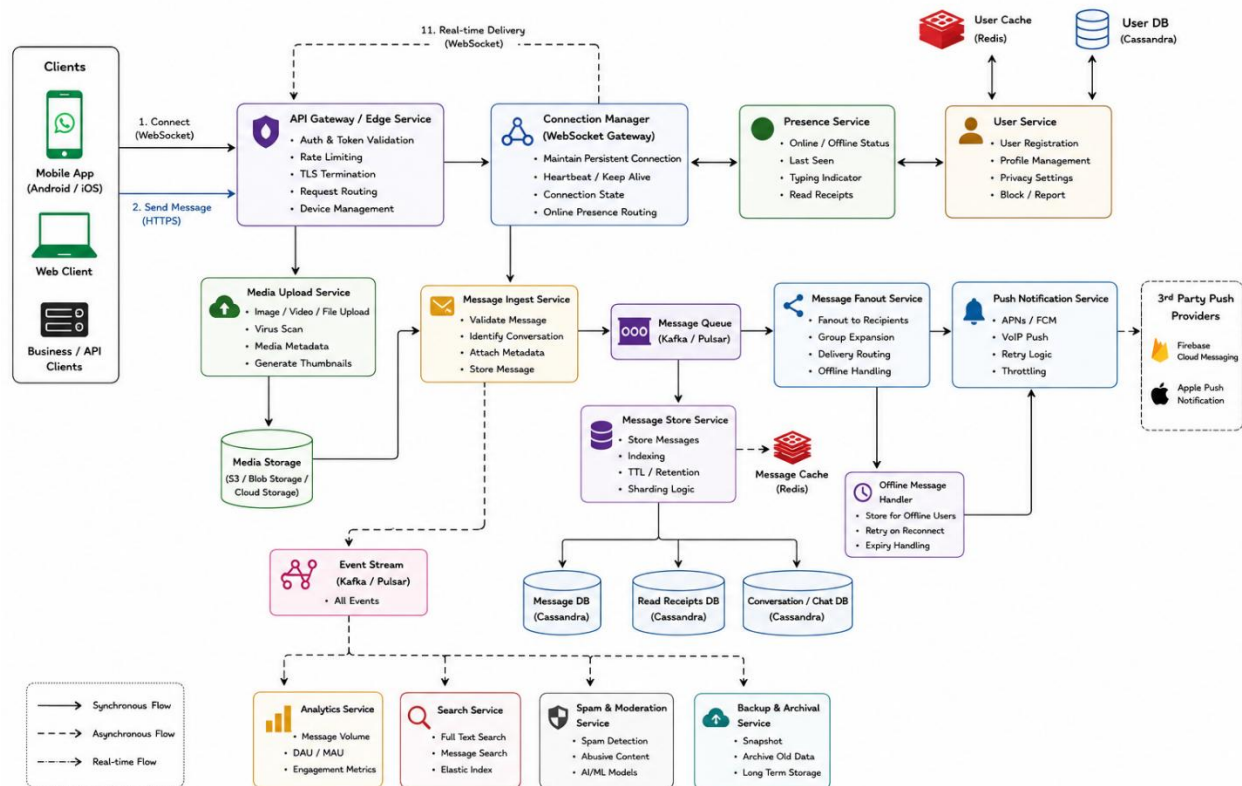
Challenges:

- persistent connections
- fan-out
- storage
- ordering
- hot users/groups

Huge scale problem.

---

## 2. High-Level Architecture Diagram



This is usually **event-driven + WebSocket + distributed storage**.

## 3. Core Components

### Q2. Why WebSockets instead of HTTP polling?

**Answer:**

WebSockets provide persistent bi-directional connection.

Better for:

- real-time messaging
- low latency
- lower network overhead
- typing indicators
- presence

Huge interview favorite.

### Q3. What is Chat Gateway?

#### Answer:

Persistent connection layer.

Handles:

- WebSocket connection
  - auth
  - connection routing
  - keepalive
  - heartbeat
  - message ingress
- 

### Q4. Why not keep DB directly behind clients?

#### Answer:

Need:

- routing
  - security
  - ordering
  - validation
  - throttling
  - retry control
- 

### Q5. What is Message Router?

#### Answer:

Maps user → correct chat shard/server.

Helps:

- routing
  - partitioning
  - horizontal scale
-

#### 4. Message Flow (1:1 Chat)

##### Q6. End-to-end message flow?

**Answer:**

- Step 1 → User A sends message
  - Step 2 → Chat Gateway receives
  - Step 3 → Auth + validation
  - Step 4 → Router identifies receiver shard
  - Step 5 → Persist message
  - Step 6 → Publish event
  - Step 7 → Deliver to receiver if online
  - Step 8 → ACK (sent/delivered/read)
  - Step 9 → Update unread counts
  - Step 10 → Push notification if offline
- 

#### 5. DB Design

##### Q7. Which DB for messages?

**Answer:**

Usually combination:

**Wide-column / NoSQL / partition-friendly**

- message history
- high write scale

**Redis**

- session state
- unread count
- presence

**Object Storage**

- media

Very common.

---

##### Q8. Why not traditional relational DB only?

**Answer:**

Chat = write-heavy + massive horizontal scale.



Cross-region + partitioning easier in distributed stores.

---

### **Q9. Partition strategy for messages?**

**Answer:**

Common:

- userId
- conversationId
- shardId
- region

Huge scale topic.

---

### **6. Ordering (Critical)**

#### **Q10. Why ordering matters?**

**Answer:**

Users expect messages in correct sequence.

Huge distributed challenge.

---

#### **Q11. How maintain ordering?**

**Answer:**

Use:

- per-conversation partition
  - sequence numbers
  - logical timestamps
  - single-writer shard patterns
- 

#### **Q12. Is global ordering needed?**

**Answer:**

No.

Usually per conversation/group ordering.

Common trap.

---

## **7. Online / Offline Delivery**

### **Q13. How detect online users?**

**Answer:**

Presence service tracks:

- active connection
  - heartbeat
  - last seen
- 

### **Q14. What if user offline?**

**Answer:**

Store message + push notification + sync later.

Critical chat behavior.

---

### **Q15. Multi-device sync?**

**Answer:**

Message events replayed/synced to all registered devices.

---

## **8. ACK / Delivery States**

### **Q16. Message states?**

**Answer:**

Usually:

- sent
  - delivered
  - read
  - failed
- 

### **Q17. Why ACK needed?**

**Answer:**

Supports reliability + UX visibility.

---

## **9. Group Messaging (Very Important)**

### **Q18. How group messaging works?**

**Answer:**

Sender → message service → fan-out to group members.

Huge scale topic.

---

### **Q19. Fan-out on Write vs Fan-out on Read?**

**Answer:**

**Fan-out on Write**

**Fan-out on Read**

Push to recipients early Compute later on fetch

Very common FAANG topic.

---

### **Q20. Which is better for chat?**

**Answer:**

Often fan-out on write for smaller groups.  
May optimize large groups differently.

Architect trade-off.

---

### **Q21. Celebrity / Huge group problem?**

**Answer:**

Large groups cause fan-out explosion.

Need:

- batching
- async queues
- partitioning
- lazy fan-out

Huge interview favorite.

---

## 10. Presence / Typing / Last Seen

### Q22. What is Presence Service?

**Answer:**

Tracks:

- online
  - offline
  - typing
  - last seen
- 

### Q23. Why keep presence separate?

**Answer:**

Highly volatile state. Better isolated.

---

## 11. Caching

### Q24. What to cache?

**Answer:**

Use Redis for:

- session mapping
  - unread count
  - recent chats
  - presence
  - routing metadata
- 

### Q25. Risks of caching?

**Answer:**

- stale presence
- hot keys

- consistency trade-offs
- 

## 12. Messaging Queue

### Q26. Why Kafka/Event Bus?

**Answer:**

Helps:

- decoupling
- async fan-out
- retries
- replay
- scaling

Huge architect topic.

---

### Q27. What if consumer fails?

**Answer:**

Retry + offset recovery + DLQ.

---

## 13. Reliability / Retry

### Q28. At-least-once or exactly-once?

**Answer:**

Usually at-least-once + idempotency.

Exactly-once is hard.

Very common.

---

### Q29. Duplicate messages?

**Answer:**

Use:

- message ID
- dedupe logic

- idempotent consumer
- 

## 14. Encryption / Security

### Q30. Security controls?

**Answer:**

Need:

- TLS
  - authentication
  - token/session security
  - rate limiting
  - abuse detection
  - encryption
- 

### Q31. End-to-End Encryption (E2EE)?

**Answer:**

Messages encrypted client-side.

Server routes ciphertext.

Huge WhatsApp concept.

---

### Q32. Why E2EE important?

**Answer:**

Privacy + zero-trust messaging path.

---

## 15. Media Handling

### Q33. How handle photos/videos?

**Answer:**

Store in object storage + CDN.

Message contains metadata/reference.

Huge scale decision.

---

**Q34. Why not store binary in message DB?**

**Answer:**

Large storage + slow access + scaling pain.

---

## **16. Scaling Strategy**

**Q35. How scale to billions of messages?**

**Answer:**

Use:

- stateless gateways
  - horizontal WebSocket servers
  - shard message DB
  - Kafka partitions
  - Redis clusters
  - geo routing
  - CDN
  - async fan-out
- 

**Q36. Hot users problem?**

**Answer:**

Some users/groups generate huge traffic.

Mitigate:

- shard spreading
  - queue buffering
  - cache partitioning
- 

## **17. HA / DR**

**Q37. HA design?**

**Answer:**

Use:

- multi-region
  - replicated brokers
  - gateway redundancy
  - DB replication
  - failover
- 

#### **Q38. DR?**

**Answer:**

- backup
  - cross-region recovery
  - replay logs
  - restore testing
- 

#### **18. Bottlenecks**

##### **Q39. Common hotspots?**

**Answer:**

- WebSocket connection scale
  - hot groups
  - presence writes
  - unread counters
  - fan-out
  - Kafka lag
- 

#### **19. Trade-offs**

##### **Q40. Strong consistency vs eventual consistency?**

**Answer:**



**Strong**

**Eventual**

Message persistence presence / last-seen / analytics

Important.

---

## 20. Anti-Patterns

### Q41. Dangerous anti-patterns?

**Answer:**

- DB per message sync blocking
  - global ordering
  - no idempotency
  - media in DB
  - no backpressure
  - single presence server
- 

## 21. FAANG Trap Questions

### Q42. Can chat require global ordering?

**Answer:**

No. Usually conversation-level ordering.

---

### Q43. Can relational DB alone handle WhatsApp scale?

**Answer:**

Usually not ideal.

---

### Q44. Can polling replace WebSocket?

**Answer:**

Possible, but inefficient at this scale.

---

## Final Architect Summary

For WhatsApp-scale system:

**WebSocket Gateways + Router + Message Service + NoSQL Message Store + Redis + Kafka + Push Notifications + Presence Service + Object Storage + E2EE + Partitioned Fan-out + Multi-region HA**

This is a strong **FAANG / Meta / Distributed Systems / L3 Architect** answer.

## Design 3 → E-Commerce (Amazon-scale)

This answers:

**“How do you design Amazon-like E-Commerce handling millions of users, products, orders, and payments?”**

We will cover:

- product catalog
- search
- cart
- inventory
- order management
- payment
- shipment
- recommendation
- caching
- scale
- consistency
- flash sale
- hot products
- fraud
- eventual consistency
- HA/DR

---

**Q1. What are core requirements of Amazon-scale E-Commerce?**

**Answer:**

### **Functional Requirements**

- user registration/login
- browse products
- search
- filter/sort

- cart
- checkout
- inventory
- order placement
- payment
- shipment tracking
- wishlist
- returns/refund
- recommendations
- notifications
- reviews/ratings

#### **Non-Functional Requirements**

- very high read scale
- strong consistency for orders/payments
- eventual consistency for search/recommendations
- HA
- low latency
- global scale
- fault tolerance
- fraud protection
- auditability

---

#### **1. Capacity Estimation**

Assume:

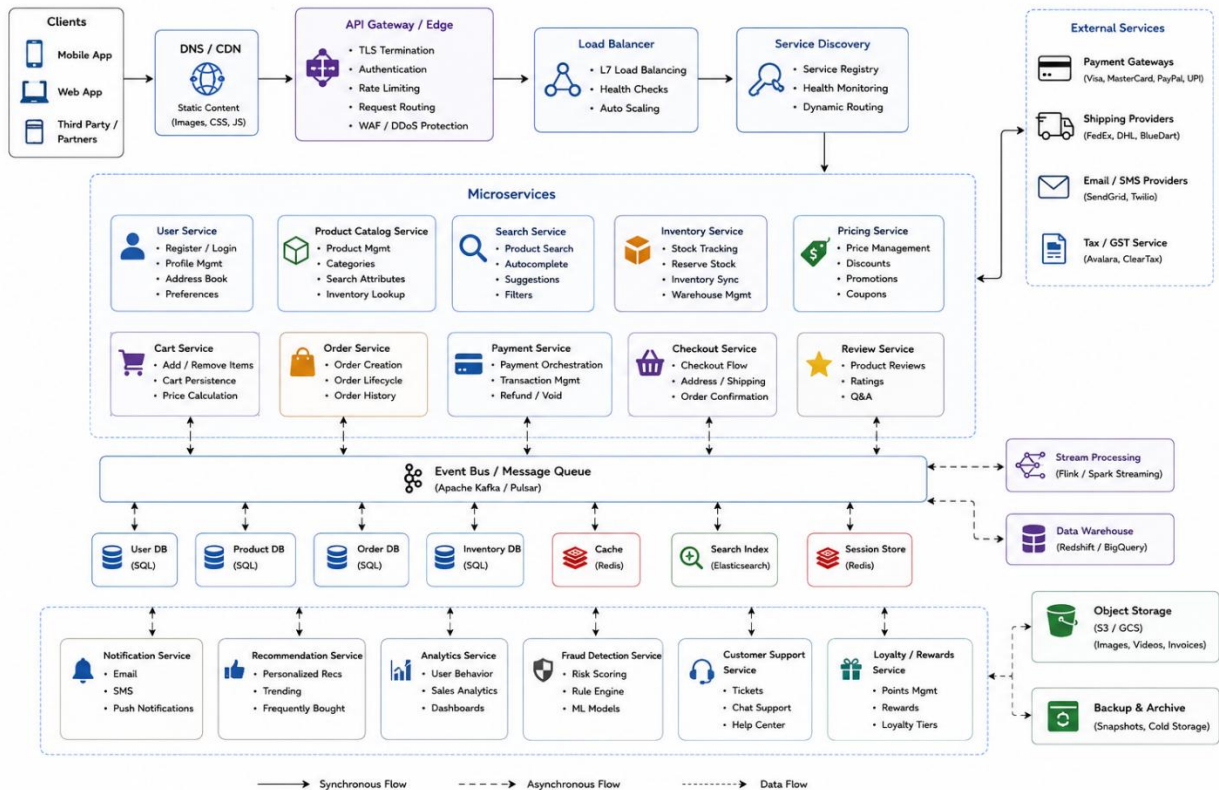
- 500M users
- 50M DAU
- 20M products
- 5M orders/day
- flash sale spikes

- read-heavy system (~90% reads)

Challenges:

- hot products
- inventory race
- cart scale
- search speed
- checkout reliability

## 2. High-Level Architecture Diagram



This is **microservices + event-driven + CQRS-friendly hybrid architecture**.

## 3. Core Services

Q2. Why microservices here?

Answer:

Because domains are independent:

- catalog
- cart
- payment
- shipping
- order
- inventory

Helps:

- scaling independently
  - team ownership
  - isolated deployment
- 

### **Q3. Main services?**

**Answer:**

Core:

- User Service
  - Product Service
  - Search Service
  - Cart Service
  - Inventory Service
  - Order Service
  - Payment Service
  - Shipping Service
  - Recommendation Service
  - Review Service
  - Notification Service
  - Fraud Service
- 

## **4. Product Catalog**

#### **Q4. How design product catalog?**

**Answer:**

Store:

- SKU
- title
- price
- variants
- category
- stock
- seller
- metadata

Usually read-heavy.

---

#### **Q5. Which DB for catalog?**

**Answer:**

Often:

- relational or document DB
- cache
- search index

Because:

- filtering
  - variants
  - metadata
- 

### **5. Search System**

#### **Q6. Why separate Search Service?**

**Answer:**

Search needs:

- full-text search

- ranking
- autocomplete
- typo tolerance
- faceting

Different from OLTP DB.

---

#### **Q7. Search consistency?**

**Answer:**

Usually eventual consistency.

Product DB → event → search index update.

Very common.

---

### **6. Cart Design**

#### **Q8. How design cart?**

**Answer:**

Usually:

- userId
- items
- qty
- coupon
- expiry

Often Redis or fast key-value layer.

---

#### **Q9. Why cache-heavy cart?**

**Answer:**

High read/write, low latency, temporary state.

---

### **7. Inventory (Critical)**

#### **Q10. Why inventory is hard?**



**Answer:**

Race conditions.

Multiple buyers may try same product.

Huge architect topic.

---

**Q11. How prevent overselling?**

**Answer:**

Use:

- reservation
- atomic decrement
- optimistic locking
- stock hold timeout
- queue

Critical.

---

**Q12. Strong or eventual consistency for inventory?**

**Answer:**

Usually stronger consistency than catalog.

Overselling is costly.

---

## **8. Order Flow**

**Q13. Order placement flow?**

**Answer:**

Step 1 → Checkout

Step 2 → Validate cart

Step 3 → Reserve inventory

Step 4 → Create order

Step 5 → Payment

Step 6 → Confirm

Step 7 → Ship

Step 8 → Notify

---

## 9. Payment Integration

### Q14. Separate payment service?

**Answer:**

Yes.

Handles:

- auth
- retries
- refunds
- PCI
- idempotency
- webhooks

---

### Q15. Why not combine order + payment?

**Answer:**

Different domain and compliance boundaries.

---

## 10. Order / Inventory / Payment Consistency

### Q16. 2PC or Saga?

**Answer:**

Usually Saga.

Order:

- reserve inventory
- create payment
- confirm order
- compensate if fail

Huge microservices topic.

---

### Q17. Compensating action?

**Answer:**

If payment fails → release inventory.

---

## **11. Shipping**

**Q18. Shipping service role?**

**Answer:**

Manages:

- warehouse assignment
  - courier
  - tracking
  - delivery state
  - ETA
- 

## **12. Recommendations**

**Q19. Recommendation system?**

**Answer:**

Uses:

- history
- clicks
- category
- collaborative filtering
- ML

Usually async.

---

## **13. Reviews**

**Q20. Review design challenges?**

**Answer:**

- spam
- abuse

- hot products
  - ranking
  - moderation
- 

## 14. Caching Strategy

### Q21. What to cache?

**Answer:**

Redis/CDN:

- hot products
  - pricing
  - cart
  - sessions
  - recommendations
  - search suggestions
- 

### Q22. Hot product problem?

**Answer:**

Popular products create huge traffic.

Mitigate:

- CDN
- cache
- replica scaling
- partition spread

Huge FAANG topic.

---

## 15. DB Strategy

### Q23. DB choices?

**Answer:**

Usually polyglot:

- Relational DB → orders/payments
  - NoSQL/Doc DB → product metadata
  - Search Index → search
  - Redis → cache/cart/session
  - Object Store → images/videos
- 

#### **Q24. Why DB-per-service?**

**Answer:**

Isolation + scale + ownership.

---

#### **16. Event Bus**

#### **Q25. Why Kafka/Event Bus?**

**Answer:**

Used for:

- order events
  - stock updates
  - shipment
  - recommendations
  - analytics
  - notifications
- 

#### **17. Flash Sale / High Scale**

#### **Q26. Flash sale challenge?**

**Answer:**

Massive spike.

Huge hotspot problem.

---

#### **Q27. How handle flash sale?**

**Answer:**

Use:

- queue
- pre-stock reservation
- throttling
- CDN
- rate limiting
- cache
- virtual waiting room

Very common.

---

## **18. Fraud / Security**

### **Q28. Fraud controls?**

**Answer:**

- abnormal orders
  - velocity checks
  - location mismatch
  - suspicious payments
  - abuse detection
- 

### **Q29. Security controls?**

**Answer:**

- auth
  - TLS
  - WAF
  - RBAC
  - PCI
  - tokenization
  - secret management
-

## 19. HA / DR

### Q30. High availability?

**Answer:**

Use:

- multi-AZ
  - stateless services
  - DB replicas
  - failover
  - queue replication
  - health checks
- 

### Q31. DR strategy?

**Answer:**

- backups
  - cross-region
  - restore tests
  - RPO/RTO
- 

## 20. Bottlenecks

### Q32. Common hotspots?

**Answer:**

- inventory writes
  - flash-sale items
  - payment
  - search spikes
  - cart hot keys
  - recommendation latency
- 

## 21. Trade-offs

### Q33. Strong consistency vs eventual consistency?

Answer:

**Strong Consistency**

**Eventual Consistency**

order, payment, inventory search, recommendation, analytics, reviews

Critical.

---

## 22. Anti-Patterns

### Q34. Dangerous anti-patterns?

Answer:

- shared DB for all services
  - sync coupling everywhere
  - no inventory reservation
  - no idempotency
  - search inside OLTP DB
  - no cache
- 

## 23. FAANG Trap Questions

### Q35. Can search be strongly consistent?

Answer:

Usually not required.

---

### Q36. Can cart stay only in DB?

Answer:

Possible, but cache/KV better at scale.

---

### Q37. Can order + payment be one service?

Answer:

Possible small-scale; usually separate at Amazon-scale.

---



---

## Final Architect Summary

Best architecture:

**Microservices + API Gateway + Product/Search/Cart/Inventory/Order/Payment + Kafka + Redis + Polyglot DB + Saga + CDN + Fraud + HA/DR + Eventual consistency where acceptable**

This is a strong **Amazon / Walmart / FAANG / Enterprise Architect** answer.

## Design 4 → Banking Ledger System (Millions / Billions of Records)

This answers:

**“How do you design a banking ledger system that handles billions of transactions with strong consistency, auditability, and zero financial corruption?”**

This is one of the strongest architect questions.

We cover:

- ledger
- ACID
- double-entry bookkeeping
- immutable events
- reconciliation
- settlement
- sharding
- HA
- DR
- fraud
- audit
- idempotency
- consistency
- replay
- compliance

---

**Q1. What are core requirements of Banking Ledger System?**

**Answer:**

### **Functional Requirements**

- account creation
- deposits
- withdrawals

- transfer
- payment posting
- reversal
- refund
- settlement
- reconciliation
- audit trail
- transaction history
- balance lookup
- freeze account
- fraud checks

#### **Non-Functional Requirements**

- strong consistency
- zero money loss
- no duplicate debit
- auditability
- immutability
- HA
- DR
- compliance
- encryption
- scalability
- fault tolerance

---

#### **1. Capacity Estimation**

Assume:

- 500M accounts
- 5B–20B ledger rows
- 20M transfers/day

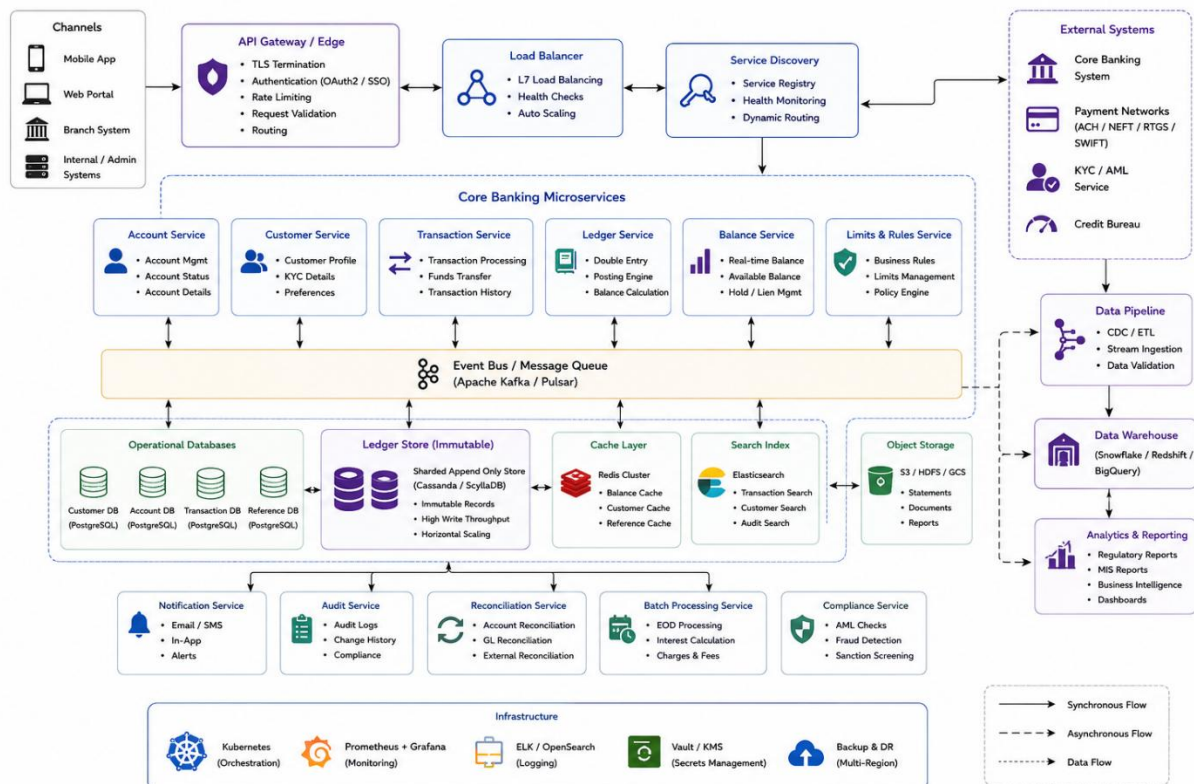
- peak = 100K TPS
- retention = 10+ years

Challenges:

- write-heavy
- financial correctness
- replay
- partitioning
- reconciliation
- DR

Huge enterprise topic.

## 2. High-Level Architecture Diagram



This is **strong-consistency core + async event-driven downstream architecture**.

### 3. Core Services

#### Q2. Why Transaction Orchestrator?

**Answer:**

Central coordinator for:

- transfer validation
- workflow
- routing
- idempotency
- balance rules
- fraud flow
- posting state

Very common architect pattern.

---

#### Q3. Why Ledger Service separate?

**Answer:**

Ledger is critical financial truth.

Should isolate:

- posting logic
- balancing
- accounting rules
- immutability

Never tightly couple with UI/business layer.

---

### 4. Double-Entry Bookkeeping (Critical)

#### Q4. What is Double-Entry Ledger?

**Answer:**

Every transaction has:

- debit
- credit

Money movement must balance.

Huge banking concept.

Example:

Transfer ₹100

Account A = -100

Account B = +100

Net = 0

---

#### **Q5. Why double-entry important?**

**Answer:**

Prevents:

- money creation
- mismatch
- accounting corruption

Critical.

---

#### **5. Ledger Flow**

##### **Q6. End-to-end transfer flow?**

**Answer:**

Step 1 → Request transfer  
Step 2 → Validate auth  
Step 3 → Idempotency check  
Step 4 → Fraud check  
Step 5 → Rules validation  
Step 6 → Begin DB transaction  
Step 7 → Debit source  
Step 8 → Credit target  
Step 9 → Ledger post  
Step 10 → Commit  
Step 11 → Publish event  
Step 12 → Notify/reconcile

---

#### **6. DB Design**

### Q7. Core tables?

**Answer:**

Usually:

#### **Accounts**

- accountId
- status
- customer
- currency

#### **Journal / Ledger Entries**

- txnId
- debitAccount
- creditAccount
- amount
- timestamp

#### **Balance View**

- optimized read model

#### **Audit**

- actor
- event
- metadata

---

### Q8. Why immutable ledger?

**Answer:**

Never overwrite transaction truth.

Use append-only journal.

Huge architect answer.

---

### Q9. Can balance be calculated every time from ledger?

**Answer:**

Possible, but expensive.

Usually:

- immutable journal
- derived balance view

CQRS-like model.

---

## **7. ACID / Consistency**

### **Q10. Why ACID mandatory?**

**Answer:**

Financial correctness.

Need:

- atomicity
  - consistency
  - isolation
  - durability
- 

### **Q11. Eventual consistency allowed?**

**Answer:**

Strong consistency:

- debit/credit
- balance
- posting

Eventual consistency:

- notifications
- analytics
- reporting

Very common.

---

## **8. Idempotency**



**Q12. Why idempotency critical?**

**Answer:**

Prevent duplicate transfer/debit.

Huge banking topic.

---

**Q13. Common idempotency implementation?**

**Answer:**

Use:

- request key
  - unique txn ID
  - dedupe store
  - short-lived cache
  - DB uniqueness
- 

**9. Sharding / Scale**

**Q14. How scale to billions of rows?**

**Answer:**

Use:

- account-based sharding
  - region sharding
  - time partitioning
  - archive strategy
  - read replicas
- 

**Q15. What to shard?**

**Answer:**

Usually:

- ledger journal
- audit

- reporting
  - settlement data
- 

#### **Q16. Risks of sharding?**

**Answer:**

- cross-shard transfers
- balancing complexity
- reconciliation overhead

Huge architect trade-off.

---

#### **10. Reconciliation / Settlement**

##### **Q17. What is Settlement?**

**Answer:**

Actual money movement between institutions/banks.

---

##### **Q18. What is Reconciliation?**

**Answer:**

Compare:  
internal ledger vs external bank/PSP records.

Critical.

---

##### **Q19. Why reconciliation mandatory?**

**Answer:**

Detect:

- missing txn
  - duplicates
  - settlement mismatch
  - corruption
-

## 11. Fraud / Rules Engine

### Q20. Fraud service checks?

Answer:

- abnormal amount
  - unusual velocity
  - geo mismatch
  - repeated failures
  - account anomaly
- 

### Q21. Why Rules Engine?

Answer:

Centralize business policies:

- limits
- AML
- KYC rules
- transfer rules
- freeze logic

Enterprise-grade.

---

## 12. Event Bus

### Q22. Why Kafka/Event Bus?

Answer:

Used for:

- notifications
  - reporting
  - reconciliation
  - downstream processing
  - replay
-

**Q23. Can event bus be source of financial truth?**

**Answer:**

No.

Ledger DB is truth.

Huge trap.

---

### **13. Audit / Compliance**

**Q24. Why audit trail critical?**

**Answer:**

Banking needs:

- who
- what
- when
- source
- traceability

Compliance.

---

**Q25. Compliance areas?**

**Answer:**

- encryption
  - PCI
  - KYC
  - AML
  - retention
  - access control
  - auditability
- 

### **14. Security**

**Q26. Security controls?**

**Answer:**

Need:

- TLS
  - mTLS
  - RBAC
  - HSM
  - tokenization
  - encryption
  - secret vault
  - key rotation
  - WAF
- 

## **15. Failure Handling**

**Q27. DB commit success but event publish fails?**

**Answer:**

Use Outbox Pattern.

Huge architect answer.

---

**Q28. If transfer partially processed?**

**Answer:**

Should not happen inside ACID posting.

Rollback.

---

**Q29. If reconciliation mismatch?**

**Answer:**

Flag:

- freeze
- investigate
- repair

- audit escalation
- 

## 16. HA / DR

### Q30. High availability?

**Answer:**

Use:

- active-active app
  - multi-AZ DB
  - replication
  - failover
  - health checks
- 

### Q31. Disaster recovery?

**Answer:**

- geo replication
- backup
- restore test
- replay journal
- RPO/RTO

Very strong.

---

## 17. Bottlenecks

### Q32. Common hotspots?

**Answer:**

- hot accounts
- balance updates
- fraud checks
- cross-shard transfers
- reconciliation

- journal writes
- 

## 18. Trade-offs

### Q33. Why CQRS useful here?

**Answer:**

Separate:

- write-heavy immutable ledger
- read-heavy balance/reporting

Huge architect topic.

---

### Q34. Why not update balance only?

**Answer:**

Loses financial history.

Dangerous.

---

## 19. Anti-Patterns

### Q35. Dangerous anti-patterns?

**Answer:**

- mutable ledger
  - single balance-only DB
  - no reconciliation
  - no audit trail
  - event bus as source of truth
  - retry without idempotency
- 

## 20. FAANG / Banking Trap Questions

### Q36. Can Redis store financial truth?

**Answer:**

No.

---

**Q37. Can eventual consistency handle debit/credit?**

**Answer:**

Usually no.

---

**Q38. Can ledger rows be updated?**

**Answer:**

Normally no. Append-only.

---

### **Final Architect Summary**

Best design:

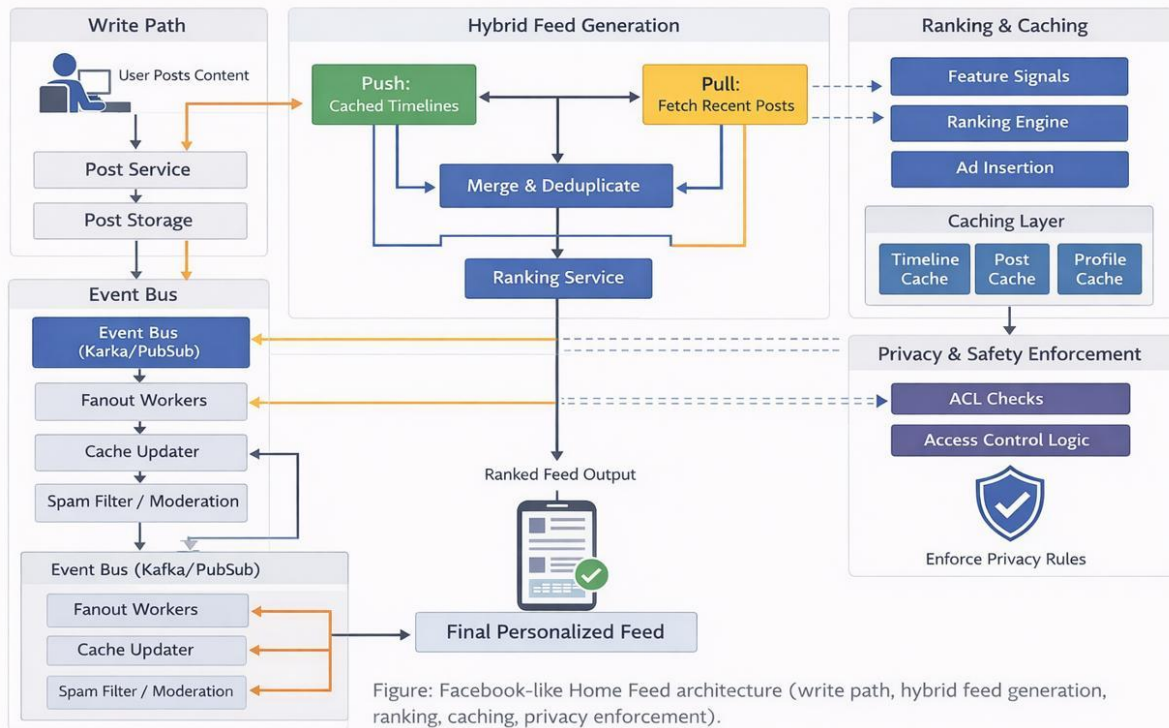
**Transaction Orchestrator + ACID DB + Double-Entry Immutable Ledger + CQRS Balance View + Fraud + Rules Engine + Idempotency + Kafka + Reconciliation + Audit + HA/DR + Strong Consistency**

This is a strong **Banking / FinTech / Accenture / Enterprise Architect** answer.



## Design 5 → Social Media Feed (Facebook / X-like)

### Facebook-Like Home Feed: High-Level System Design





- recommendation
  - notifications
  - media scale
- 

### **Q1. What are core requirements of Social Media Feed System?**

**Answer:**

#### **Functional Requirements**

- create post
- follow/unfollow
- personalized feed
- like/comment/share
- media upload
- notifications
- trending feed
- search
- real-time updates
- ads insertion

#### **Non-Functional Requirements**

- billions of users
  - low latency (<200ms)
  - massive read scale
  - HA
  - geo-distribution
  - feed personalization
  - eventual consistency acceptable
  - horizontal scale
- 

### **1. Capacity Estimation**

Assume:

- 2B users
- 400M DAU
- 50M posts/day
- millions of reads/sec
- feed = highly read-heavy

Challenges:

- celebrity users
- hot posts
- feed ranking
- cache invalidation

---

## 2. High-Level Architecture

### Q2. Core services?

**Answer:**

Usually:

- Auth Service
- User Service
- Post Service
- Feed Service
- Social Graph Service
- Media Service
- Notification Service
- Search Service
- Ranking/Recommendation Service
- Ad Service

---

## 3. Social Graph

### Q3. What is Social Graph Service?

**Answer:**

Stores relationships:

- followers
- following
- blocks
- friend edges

Critical for feed generation.

Often graph DB or adjacency model.

---

#### 4. Feed Generation (Critical)

##### Q4. What is Fan-out on Write?

**Answer:**

When user posts → push post to follower feeds immediately.

Good for:

- regular users
- lower follower count

Huge FAANG topic.

---

##### Q5. What is Fan-out on Read?

**Answer:**

When user opens app → generate feed dynamically.

Good for:

- celebrities
  - massive followers
- 

##### Q6. Fan-out on Write vs Read?

**Answer:**

**Fan-out on Write**

Push early

**Fan-out on Read**

Compute later

### Fan-out on Write

Fast reads

Can explode for celebrities

Critical.

### Fan-out on Read

Heavier reads

Better for hot users

---

## Q7. Best approach?

**Answer:**

Usually **Hybrid**.

- Normal users → Fan-out on write
- Celebrity/high-follower users → Fan-out on read

Very common architect answer.

---

## 5. Post Flow

### Q8. End-to-end post flow?

**Answer:**

Step 1 → User creates post

Step 2 → Post Service stores metadata

Step 3 → Media to object storage

Step 4 → Event published

Step 5 → Feed Service updates timelines

Step 6 → Ranking service orders content

Step 7 → Cache update

Step 8 → Notification

---

## 6. Feed Ranking

### Q9. How rank feed?

**Answer:**

Based on:

- recency
- engagement
- relevance

- relationship strength
- ads
- ML ranking

Huge FAANG concept.

---

## 7. DB Design

### Q10. DB choices?

**Answer:**

Usually polyglot:

- NoSQL → posts/feed store
  - Graph DB → follower graph
  - Redis → hot feed cache
  - Search Index → search
  - Object Storage → media
  - Relational → users/auth
- 

### Q11. Why graph DB?

**Answer:**

Fast traversal for:

- follower network
  - recommendations
  - relationship queries
- 

## 8. Caching

### Q12. What to cache?

**Answer:**

Redis/CDN:

- user feed
- trending posts

- profile metadata
  - follower count
  - hot posts
- 

### **Q13. Hot post problem?**

#### **Answer:**

One viral post causes huge reads.

Mitigate:

- CDN
  - feed cache
  - replicas
  - partition spread
- 

## **9. Celebrity Problem**

### **Q14. What is celebrity problem?**

#### **Answer:**

A user with millions of followers.

Fan-out on write becomes too expensive.

Huge FAANG favorite.

---

### **Q15. How solve celebrity problem?**

#### **Answer:**

Use:

- fan-out on read
  - async queue
  - partial lazy generation
  - cache
- 

## **10. Event Bus**



## Q16. Why Kafka/Event Bus?

**Answer:**

Used for:

- feed update
  - likes/comments
  - notifications
  - analytics
  - ranking pipelines
- 

## 11. Consistency

### Q17. Strong or eventual consistency?

**Answer:**

Strong:

- post creation
- auth/security

Eventual:

- feed propagation
  - likes count
  - analytics
  - recommendations
- 

## 12. Media Handling

### Q18. How store videos/images?

**Answer:**

Object storage + CDN.

Metadata in DB.

---

## 13. HA / DR

### Q19. HA design?

**Answer:**

Use:

- multi-region
  - cache clusters
  - replicated DB
  - stateless services
  - failover
- 

## **14. Security**

**Q20. Security concerns?**

**Answer:**

- auth
  - abuse/spam
  - rate limiting
  - DDoS
  - media validation
  - privacy rules
- 

## **15. Bottlenecks**

**Q21. Common hotspots?**

**Answer:**

- celebrity fan-out
  - feed ranking
  - cache hot keys
  - graph traversal
  - viral media
- 

## **16. Anti-Patterns**

**Q22. Dangerous anti-patterns?**

**Answer:**

- only fan-out on write
  - no cache
  - global feed ordering
  - media in DB
  - sync-heavy ranking
- 

## **17. FAANG Trap Questions**

**Q23. Is global ordering needed?**

**Answer:**

No. Personalized feed matters more.

---

**Q24. Can relational DB alone scale for this?**

**Answer:**

Usually not ideal.

---

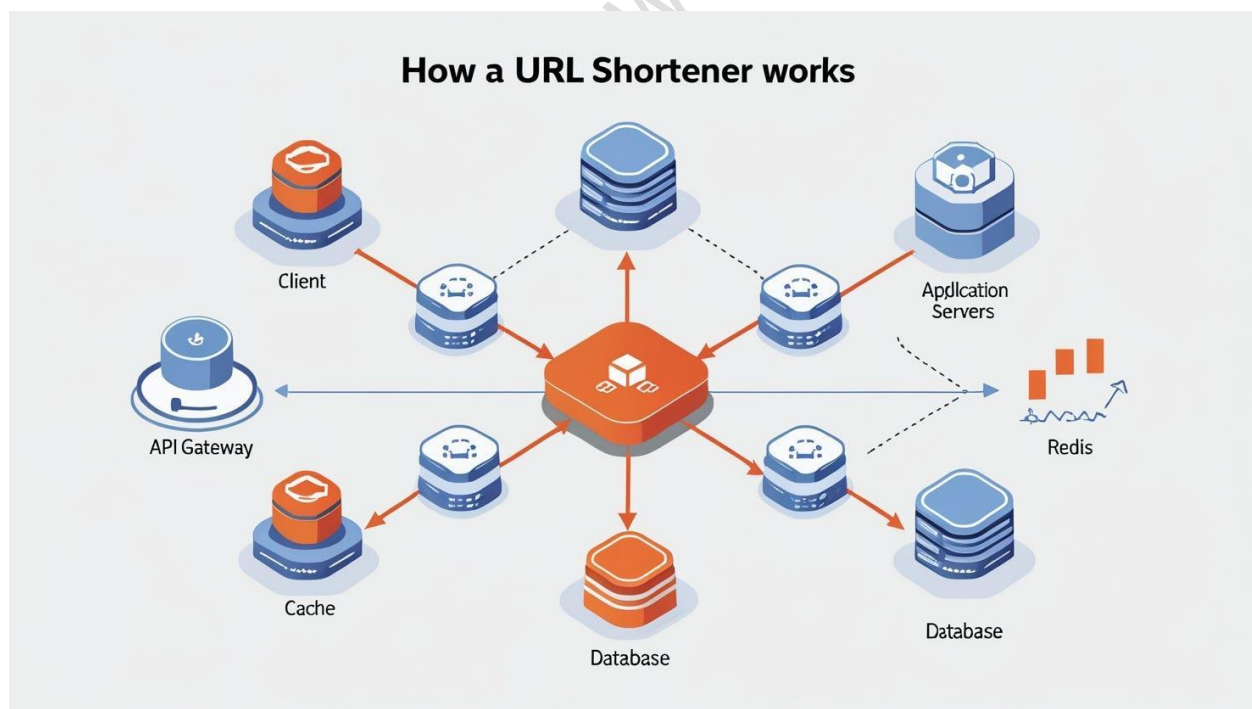
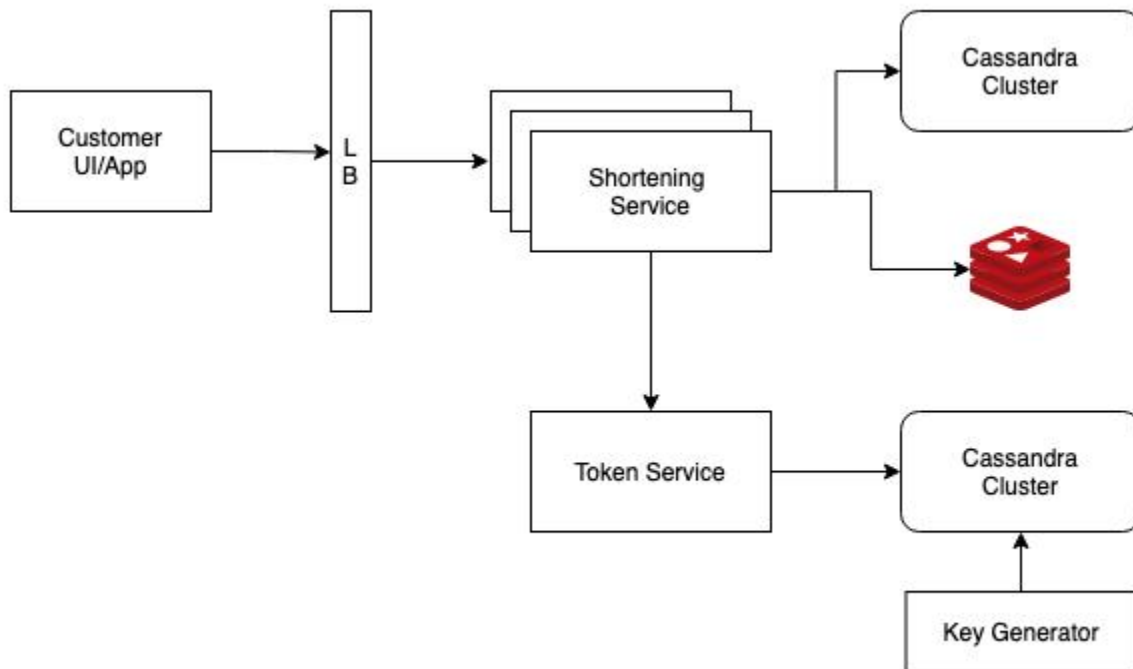
## **Final Architect Summary**

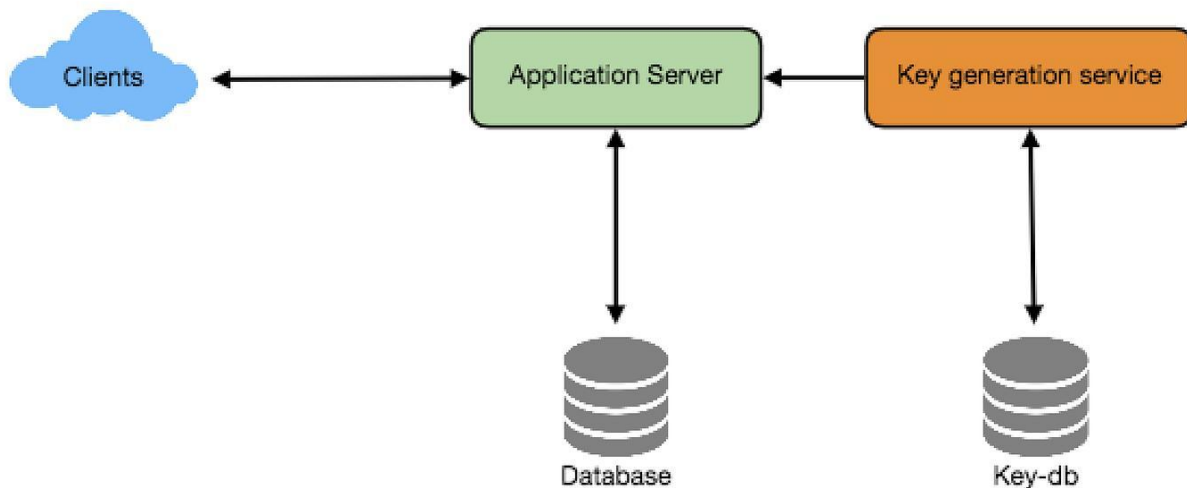
Best design:

**Hybrid Fan-out (Write + Read) + Feed Service + Social Graph + Ranking Engine + Kafka + Redis + NoSQL + Object Storage + CDN + Multi-region HA**

This is a strong **Meta / X / FAANG / Distributed Architect** answer.

## Design 6 → URL Shortener (TinyURL / Bitly)





7

This is one of the most common **FAANG / Distributed Systems / Backend / Architect** design questions.

This answers:

**“How do you design a URL shortener like TinyURL / Bitly handling billions of URLs and massive redirects?”**

We cover:

- short URL generation
- unique ID strategy
- collision handling
- redirects
- analytics
- caching
- DB sharding
- hot URLs
- rate limiting
- abuse prevention
- HA/DR
- scale

---

**Q1. What are core requirements of URL Shortener?**

**Answer:**

### **Functional Requirements**

- create short URL
- redirect short → long URL
- custom alias
- expiration support
- analytics (clicks, geo, referrer)
- QR support (optional)
- delete/disable links
- spam/abuse checks

### **Non-Functional Requirements**

- billions of URLs
- ultra-low redirect latency (<50–100ms)
- high read-heavy scale
- uniqueness
- HA
- fault tolerance
- geo-distribution
- high availability

---

## **1. Capacity Estimation**

Assume:

- 10B+ URLs stored
- 100M new URLs/day
- 5–10B redirects/day
- 95% read-heavy traffic

Challenges:

- uniqueness
- hot URLs

- redirect speed
- cache pressure
- analytics scale

Huge FAANG starter topic.

---

## 2. High-Level Architecture

### Q2. Core services?

**Answer:**

Usually:

- API Gateway
  - Shortening Service
  - Redirect Service
  - ID Generation Service
  - Analytics Service
  - Abuse Detection Service
  - Cache Layer
  - DB Layer
- 

## 3. Short URL Generation (Critical)

### Q3. How generate short URL?

**Answer:**

Common flow:

Long URL → generate unique ID → encode (Base62) → short URL

Example:

ID = 125634

Base62 = aZ91x

Huge interview favorite.

---

### Q4. Why Base62?

**Answer:**

Uses:

- a-z
- A-Z
- 0-9

Benefits:

- compact
  - URL-safe
  - short readable links
- 

**Q5. What are common ID generation strategies?**

**Answer:**

Usually:

- auto-increment + Base62
- Snowflake-like distributed IDs
- random hash
- pre-generated pool

Must know.

---

**Q6. Best ID strategy at scale?**

**Answer:**

Usually distributed unique ID (Snowflake-like).

Why:

- no central bottleneck
  - scalable
  - unique
- 

#### **4. Collision Handling**

**Q7. What is collision?**

**Answer:**



Two URLs generating same short key.

Huge topic.

---

#### **Q8. How avoid collisions?**

**Answer:**

Use:

- unique DB constraint
  - distributed ID generation
  - retry generation
  - deterministic mapping rules
- 

#### **Q9. Can random hash alone be risky?**

**Answer:**

Yes. Collision probability grows.

Common trap.

---

### **5. Shorten Flow**

#### **Q10. End-to-end shorten URL flow?**

**Answer:**

Step 1 → User submits long URL

Step 2 → Validate URL

Step 3 → Spam/abuse check

Step 4 → Generate ID

Step 5 → Encode Base62

Step 6 → Store mapping

Step 7 → Return short URL

---

### **6. Redirect Flow (Critical)**

#### **Q11. Redirect flow?**

**Answer:**

Step 1 → User hits short URL  
Step 2 → Redirect service receives  
Step 3 → Cache lookup  
Step 4 → DB lookup if miss  
Step 5 → Return long URL  
Step 6 → HTTP 301/302 redirect  
Step 7 → Publish analytics event

Huge architect question.

---

#### **Q12. 301 vs 302 redirect?**

**Answer:**

| 301 | 302 |
|-----|-----|
|-----|-----|

|           |           |
|-----------|-----------|
| Permanent | Temporary |
|-----------|-----------|

Often URL shorteners may choose based on use case.

---

#### **7. DB Design**

##### **Q13. Main data model?**

**Answer:**

Usually:

##### **URL Mapping**

- shortCode
- longUrl
- createdBy
- createdAt
- expiry
- status

##### **Analytics**

- clickId
- shortCode
- geo
- device

- referrer
  - timestamp
- 

#### **Q14. Which DB?**

**Answer:**

Usually:

- NoSQL / KV / partitioned store → URL mapping
  - Redis → hot redirect cache
  - analytics store → click events
  - object/cold storage → long retention logs
- 

#### **Q15. Why KV/NoSQL good here?**

**Answer:**

Mostly:

shortCode → longUrl lookup

Simple read-heavy key access.

---

#### **8. Caching (Very Important)**

##### **Q16. Why cache critical?**

**Answer:**

Redirects are extremely read-heavy.

Huge performance topic.

---

##### **Q17. What to cache?**

**Answer:**

Usually:

- hot short URLs
- redirect targets
- metadata

Redis/CDN edge.

---

#### **Q18. Hot URL problem?**

**Answer:**

One viral URL gets massive traffic.

Huge FAANG hotspot.

---

#### **Q19. How handle hot URLs?**

**Answer:**

Use:

- Redis cluster
  - CDN edge cache
  - replica scaling
  - partition spread
- 

#### **9. Sharding / Scale**

#### **Q20. How scale to 10B+ URLs?**

**Answer:**

Use:

- hash-based sharding
  - region sharding
  - read replicas
  - archive cold data
- 

#### **Q21. What gets sharded?**

**Answer:**

Usually:

- URL mapping
- analytics data

- click logs
- 

## 10. Analytics

### Q22. Why analytics separate?

**Answer:**

Clicks are huge write volume.

Do async processing.

---

### Q23. What analytics captured?

**Answer:**

- click count
  - geo
  - browser
  - device
  - referrer
  - timestamp
- 

### Q24. Why async analytics?

**Answer:**

Redirect path must stay fast.

Never block redirect.

Critical architect answer.

---

## 11. Security / Abuse

### Q25. Abuse risks?

**Answer:**

- phishing
- malware
- spam

- brute-force enumeration
- malicious redirects

Very common.

---

#### **Q26. Security controls?**

**Answer:**

Need:

- rate limiting
  - URL validation
  - abuse detection
  - blacklist
  - auth (for admin/custom alias)
  - WAF
- 

#### **12. Expiry / Deletion**

##### **Q27. How support expiring links?**

**Answer:**

Store expiry timestamp.

Redirect service checks validity.

---

##### **Q28. Soft delete vs hard delete?**

**Answer:**

Usually soft delete first for audit/control.

---

#### **13. HA / DR**

##### **Q29. HA design?**

**Answer:**

Use:

- stateless redirect nodes

- global LB
- cache clusters
- DB replication
- failover

---

### Q30. DR?

**Answer:**

- backup
  - geo replication
  - restore testing
  - archive strategy
- 

### 14. Bottlenecks

#### Q31. Common hotspots?

**Answer:**

- hot URLs
  - ID generation
  - cache hot keys
  - analytics spikes
  - DB shard imbalance
- 

### 15. Trade-offs

#### Q32. Pre-generate IDs vs On-demand IDs?

**Answer:**

| <b>Pre-generated</b> | <b>On-demand</b> |
|----------------------|------------------|
|----------------------|------------------|

|                   |                       |
|-------------------|-----------------------|
| Faster issue path | Simpler dynamic logic |
|-------------------|-----------------------|

|                 |                 |
|-----------------|-----------------|
| Pool management | Can add latency |
|-----------------|-----------------|

Architect trade-off.

---

**Q33. Strong consistency vs eventual consistency?**

**Answer:**

Strong:

- URL uniqueness
- URL mapping

Eventual:

- analytics
  - reports
  - dashboards
- 

**16. Anti-Patterns****Q34. Dangerous anti-patterns?**

**Answer:**

- DB hit on every redirect
  - random collisions ignored
  - analytics in sync path
  - no cache
  - sequential single-node ID generator
  - no abuse control
- 

**17. FAANG Trap Questions****Q35. Can Redis be source of truth?**

**Answer:**

No.

---

**Q36. Can analytics slow redirect path?**

**Answer:**

Should not.



Async only.

---

**Q37. Can relational DB alone scale?**

**Answer:**

Possible small-scale. Large-scale often distributed KV/NoSQL.

---

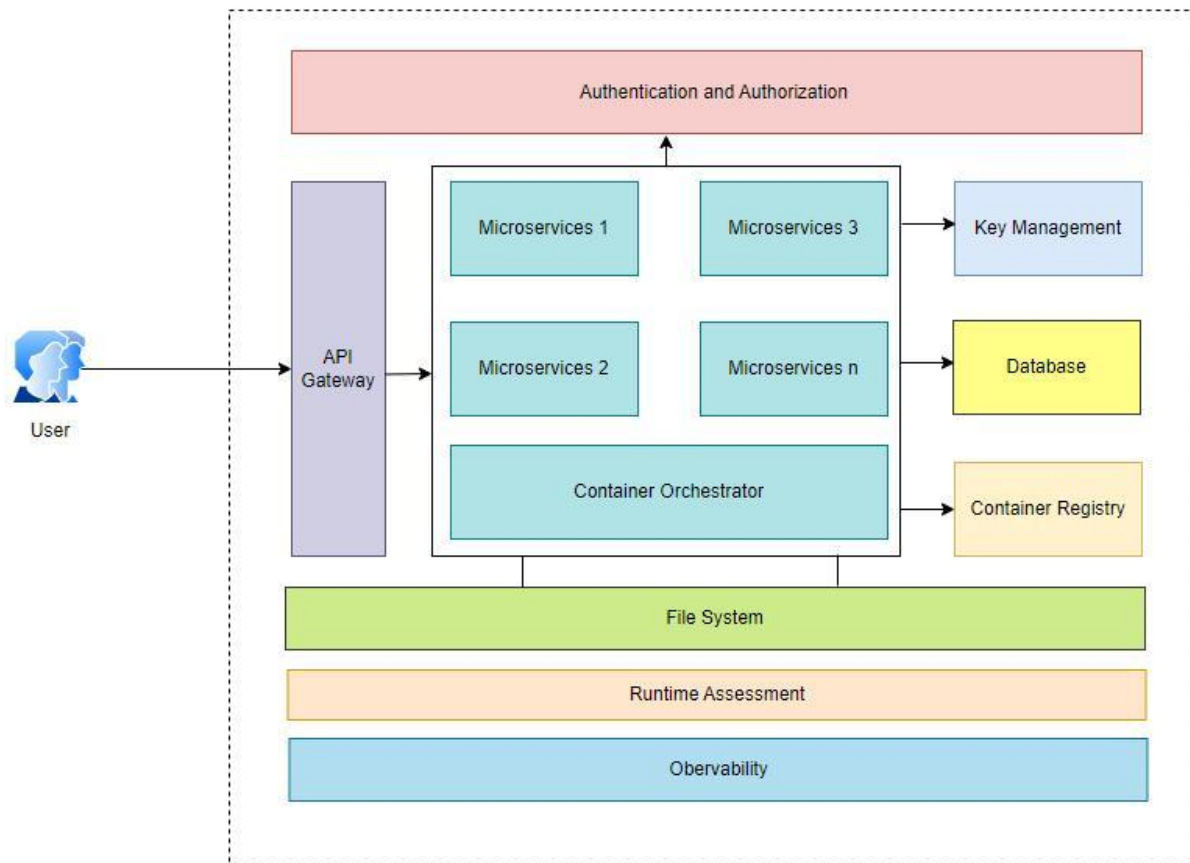
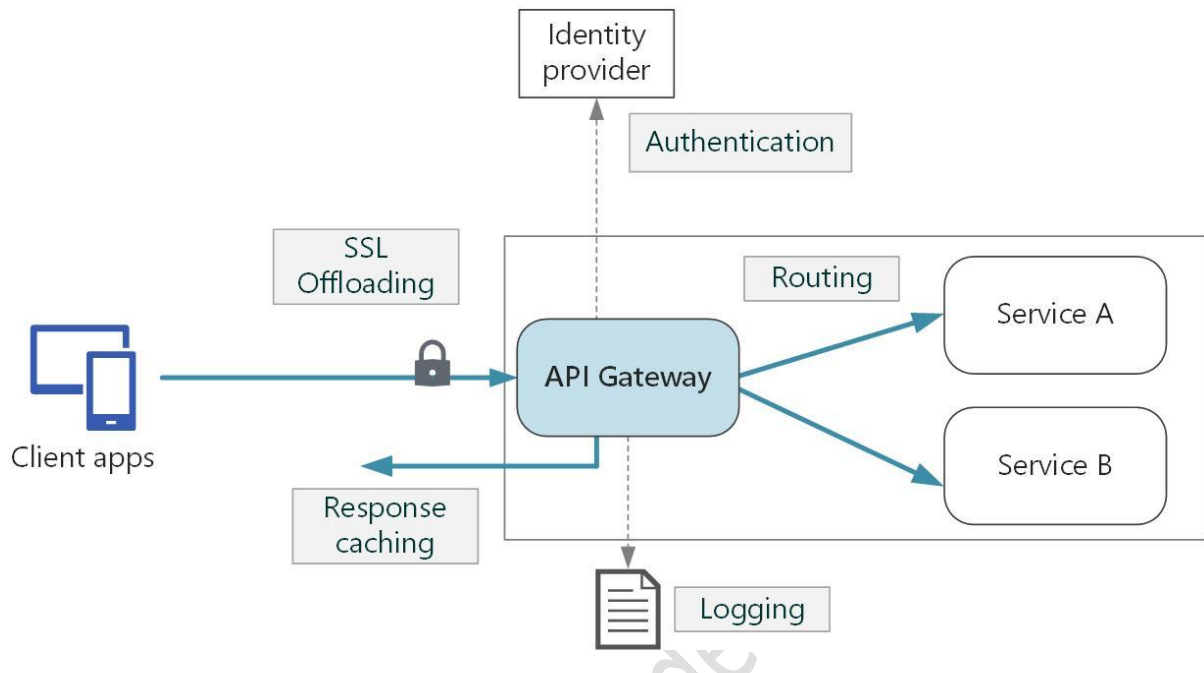
**Final Architect Summary**

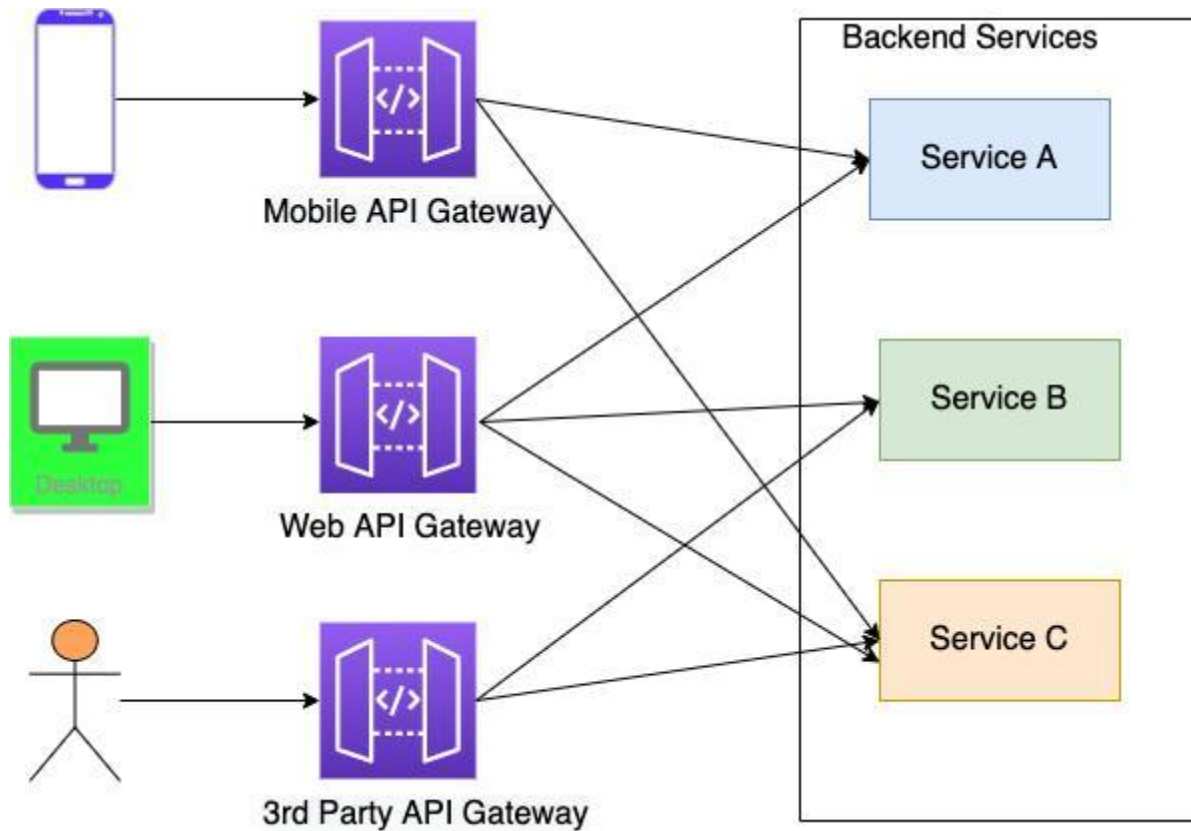
Best design:

**API Gateway + Shortening Service + Distributed ID Generator + Base62 Encoding + Redirect Service + Redis + NoSQL/KV Store + Async Analytics + Abuse Detection + Global LB + HA/DR**

This is a strong **Bitly / TinyURL / FAANG / Distributed Backend Architect** answer.

## Design 7 → Global API Gateway





7

This answers:

**“How do you design a Global API Gateway that handles millions of requests/sec securely across regions and microservices?”**

Covers:

- auth
- routing
- load balancing
- rate limiting
- throttling
- WAF
- mTLS
- circuit breaker
- service discovery
- caching

- observability
  - zero trust
  - HA/DR
  - geo routing
- 

### **Q1. What are core requirements of Global API Gateway?**

**Answer:**

#### **Functional Requirements**

- route API requests
- authentication
- authorization
- SSL/TLS termination
- rate limiting
- throttling
- request validation
- API versioning
- caching
- service discovery
- traffic shaping
- observability
- retries
- failover routing

#### **Non-Functional Requirements**

- millions req/sec
- global availability
- low latency
- fault tolerance
- security
- zero trust

- HA
  - multi-region
  - horizontal scale
- 

## 1. Capacity Estimation

Assume:

- 100M API users
- 5M concurrent sessions
- 1M+ req/sec peak
- multi-region traffic

Challenges:

- auth bottleneck
- hot APIs
- DDoS
- cascading failures
- service discovery latency

Huge cloud architect topic.

---

## 2. High-Level Architecture

### Q2. Core components?

Answer:

Usually:

- Global DNS / Geo Router
- Edge Load Balancer
- WAF / DDoS Layer
- API Gateway Cluster
- Auth Service
- Rate Limit Engine
- Service Discovery

- Cache Layer
  - Observability Stack
  - Policy Engine
  - Circuit Breaker Layer
- 

### 3. Request Flow

#### Q3. End-to-end request flow?

**Answer:**

Step 1 → Client sends API request  
Step 2 → Global DNS routes nearest region  
Step 3 → WAF validates  
Step 4 → TLS termination  
Step 5 → Auth check  
Step 6 → Rate limit check  
Step 7 → Request validation  
Step 8 → Route to microservice  
Step 9 → Response  
Step 10 → Metrics/logs/traces

---

### 4. Global Routing

#### Q4. Why Global DNS / Geo routing?

**Answer:**

Routes users to nearest healthy region.

Benefits:

- lower latency
- HA
- failover
- geo optimization

Very common architect topic.

---

#### Q5. Active-active vs active-passive?

**Answer:**

### Active-Active

### Active-Passive

Multiple regions serve traffic Backup region waits

Higher availability

Simpler failover

Often active-active for global APIs.

---

## 5. API Gateway Core Role

### Q6. What does API Gateway do?

#### Answer:

Central entry point for APIs.

Handles:

- auth
- routing
- policies
- throttling
- caching
- logging
- transformation
- security

Huge microservices topic.

---

### Q7. Why API Gateway needed?

#### Answer:

Without it:

- duplicated auth
  - inconsistent policies
  - harder observability
  - tight client-service coupling
-

## 6. Authentication / Authorization

### Q8. Auth at gateway?

**Answer:**

Usually:

- JWT validation
  - OAuth2
  - API keys
  - token introspection
  - RBAC policy
- 

### Q9. Why auth at edge?

**Answer:**

Reject unauthorized traffic early.

Protects downstream services.

---

## 7. Zero Trust / mTLS

### Q10. What is Zero Trust here?

**Answer:**

Never automatically trust request/service.

Always verify:

- identity
- policy
- transport
- authorization

Huge enterprise topic.

---

### Q11. Why mTLS between services?

**Answer:**

Mutual authentication between internal services.



Very common cloud architect topic.

---

## 8. Rate Limiting / Throttling

### Q12. Why rate limiting?

**Answer:**

Protect:

- abuse
  - DDoS
  - noisy clients
  - backend overload
- 

### Q13. Common rate limiting algorithms?

**Answer:**

- token bucket
  - leaky bucket
  - sliding window
  - fixed window
- 

Must know.

---

### Q14. Throttling vs Rate Limiting?

**Answer:**

| Throttling                  | Rate Limiting    |
|-----------------------------|------------------|
| Reduce/control traffic pace | Hard request cap |

---

## 9. Service Discovery

### Q15. Why service discovery?

**Answer:**

Dynamic routing to healthy service instances.

Huge microservices topic.

---

#### **Q16. Common approaches?**

**Answer:**

- DNS-based
  - registry-based
  - mesh integration
- 

### **10. Circuit Breaker / Resilience**

#### **Q17. Why circuit breaker in gateway?**

**Answer:**

Stops repeated calls to failing downstream service.

Prevents cascading failure.

Huge reliability topic.

---

#### **Q18. Retry at gateway?**

**Answer:**

Only safe/idempotent cases.

Never blindly retry unsafe requests.

Critical trap.

---

### **11. Request Transformation**

#### **Q19. Why transform requests?**

**Answer:**

Useful for:

- version translation
- header enrichment
- protocol adaptation
- normalization

---

## 12. Caching

### Q20. What can gateway cache?

Answer:

- GET responses
- metadata
- auth policy
- static configs

---

### Q21. Risks of gateway caching?

Answer:

- stale data
- auth leakage
- invalidation complexity

---

## 13. Observability

### Q22. Why observability critical?

Answer:

Gateway sees all traffic.

Great place for:

- logs
- metrics
- tracing
- latency
- error tracking

---

### Q23. Common metrics?

Answer:

- latency

- RPS
  - 4xx/5xx
  - saturation
  - auth failures
  - rate-limit hits
- 

## 14. Security

### Q24. Security controls?

Answer:

Need:

- WAF
- TLS
- mTLS
- JWT validation
- DDoS protection
- schema validation
- RBAC
- secret management
- IP filtering

Huge architect answer.

---

## 15. Multi-Region HA / DR

### Q25. HA design?

Answer:

Use:

- active-active gateway clusters
- geo routing
- stateless nodes
- replicated config

- failover
- 

#### **Q26. DR strategy?**

**Answer:**

- cross-region failover
  - config backup
  - health-based DNS reroute
  - tested recovery
- 

#### **16. Scale**

#### **Q27. How scale to millions req/sec?**

**Answer:**

Use:

- stateless gateway nodes
  - horizontal autoscaling
  - edge caching
  - async logging
  - config distribution
  - regional clusters
- 

#### **Q28. Hot API problem?**

**Answer:**

One endpoint gets massive traffic.

Mitigate:

- cache
  - shard backend
  - isolate rate limits
  - edge CDN
-

## 17. Bottlenecks

### Q29. Common hotspots?

Answer:

- auth validation
  - policy engine
  - config reload
  - TLS termination
  - rate-limit store
  - hot APIs
- 

## 18. Trade-offs

### Q30. Gateway vs Service Mesh?

Answer:

| API Gateway              | Service Mesh             |
|--------------------------|--------------------------|
| North-South traffic      | East-West traffic        |
| Client-facing            | Internal service traffic |
| Huge architect favorite. |                          |

### Q31. Centralized gateway risk?

Answer:

Can become bottleneck or SPOF if poorly designed.

---

## 19. Anti-Patterns

### Q32. Dangerous anti-patterns?

Answer:

- stateful gateway nodes
- auth inside every client only
- no rate limiting

- retry unsafe POST blindly
  - sync-heavy logging
  - single-region gateway
- 

## 20. FAANG / Architect Trap Questions

### Q33. Can gateway become SPOF?

**Answer:**

Yes, if not horizontally scaled.

---

### Q34. Should gateway store business logic?

**Answer:**

Usually no. Keep edge concerns only.

Huge trap.

---

### Q35. Is API Gateway same as load balancer?

**Answer:**

No.

LB = traffic distribution

Gateway = policy + routing + edge intelligence

Must know.

---

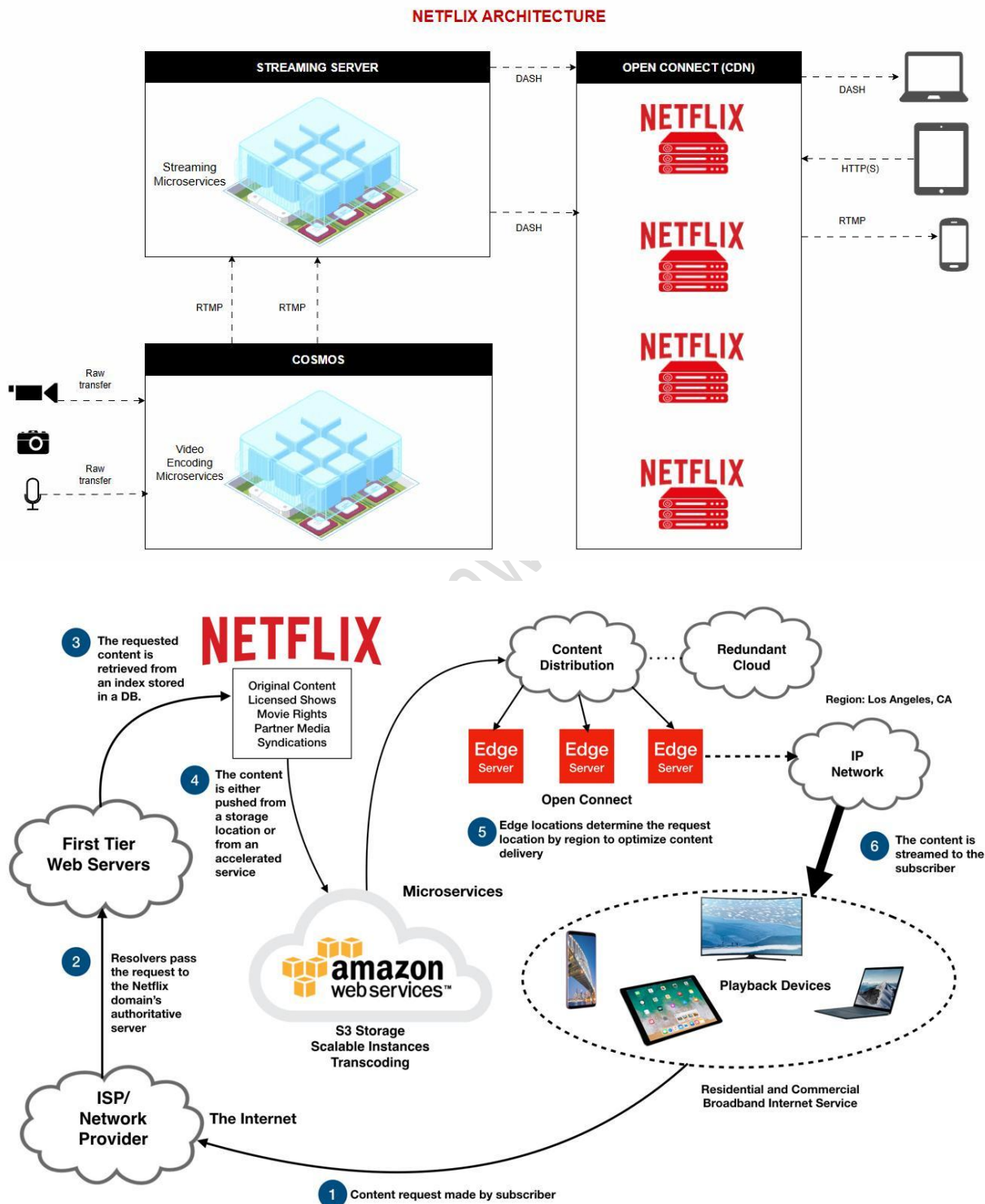
## Final Architect Summary

Best design:

**Global DNS + WAF + Stateless API Gateway Cluster + Auth + Rate Limit Engine + Service Discovery + Circuit Breaker + Cache + Observability + Zero Trust + mTLS + Active-Active Multi-region HA**

This is a strong **Cloud / FAANG / Enterprise / Accenture Architect** answer.

## Design 8 → Netflix / Video Streaming Platform (Azure + AWS)





This answers:

**“How do you design a Netflix-like video streaming platform serving millions of concurrent users globally?”**

Covers:

- video upload
  - transcoding
  - metadata
  - recommendation
  - streaming
  - CDN
  - adaptive bitrate
  - geo-distribution
  - DRM
  - analytics
  - HA/DR
  - Azure + AWS mapping
- 

**Q1. What are core requirements of Video Streaming Platform?**

**Answer:**

**Functional Requirements**

- upload video
- encode/transcode
- stream video
- pause/resume
- adaptive bitrate
- subtitles
- recommendations
- search
- watch history

- continue watching
- likes/ratings
- analytics
- DRM/content protection

### **Non-Functional Requirements**

- millions concurrent streams
  - ultra-low latency startup
  - high throughput
  - geo-distribution
  - HA
  - fault tolerance
  - CDN heavy
  - secure streaming
  - cost optimization
- 

### **1. Capacity Estimation**

Assume:

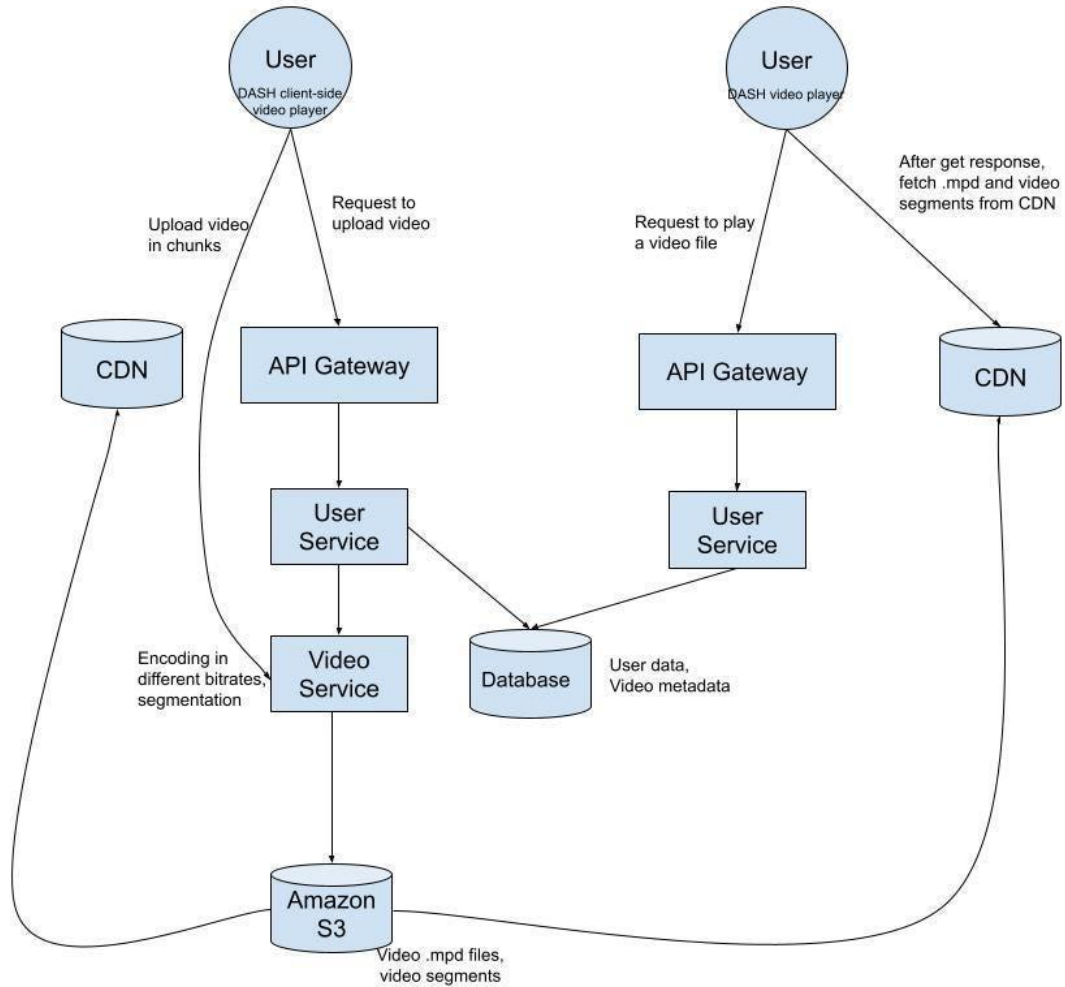
- 300M users
- 100M DAU
- 20M concurrent viewers
- petabytes of media
- TB/PB daily transfer

Challenges:

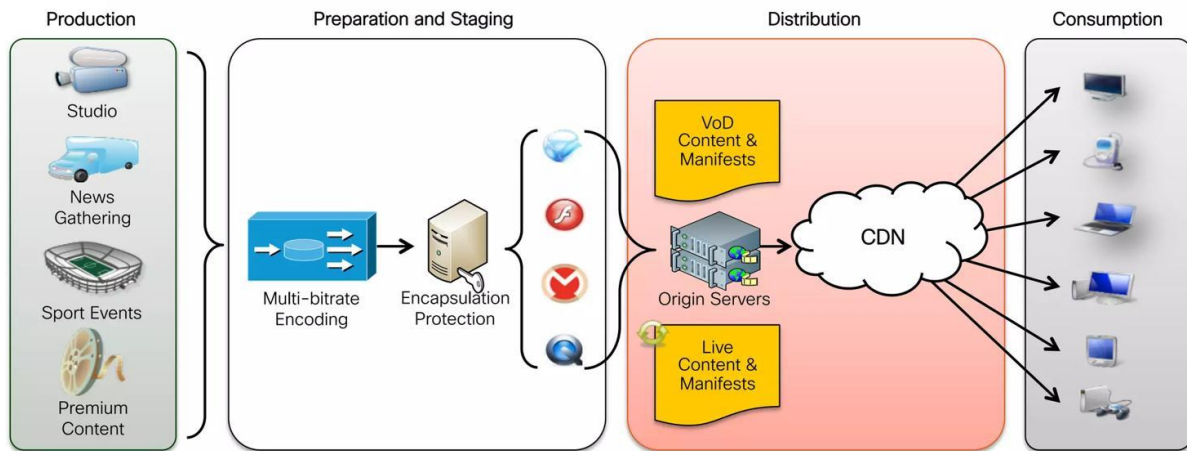
- hot videos
  - bandwidth
  - transcoding cost
  - geo delivery
  - storage lifecycle
-

## 2. High-Level Architecture

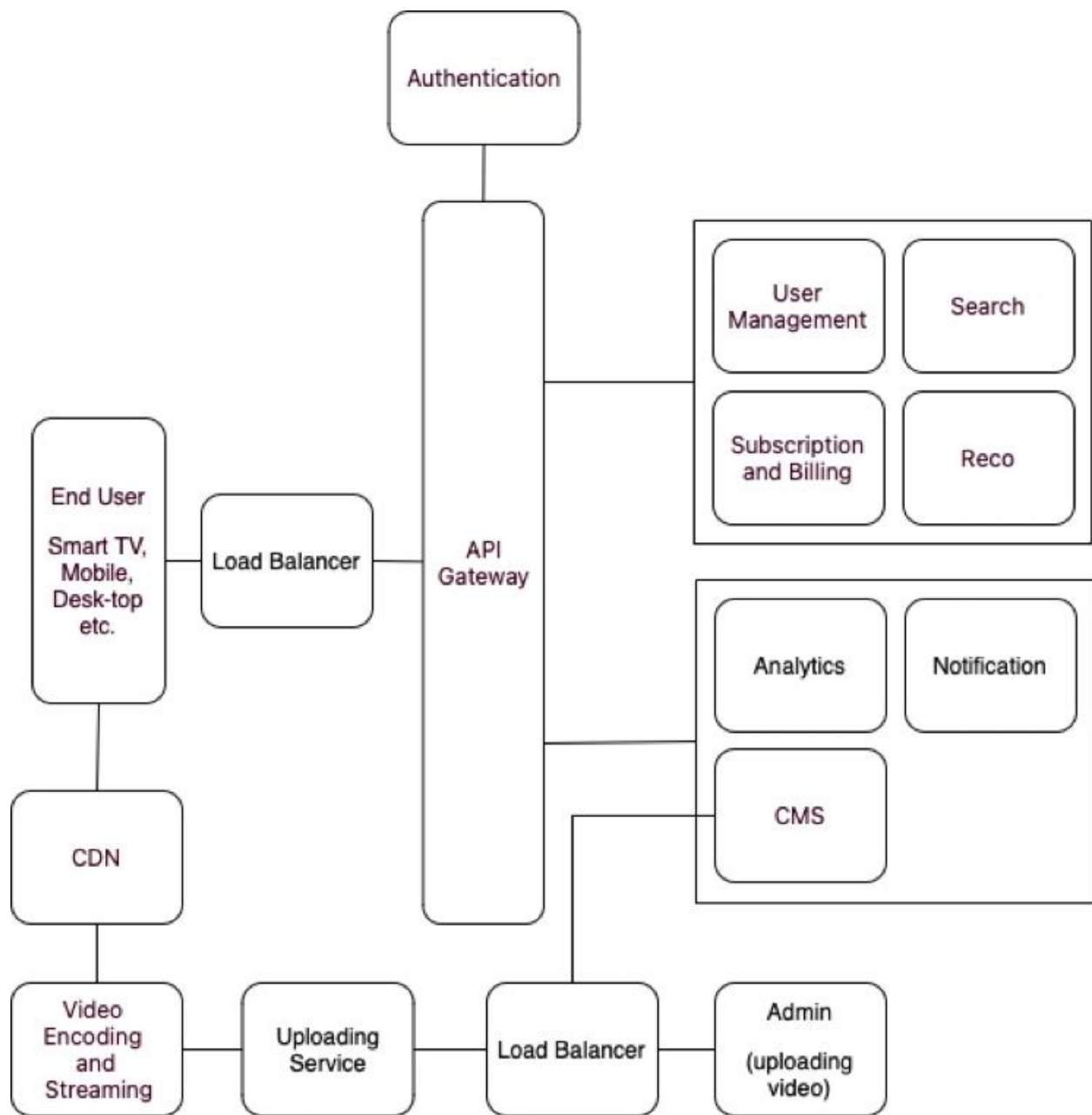
A diagram of a video streaming system



## End-to-End Over-the-Top Adaptive Streaming Delivery



46



8

**Q2. Core services?**

**Answer:**

Usually:

- API Gateway
- Upload Service
- Transcoding Service

- Metadata Service
  - Streaming Manifest Service
  - CDN
  - Recommendation Service
  - Search Service
  - User Profile Service
  - Analytics Service
  - DRM/Auth Service
- 

### **3. Video Upload Flow**

#### **Q3. Upload flow?**

**Answer:**

Step 1 → Creator uploads video  
Step 2 → Upload Service validates  
Step 3 → Store raw video  
Step 4 → Event published  
Step 5 → Transcoding pipeline  
Step 6 → Multi-resolution output  
Step 7 → Metadata update  
Step 8 → CDN distribution

Huge architect flow.

---

### **4. Why transcoding?**

#### **Q4. Why Transcoding Service?**

**Answer:**

Converts source video into multiple formats/resolutions.

Examples:

- 240p
- 480p
- 720p
- 1080p

- 4K

Supports device compatibility.

---

## 5. Adaptive Bitrate Streaming (Critical)

### Q5. What is Adaptive Bitrate (ABR)?

**Answer:**

Player dynamically switches quality based on bandwidth.

Huge streaming concept.

---

### Q6. Why ABR important?

**Answer:**

Improves:

- playback quality
  - reduced buffering
  - user experience
- 

## 6. Storage Design

### Q7. What storage layers?

**Answer:**

Usually:

- Raw upload storage
  - Encoded media storage
  - CDN edge cache
  - Cold archive
  - Metadata DB
- 

### Q8. Why object storage?

**Answer:**

Large immutable media files.

Scalable + cheaper than DB.

Huge cloud topic.

---

## 7. Streaming Path

### Q9. Playback flow?

**Answer:**

Step 1 → User opens video

Step 2 → Auth check

Step 3 → Manifest request

Step 4 → CDN edge lookup

Step 5 → Segment fetch

Step 6 → Adaptive bitrate switch

Step 7 → Analytics emitted

---

## 8. CDN (Very Critical)

### Q10. Why CDN critical?

**Answer:**

Moves content closer to users.

Reduces:

- latency
- origin load
- bandwidth bottlenecks

Huge FAANG topic.

---

### Q11. Hot video problem?

**Answer:**

Viral video causes huge traffic.

CDN absorbs load.

---

## 9. Metadata / Search

### Q12. Metadata service stores?



**Answer:**

- title
  - category
  - cast
  - language
  - duration
  - subtitles
  - tags
- 

**Q13. Search consistency?**

**Answer:**

Usually eventual consistency.

---

**10. Recommendation Engine**

**Q14. Recommendation inputs?**

**Answer:**

- watch history
- likes
- watch time
- similarity
- trending
- collaborative filtering

Huge Netflix topic.

---

**Q15. Why async recommendation pipeline?**

**Answer:**

ML/ranking can be expensive.

Do offline + near-real-time updates.

---

## 11. Caching

### Q16. What to cache?

Answer:

- metadata
  - recommendations
  - auth tokens
  - manifests
  - hot thumbnails
  - recently watched
- 

## 12. Security / DRM

### Q17. Security concerns?

Answer:

- content piracy
  - token abuse
  - unauthorized playback
  - credential sharing
  - geo restrictions
- 

### Q18. DRM?

Answer:

Digital Rights Management.

Protects content from unauthorized playback/copy.

Huge architect topic.

---

## 13. HA / DR

### Q19. HA design?

Answer:

Use:

- multi-region
  - CDN
  - stateless APIs
  - replicated metadata DB
  - failover
  - queue durability
- 

#### **Q20. DR strategy?**

**Answer:**

- cross-region media replication
  - metadata backup
  - restore testing
  - geo failover
- 

#### **14. Scale**

##### **Q21. How scale to millions of streams?**

**Answer:**

Use:

- CDN
  - edge cache
  - autoscaling APIs
  - async pipelines
  - object storage
  - partitioned analytics
- 

#### **15. Bottlenecks**

##### **Q22. Common hotspots?**

**Answer:**

- transcoding farm

- origin storage
- metadata DB
- recommendation latency
- CDN miss
- viral video

---

## 16. Azure Solution Mapping (.NET-focused)

### Q23. Azure services mapping?

**Answer:**

| Need           | Azure                                 |
|----------------|---------------------------------------|
| API Gateway    | API Management / Front Door           |
| Global LB      | Front Door / Traffic Manager          |
| Upload APIs    | App Service / AKS                     |
| Transcoding    | Azure Media Services / AKS jobs       |
| Object Storage | Blob Storage                          |
| CDN            | Azure CDN                             |
| Event Bus      | Event Grid / Service Bus / Event Hubs |
| Cache          | Azure Cache for Redis                 |
| Metadata DB    | Azure SQL / Cosmos DB                 |
| Search         | Azure AI Search                       |
| Analytics      | Event Hubs + Data Lake + Synapse      |
| Auth           | Entra ID / B2C                        |
| Secrets        | Key Vault                             |
| Monitoring     | Azure Monitor / App Insights          |
| Containers     | AKS                                   |

Very strong for your Azure prep.

---

## 17. AWS Solution Mapping (.NET-focused)

### Q24. AWS services mapping?

Answer:

| Need           | AWS                         |
|----------------|-----------------------------|
| API Gateway    | API Gateway                 |
| Global LB      | Route 53 + CloudFront + ALB |
| Upload APIs    | ECS / EKS / EC2             |
| Transcoding    | MediaConvert / EKS jobs     |
| Object Storage | S3                          |
| CDN            | CloudFront                  |
| Event Bus      | SNS / SQS / Kinesis         |
| Cache          | ElastiCache Redis           |
| Metadata DB    | RDS / DynamoDB              |
| Search         | OpenSearch                  |
| Analytics      | Kinesis + S3 + Athena       |
| Auth           | Cognito                     |
| Secrets        | Secrets Manager             |
| Monitoring     | CloudWatch / X-Ray          |
| Containers     | EKS                         |

Very common cloud architect mapping.

---

## 18. Trade-offs

### Q25. Azure vs AWS for this?

Answer:

| Azure                         | AWS                             |
|-------------------------------|---------------------------------|
| Strong enterprise integration | Very mature streaming ecosystem |

## Azure

Good .NET ecosystem

## AWS

Strong media/CDN tooling

---

### 19. Anti-Patterns

#### Q26. Dangerous anti-patterns?

Answer:

- serving video from DB
  - no CDN
  - sync transcoding on request
  - no DRM
  - single-region storage
  - recommendation in sync path
- 

### 20. FAANG Trap Questions

#### Q27. Can DB store videos?

Answer:

No. Object storage preferred.

---

#### Q28. Can recommendation be strongly consistent?

Answer:

Usually no.

---

#### Q29. Can CDN replace origin?

Answer:

No.

---

### Final Architect Summary

Best design:

**API Gateway + Upload Service + Object Storage + Async Transcoding + Manifest Service + CDN + Recommendation Engine + Metadata DB + Search + DRM + Multi-region HA**

**Azure:**

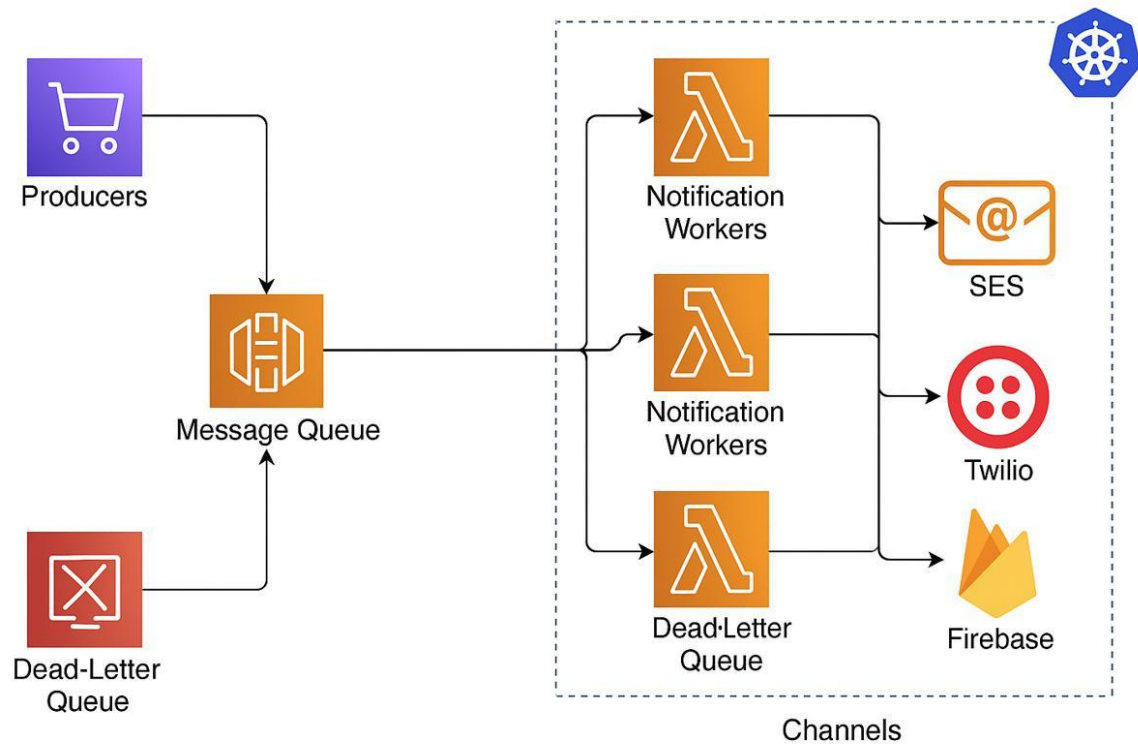
Front Door + APIM + Blob + Media Services + Redis + Cosmos/Azure SQL + Event Hubs + AKS

**AWS:**

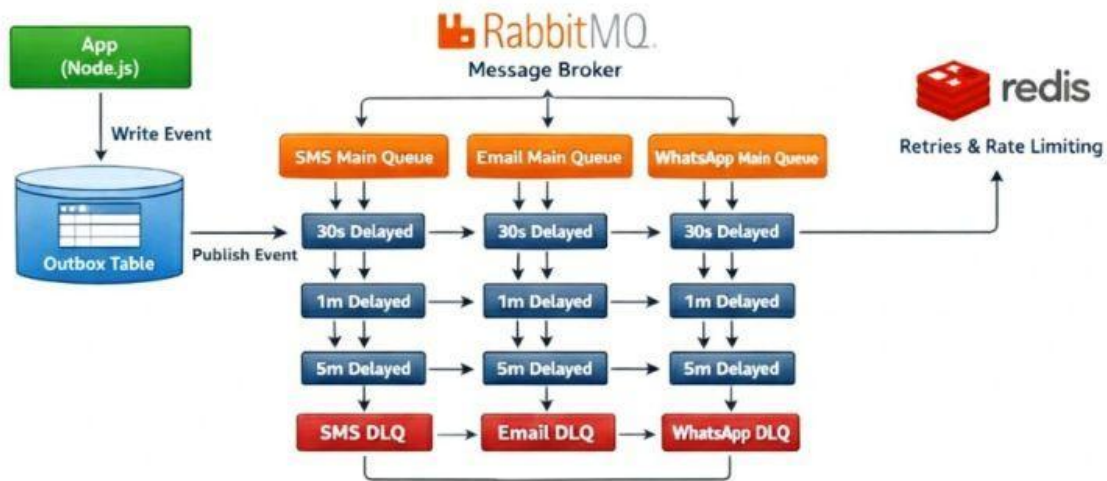
CloudFront + API Gateway + S3 + MediaConvert + ElastiCache + RDS/Dynamo + Kinesis + EKS

This is a strong **Netflix / Cloud Architect / FAANG / Accenture** answer.

## Design 9 → Notification System (Email / SMS / Push)







This answers:

**“How do you design a large-scale Notification Platform handling billions of email/SMS/push events reliably?”**

Covers:

- multi-channel delivery
- event-driven notifications
- retries
- provider failover
- DLQ
- idempotency
- user preference
- template engine
- rate limiting
- HA/DR
- Azure + AWS

---

**Q1. What are core requirements of Notification System?**

**Answer:**

**Functional Requirements**

- send email
- send SMS
- send push notifications
- scheduled notifications
- bulk notifications
- retries
- delivery status
- user preferences
- templates
- localization
- unsubscribe/opt-out
- analytics
- provider failover

**Non-Functional Requirements**

- billions of events
- reliable delivery
- fault tolerance
- HA
- low latency
- high throughput
- idempotency
- observability
- horizontal scale

---

**1. Capacity Estimation**

Assume:

- 500M users
- 2B notifications/day
- burst traffic (OTP, marketing, alerts)

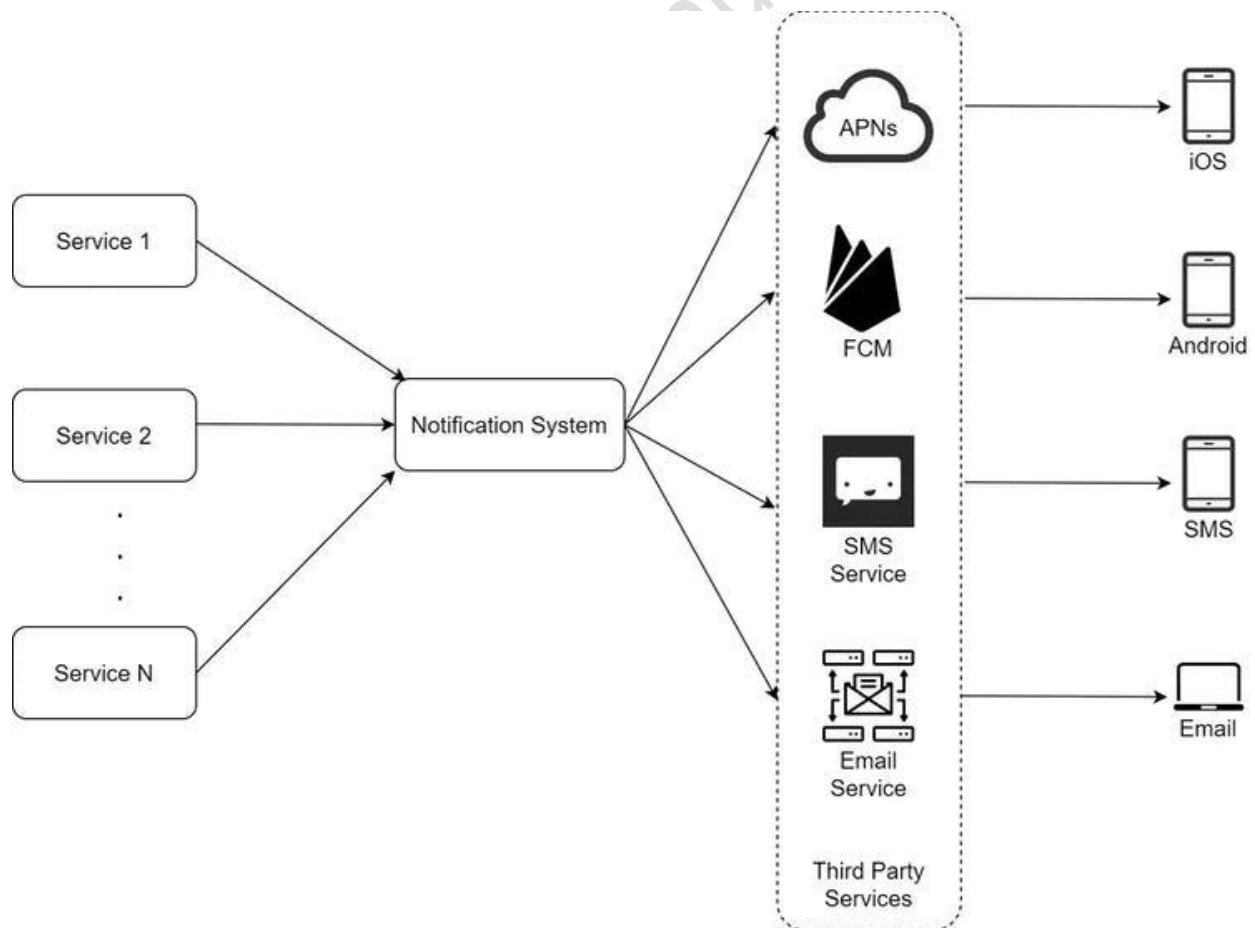
Challenges:

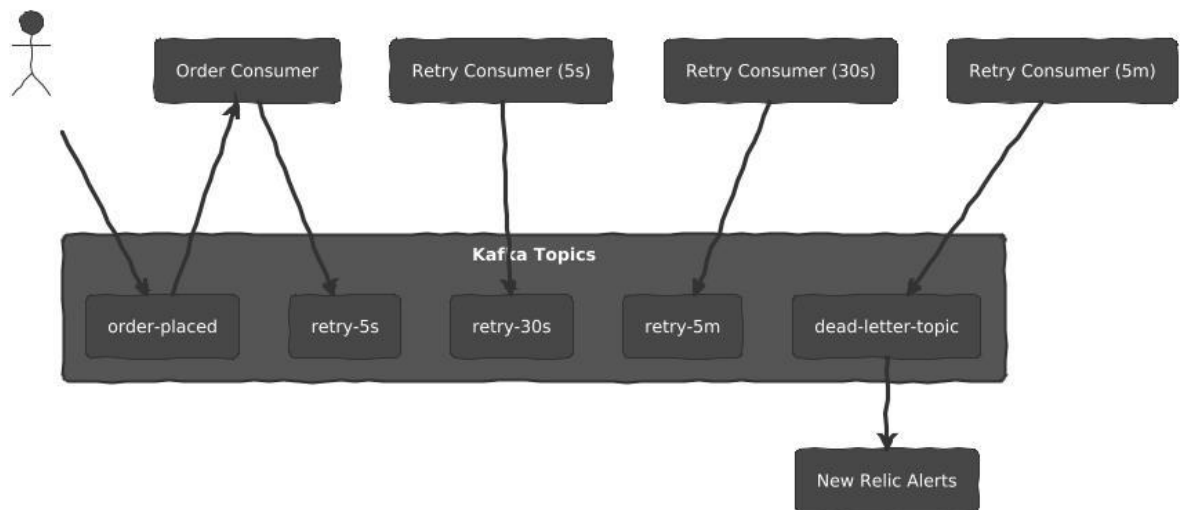
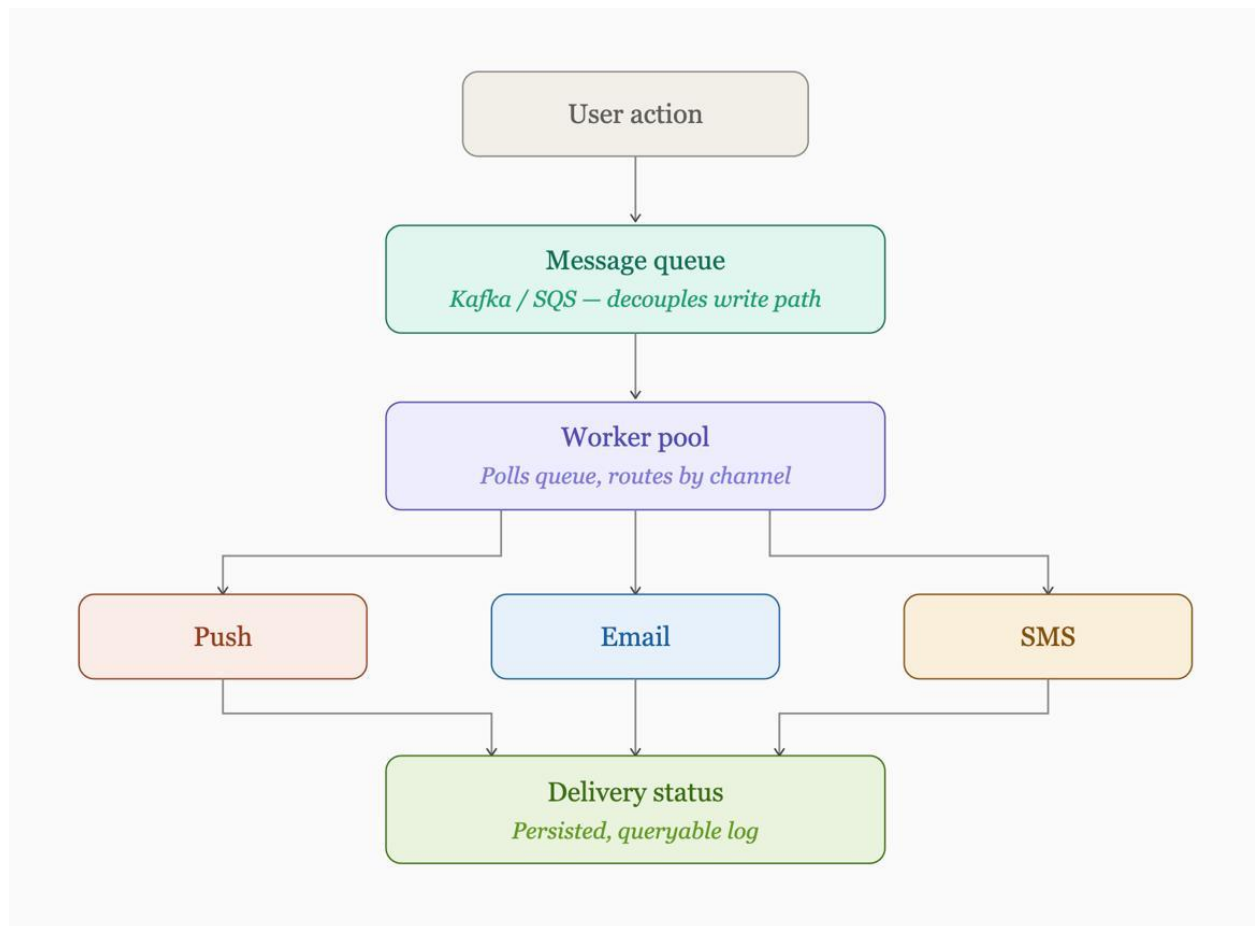
- retries
- provider failure
- duplicate sends
- rate limits
- fan-out spikes

Huge distributed topic.

---

## 2. High-Level Architecture





8

**Q2. Core services?**

**Answer:**

Usually:

- API Gateway
  - Notification API
  - Preference Service
  - Template Service
  - Orchestrator
  - Queue/Event Bus
  - Email Worker
  - SMS Worker
  - Push Worker
  - Retry Service
  - DLQ
  - Provider Adapter Layer
  - Analytics Service
- 

### **3. Flow**

#### **Q3. End-to-end notification flow?**

**Answer:**

Step 1 → Event triggered (order/payment/login etc.)  
Step 2 → Notification API receives  
Step 3 → Preference validation  
Step 4 → Template resolution  
Step 5 → Idempotency check  
Step 6 → Queue publish  
Step 7 → Worker consumes  
Step 8 → Provider send  
Step 9 → Status update  
Step 10 → Retry/DLQ if fail  
Step 11 → Analytics event

---

### **4. Why event-driven?**

#### **Q4. Why Queue/Event Bus?**

**Answer:**

Decouples producers and delivery workers.

Benefits:

- async processing
- scaling
- retries
- replay
- durability

Huge architect topic.

---

## 5. Orchestrator

### Q5. What is Notification Orchestrator?

Answer:

Central workflow coordinator.

Handles:

- channel selection
  - retries
  - fallback
  - routing
  - status flow
- 

## 6. Multi-Channel Delivery

### Q6. Why separate workers?

Answer:

Different delivery logic:

- Email
- SMS
- Push

Independent scale/failures.

---

### **Q7. Example channel routing?**

**Answer:**

OTP → SMS/Push

Marketing → Email/Push

Critical alert → SMS + Email + Push

Very common.

---

## **7. Template Engine**

### **Q8. Why template service?**

**Answer:**

Manages:

- reusable templates
  - placeholders
  - localization
  - branding
- 

### **Q9. Example template fields?**

**Answer:**

- name
  - orderId
  - OTP
  - amount
  - language
- 

## **8. User Preference / Opt-out**

### **Q10. Why Preference Service?**

**Answer:**

Stores:

- allowed channels

- mute
- opt-out
- quiet hours
- marketing consent

Critical enterprise topic.

---

## 9. Idempotency

### Q11. Why idempotency important?

**Answer:**

Prevent duplicate notifications.

Example:

Queue retry should not send OTP twice.

Huge interview favorite.

---

### Q12. Common implementation?

**Answer:**

Use:

- message ID
  - dedupe key
  - status store
  - TTL cache
- 

## 10. Retry / DLQ (Very Important)

### Q13. Why retries?

**Answer:**

Temporary failures:

- provider timeout
- network issue
- throttling



---

**Q14. Retry strategy?**

**Answer:**

Use:

- exponential backoff
- bounded retries
- jitter

---

**Q15. What is DLQ?**

**Answer:**

Dead Letter Queue.

Stores failed messages after retry exhaustion.

Huge architect topic.

---

**Q16. Why DLQ critical?**

**Answer:**

Prevents silent message loss.

---

**11. Provider Failover****Q17. Why provider adapter layer?**

**Answer:**

Abstraction over:

- SendGrid / SES
- Twilio
- FCM / APNS

Avoid vendor lock.

---

**Q18. If provider down?**

**Answer:**

Fallback to secondary provider.

Critical HA design.

---

## **12. Rate Limiting / Bulk**

### **Q19. Why rate limiting?**

**Answer:**

Avoid:

- spam
  - provider quota breach
  - abuse
  - overload
- 

### **Q20. Bulk notification challenge?**

**Answer:**

Millions fan-out.

Need:

- batching
  - queues
  - throttling
  - partitioning
- 

## **13. DB Design**

### **Q21. Core data stores?**

**Answer:**

Usually:

#### **Notification Store**

- id
- userId
- channel

- payload
- state

#### Preference Store

- channel settings
- opt-out

#### Template Store

- localized templates

#### Analytics Store

- open/click/delivery
- 

#### Q22. Which DB?

Answer:

Usually:

- relational/NoSQL → metadata
  - Redis → idempotency/cache
  - event bus → delivery pipeline
  - analytics store → reporting
- 

#### 14. Push Notifications

##### Q23. Push path?

Answer:

Service → Push Worker → FCM/APNS → Device.

---

##### Q24. Offline device?

Answer:

Provider/device queue + retry strategy.

---

#### 15. Email Delivery

##### Q25. Email challenges?

**Answer:**

- spam reputation
- bounce
- DKIM/SPF
- rate caps
- blacklisting

Enterprise-level.

---

## **16. SMS Delivery**

**Q26. SMS challenges?**

**Answer:**

- carrier delay
  - region routing
  - cost
  - OTP latency
  - provider dependency
- 

## **17. Analytics**

**Q27. Why analytics?**

**Answer:**

Track:

- delivered
  - failed
  - opened
  - clicked
  - bounce
  - opt-out
- 

## **18. Caching**

## Q28. What to cache?

Answer:

- templates
  - preferences
  - idempotency keys
  - provider metadata
- 

## 19. HA / DR

### Q29. HA design?

Answer:

Use:

- multi-region workers
  - queue replication
  - provider failover
  - stateless services
  - DB replication
- 

### Q30. DR?

Answer:

- message replay
  - cross-region failover
  - backups
  - restore tests
- 

## 20. Scale

### Q31. How scale to billions/day?

Answer:

Use:

- partitioned queues

- worker pools
  - async delivery
  - batching
  - provider abstraction
  - distributed cache
- 

## **21. Bottlenecks**

### **Q32. Common hotspots?**

**Answer:**

- queue lag
  - provider throttling
  - bulk campaigns
  - OTP spikes
  - retry storms
- 

## **22. Security**

### **Q33. Security controls?**

**Answer:**

- auth
  - RBAC
  - encryption
  - PII masking
  - secret vault
  - rate limiting
  - audit trail
- 

## **23. Azure Mapping (.NET-focused)**

### **Q34. Azure services?**

**Answer:**

| Need                  | Azure                          |
|-----------------------|--------------------------------|
| API                   | App Service / AKS              |
| Queue                 | Service Bus / Event Hubs       |
| Push workflow         | Functions                      |
| Cache                 | Azure Redis                    |
| DB                    | Azure SQL / Cosmos DB          |
| Email/SMS integration | Logic Apps / provider adapters |
| Secrets               | Key Vault                      |
| Monitoring            | App Insights                   |
| Global routing        | Front Door                     |
| Containers            | AKS                            |

---

## 24. AWS Mapping (.NET-focused)

### Q35. AWS services?

**Answer:**

| Need           | AWS                     |
|----------------|-------------------------|
| API            | API Gateway / ECS / EKS |
| Queue          | SQS / SNS / Kinesis     |
| Push workflows | Lambda                  |
| Cache          | ElastiCache             |
| DB             | DynamoDB / RDS          |
| Email          | SES                     |
| SMS            | SNS                     |
| Secrets        | Secrets Manager         |
| Monitoring     | CloudWatch              |

**Need**                      **AWS**

Global routing    Route53 + CloudFront

---

## 25. Trade-offs

**Q36. Queue vs Direct Send?**

**Answer:**

**Queue**                      **Direct Send**

Reliable, async    Lower complexity

Retry/DLQ              Weak resilience

Queue preferred at scale.

---

## 26. Anti-Patterns

**Q37. Dangerous anti-patterns?**

**Answer:**

- sync send in request path
  - no DLQ
  - no idempotency
  - no provider fallback
  - single provider lock
  - no preference checks
- 

## 27. FAANG Trap Questions

**Q38. Can notification be exactly-once?**

**Answer:**

Hard. Usually at-least-once + idempotency.

---

**Q39. Can queue be source of truth?**

**Answer:**



No.

---

**Q40. Can retry cause duplicate OTP?**

**Answer:**

Yes, without idempotency.

Huge trap.

---

### **Final Architect Summary**

Best design:

**API + Orchestrator + Template + Preference Service + Event Bus + Channel Workers + Retry + DLQ + Provider Abstraction + Cache + Analytics + Multi-region HA**

**Azure:**

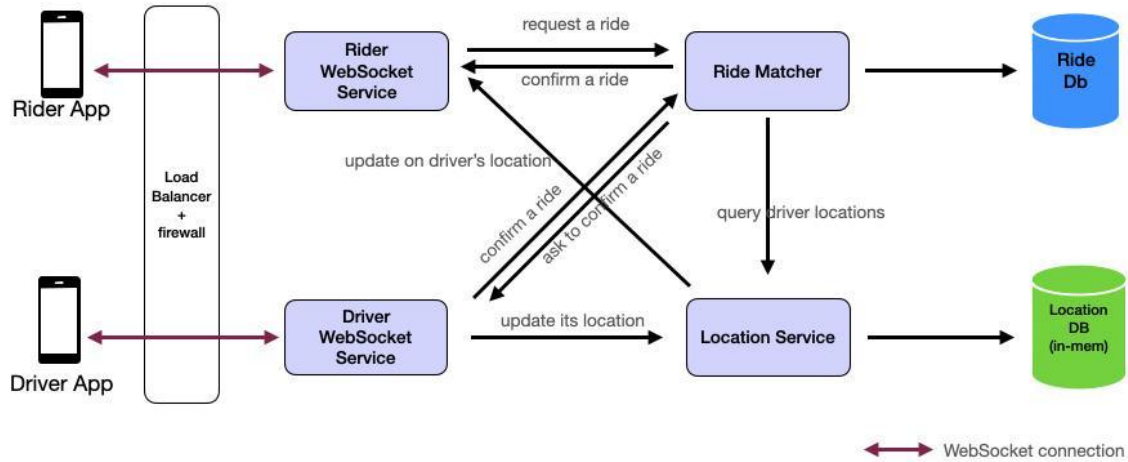
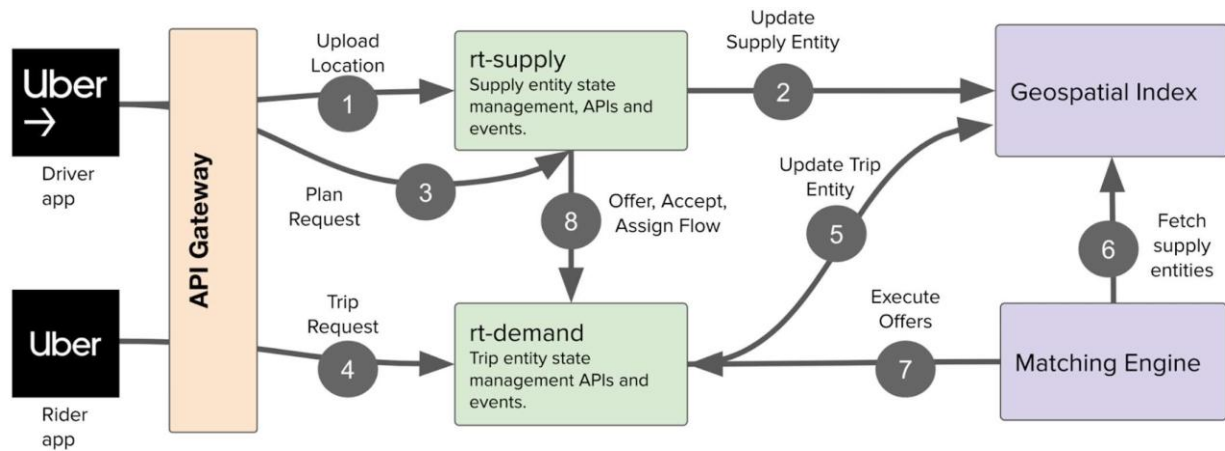
AKS/App Service + Service Bus + Functions + Redis + Cosmos/SQL + Key Vault

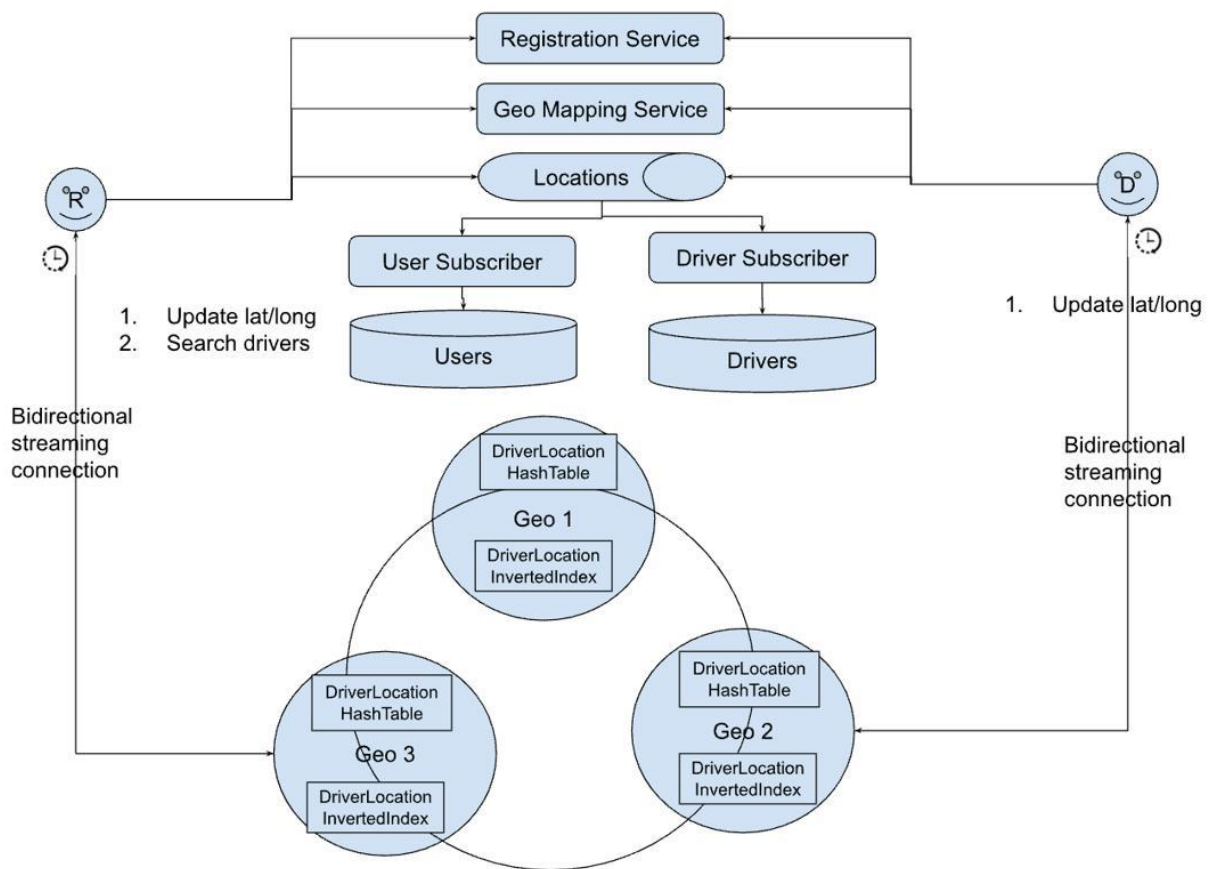
**AWS:**

EKS/ECS + SQS/SNS + Lambda + ElastiCache + Dynamo/RDS + SES/SNS

This is a strong **Enterprise / Cloud / FAANG / Accenture Architect answer.**

## Design 10 → Uber / Ride Matching System





7

This answers:

**“How do you design Uber-like ride matching system handling millions of drivers, riders, and live GPS events?”**

**Q1. What are core requirements of Ride Matching System?**

**Answer:**

**Functional Requirements**

- rider requests trip
- nearest driver matching
- live driver GPS updates
- ETA estimation

- trip lifecycle
- surge pricing
- route tracking
- payment
- cancellation
- rating/review
- notifications
- fraud detection

### Non-Functional Requirements

- ultra-low latency (<100–300ms matching)
- millions of concurrent drivers
- real-time updates
- HA
- geo-distributed
- fault tolerance
- horizontal scale
- accurate geospatial queries

---

### 1. Capacity Estimation

Assume:

- 100M riders
- 10M drivers
- 2M active drivers
- millions GPS updates/minute
- millions trips/day

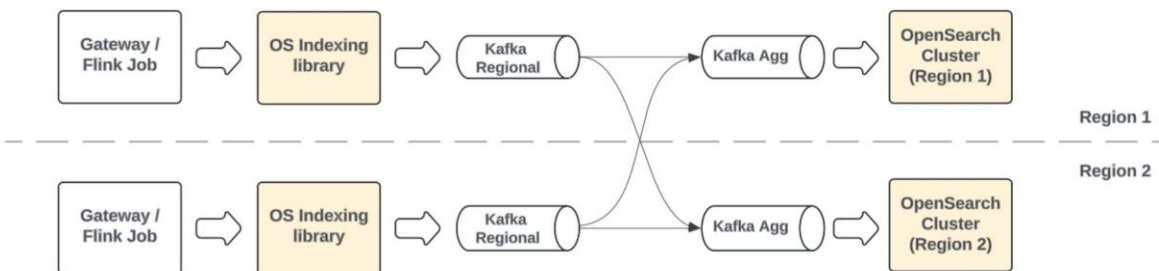
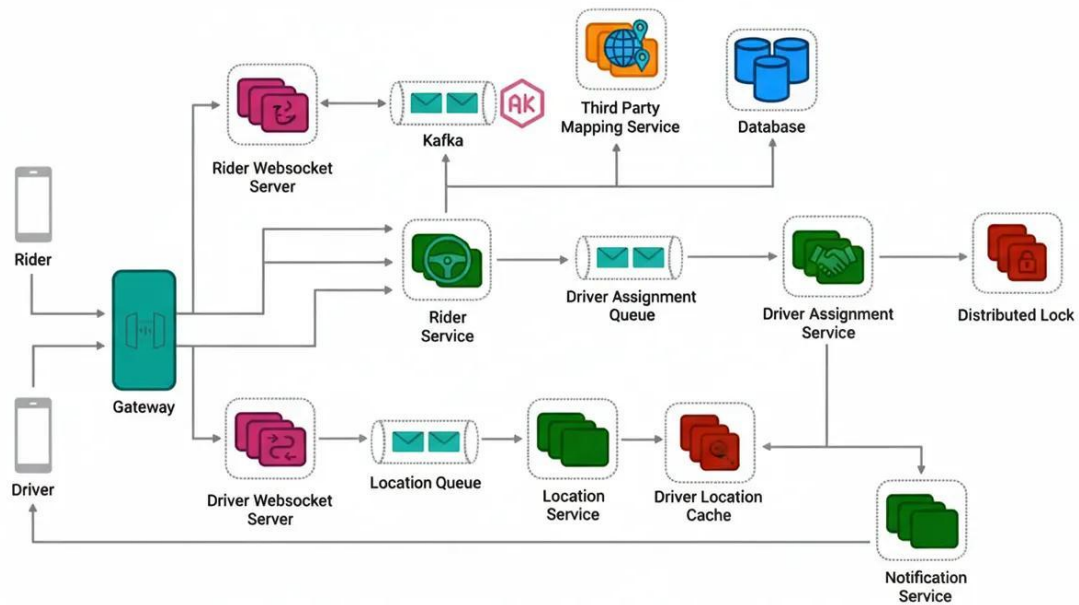
Challenges:

- geospatial search
- hot city zones
- matching speed

- route recalculation
- event scale

Huge FAANG real-time topic.

## 2. High-Level Architecture



| Layer                                        | Problem solved                                                                  | Technology                                                   | Tradeoff                                  |
|----------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------|
| <b>Edge</b><br>ingestion<br>layer 1          | Raw firehose must not hit the database — validate, deduplicate, and gate        | WebSocket over TCP<br>heartbeat pruning<br>adaptive sampling | Stateless — no local durability for pings |
| <b>Kafka</b><br>geo-partition<br>layer 2     | Driver identity is the wrong partition key — dispatch needs geographic locality | Kafka + H3 cell as partition key<br>gridDisk for range       | Cell boundary handoff needs dual publish  |
| <b>Ring</b><br>buffer<br>layer 3             | GPS data expires in seconds — real-time serving needs bounded, fresh memory     | Redis capped list<br>TTL eviction<br>per-driver ring         | No history — last 10 pings only           |
| <b>Cassandra</b><br>durable store<br>layer 4 | Analytics, fraud, compliance need full history — 83k writes/sec must not block  | Cassandra LSM tree<br>LOCAL_QUORUM<br>date-bucket partition  | Write amplification from compaction       |
| <b>Dispatch</b><br>engine<br>layer 5         | Match rider to driver in under 100ms — $O(N)$ DB scan is too slow               | H3 gridDisk + Redis<br>MGET + parallel<br>ETA routing        | Sparse areas need ring expansion          |
| <b>Rendering</b><br>map layer<br>layer 6     | Raw GPS is noisy, discrete, and off-road — rendering must feel smooth           | Kalman filter<br>dead reckoning<br>map matching              | Display always 200ms behind reality       |

7

## Q2. Core services?

Answer:

Usually:

- API Gateway
- Rider Service
- Driver Service
- Location Ingestion Service
- Geo Index / Location Service
- Matching Engine
- ETA Service
- Pricing/Surge Service

- Trip Service
  - Notification Service
  - Payment Service
  - Fraud Service
  - Event Bus
  - Analytics
- 

### **3. GPS / Location Ingestion**

#### **Q3. How driver location updates handled?**

**Answer:**

Driver app sends GPS periodically.

Flow:

Driver → Gateway → Location Service → Geo Index → Event Stream

Very high-write system.

---

#### **Q4. Why separate Location Service?**

**Answer:**

Location is:

- highly volatile
- massive write-heavy
- geo-query focused

Should be isolated.

---

### **4. Geospatial Indexing (Critical)**

#### **Q5. What is geospatial indexing?**

**Answer:**

Efficiently storing/searching nearby coordinates.

Huge Uber topic.

---

#### **Q6. Common approaches?**

**Answer:**

- Geohash
- QuadTree
- R-Tree
- Grid partitioning

Must know.

---

#### **Q7. Why geohash popular?**

**Answer:**

Easy partitioning + nearby search.

Very common architect answer.

---

#### **5. Matching Engine**

#### **Q8. Ride matching flow?**

**Answer:**

Step 1 → Rider requests trip

Step 2 → Find nearby drivers

Step 3 → Rank candidates

Step 4 → Send offer

Step 5 → Accept

Step 6 → Lock driver

Step 7 → Start trip

---

#### **Q9. What factors decide driver ranking?**

**Answer:**

- distance
- ETA
- availability
- driver score
- surge zone



- trip priority
- 

#### **Q10. Why separate Matching Engine?**

**Answer:**

This is core business logic.

Heavy real-time decisions.

---

#### **6. Hot Zone / Surge Pricing**

#### **Q11. What is surge pricing?**

**Answer:**

Price increases when demand > supply.

Huge Uber favorite.

---

#### **Q12. How calculate surge?**

**Answer:**

Based on:

- active riders
- available drivers
- region demand
- traffic
- events

Usually geo-based.

---

#### **Q13. Hot zone problem?**

**Answer:**

Cities/airports/events create huge traffic.

Need:

- shard geo regions
- cache

- autoscale
- 

## 7. Trip Lifecycle

### Q14. Trip states?

**Answer:**

Usually:

- requested
  - matched
  - driver\_arriving
  - started
  - completed
  - cancelled
  - failed
- 

### Q15. Why state machine?

**Answer:**

Trips are workflow-driven.

Helps:

- consistency
  - retries
  - auditing
- 

## 8. Live Tracking

### Q16. How live tracking works?

**Answer:**

Driver GPS → Event Stream → Rider app updates.

Usually WebSocket / push updates.

---

### Q17. Why event stream?

**Answer:**

Decouples ingestion and consumers.

Supports:

- tracking
- ETA
- analytics
- fraud

Huge distributed topic.

---

## **9. ETA Calculation**

### **Q18. ETA inputs?**

**Answer:**

- driver distance
  - traffic
  - route
  - weather
  - speed history
- 

## **10. Payment**

### **Q19. Why separate Payment Service?**

**Answer:**

Different domain:

- wallets
  - refunds
  - settlement
  - PCI
  - invoices
- 

## **11. Notifications**

## Q20. What notifications?

Answer:

- driver assigned
  - trip started
  - arrival
  - cancellation
  - receipt
  - payment success
- 

## 12. DB Design

### Q21. Which DBs?

Answer:

Usually polyglot:

- Relational → trips/payments
  - Geo store / fast location DB → live coordinates
  - Redis → hot region cache
  - Event bus → streaming
  - Data lake → analytics
- 

### Q22. Why not relational DB for live GPS only?

Answer:

Location writes are huge.

Geo-query optimized stores are better.

---

## 13. Caching

### Q23. What to cache?

Answer:

- hot geo cells
- active drivers

- ETA
  - pricing
  - rider profile
- 

#### 14. Reliability

**Q24. What if two riders get same driver?**

**Answer:**

Use atomic reservation/locking.

Huge race-condition topic.

---

**Q25. Idempotency needed?**

**Answer:**

Yes:

- trip creation
  - payment
  - notifications
- 

#### 15. Fraud / Security

**Q26. Fraud checks?**

**Answer:**

- fake GPS
  - driver spoofing
  - payment abuse
  - account fraud
  - unusual ride behavior
- 

**Q27. Security controls?**

**Answer:**

- auth

- TLS
  - token validation
  - RBAC
  - PII masking
  - rate limiting
- 

## 16. HA / DR

### Q28. HA design?

**Answer:**

Use:

- multi-region APIs
  - geo sharding
  - stateless services
  - queue durability
  - replicated DB
- 

### Q29. DR strategy?

**Answer:**

- cross-region failover
  - backups
  - replay streams
  - tested recovery
- 

## 17. Scale

### Q30. How scale to millions GPS updates?

**Answer:**

Use:

- partitioned event streams
- geo partitioning

- distributed caches
- async consumers
- autoscaling

---

### Q31. Hotspot bottlenecks?

**Answer:**

- airport zones
  - event areas
  - surge recalculation
  - hot geohashes
  - matching spikes
- 

### 18. Azure Mapping (.NET-focused)

#### Q32. Azure services?

**Answer:**

| Need              | Azure                 |
|-------------------|-----------------------|
| API               | App Service / AKS     |
| Event streaming   | Event Hubs            |
| Queue             | Service Bus           |
| Cache             | Azure Redis           |
| DB                | Azure SQL / Cosmos DB |
| Maps/Geo          | Azure Maps            |
| Real-time updates | SignalR               |
| Analytics         | Synapse + Data Lake   |
| Secrets           | Key Vault             |
| Global routing    | Front Door            |
| Containers        | AKS                   |

---

## 19. AWS Mapping (.NET-focused)

### Q33. AWS services?

Answer:

| Need           | AWS                     |
|----------------|-------------------------|
| API            | API Gateway / ECS / EKS |
| Streaming      | Kinesis                 |
| Queue          | SQS / SNS               |
| Cache          | ElastiCache             |
| DB             | DynamoDB / RDS          |
| Geo            | Amazon Location Service |
| Real-time      | AppSync / WebSocket     |
| Analytics      | Athena / S3 / EMR       |
| Secrets        | Secrets Manager         |
| Global routing | Route53                 |

---

## 20. Trade-offs

### Q34. Push GPS vs pull GPS?

Answer:

| Push            | Pull           |
|-----------------|----------------|
| Real-time       | Simpler        |
| Better tracking | Higher latency |
| Push preferred. |                |

---

### Q35. Strong vs eventual consistency?

Answer:

Strong:



- driver assignment
- payment
- trip lock

Eventual:

- analytics
  - ETA updates
  - reporting
- 

## 21. Anti-Patterns

### Q36. Dangerous anti-patterns?

Answer:

- no geo partitioning
  - global driver search
  - DB-only GPS writes
  - no lock for matching
  - sync-heavy ETA
  - no idempotency
- 

## 22. FAANG Trap Questions

### Q37. Can one driver match two riders?

Answer:

Must prevent with atomic lock.

---

### Q38. Can Redis be source of trip truth?

Answer:

No.

---

### Q39. Can relational DB alone handle geo scale?

Answer:

Usually not ideal.

---

### **Final Architect Summary**

Best design:

**API Gateway + Location Service + Geo Index + Matching Engine + Pricing + Trip State Machine + Event Stream + Redis + Polyglot DB + WebSocket/SignalR + Payment + Multi-region HA**

**Azure:**

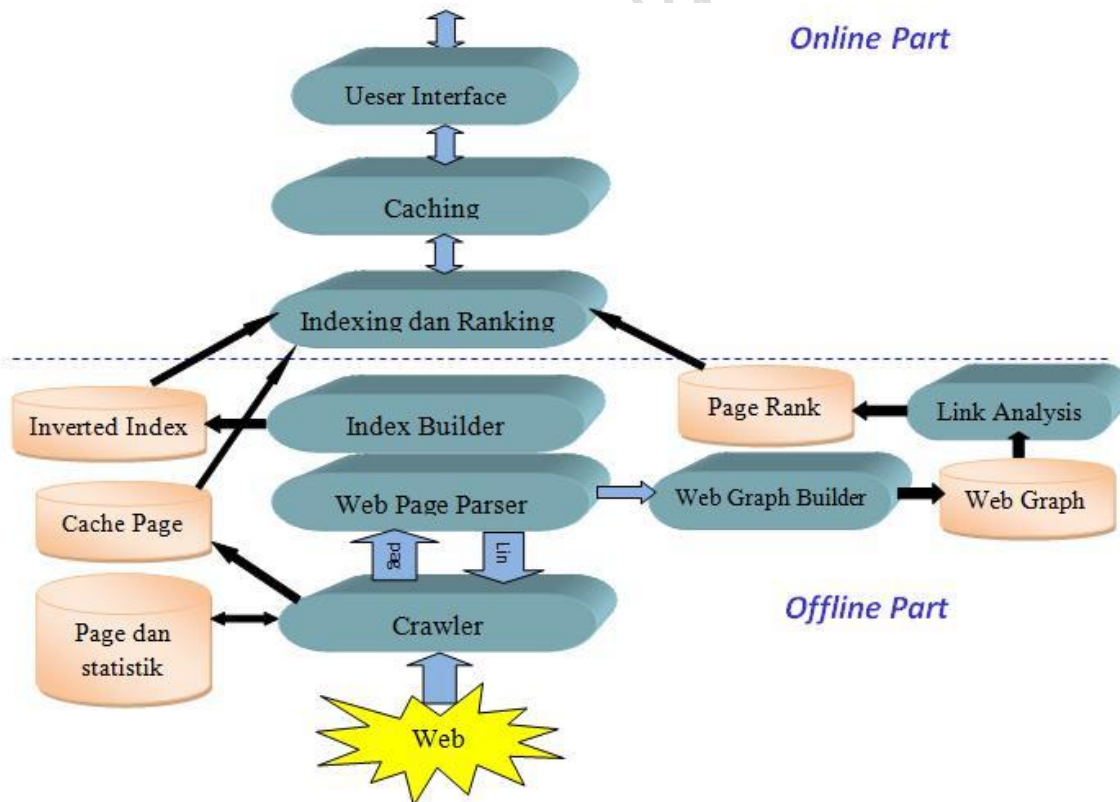
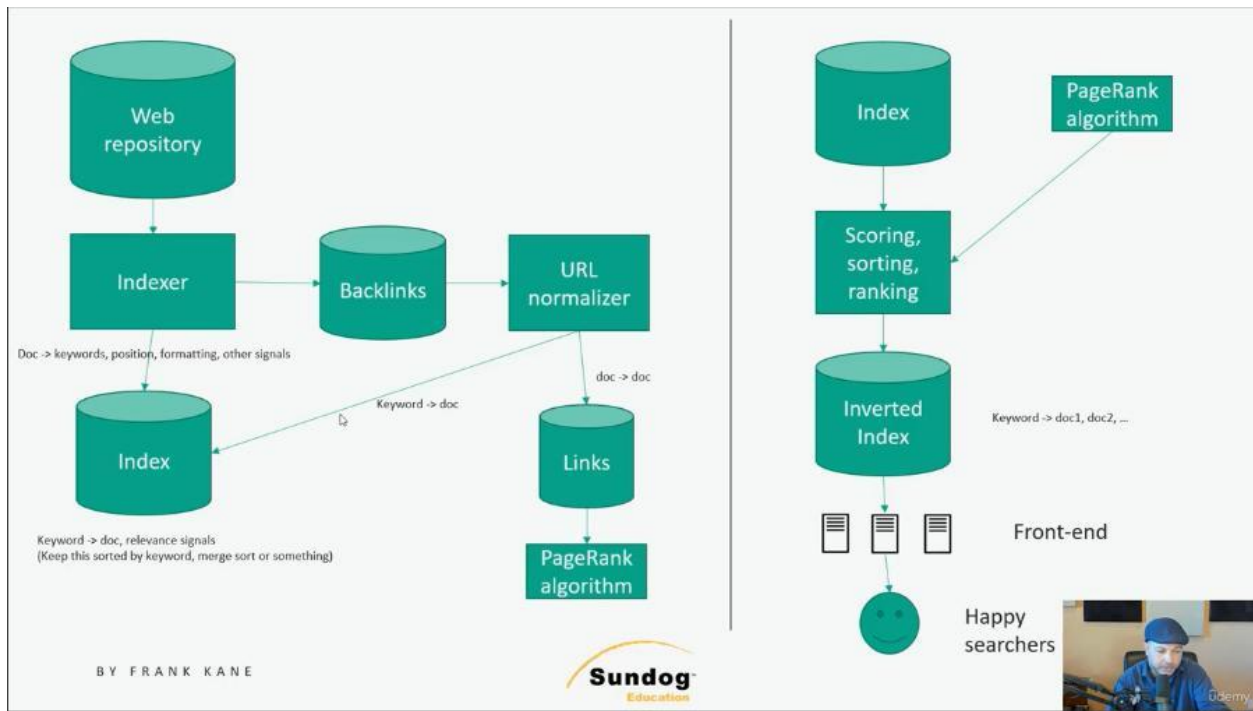
AKS + Event Hubs + Cosmos + Azure Maps + SignalR + Redis + Front Door

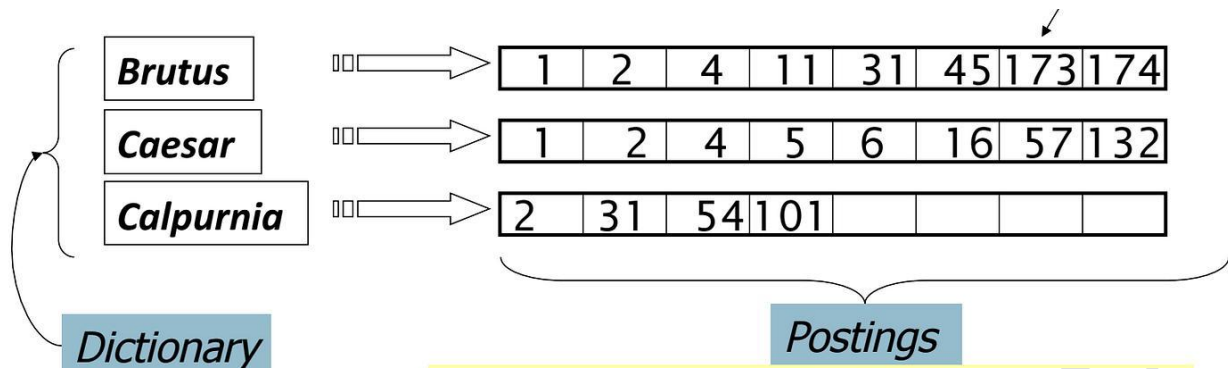
**AWS:**

EKS + Kinesis + DynamoDB + Amazon Location + AppSync/WebSocket + ElastiCache + Route53

This is a strong **Uber / FAANG / Cloud / Distributed Systems Architect** answer.

## Design 11 → Search Engine (Google-like basics)





6

This answers:

**“How do you design a Google-like search engine handling billions of pages and millions of search queries/sec?”**

**Q1. What are core requirements of Search Engine?**

**Answer:**

#### **Functional Requirements**

- crawl web/documents
- parse content
- index pages
- keyword search
- ranking
- autocomplete
- spell correction
- pagination
- filters
- caching
- analytics
- freshness updates

#### **Non-Functional Requirements**

- billions of documents
- low-latency search (<100ms ideal)

- huge read scale
  - relevance ranking
  - HA
  - geo-distribution
  - fault tolerance
  - horizontal scaling
- 

## 1. Capacity Estimation

Assume:

- 50B documents/pages
- PB-scale content
- 100M–1B searches/day
- millions QPS globally

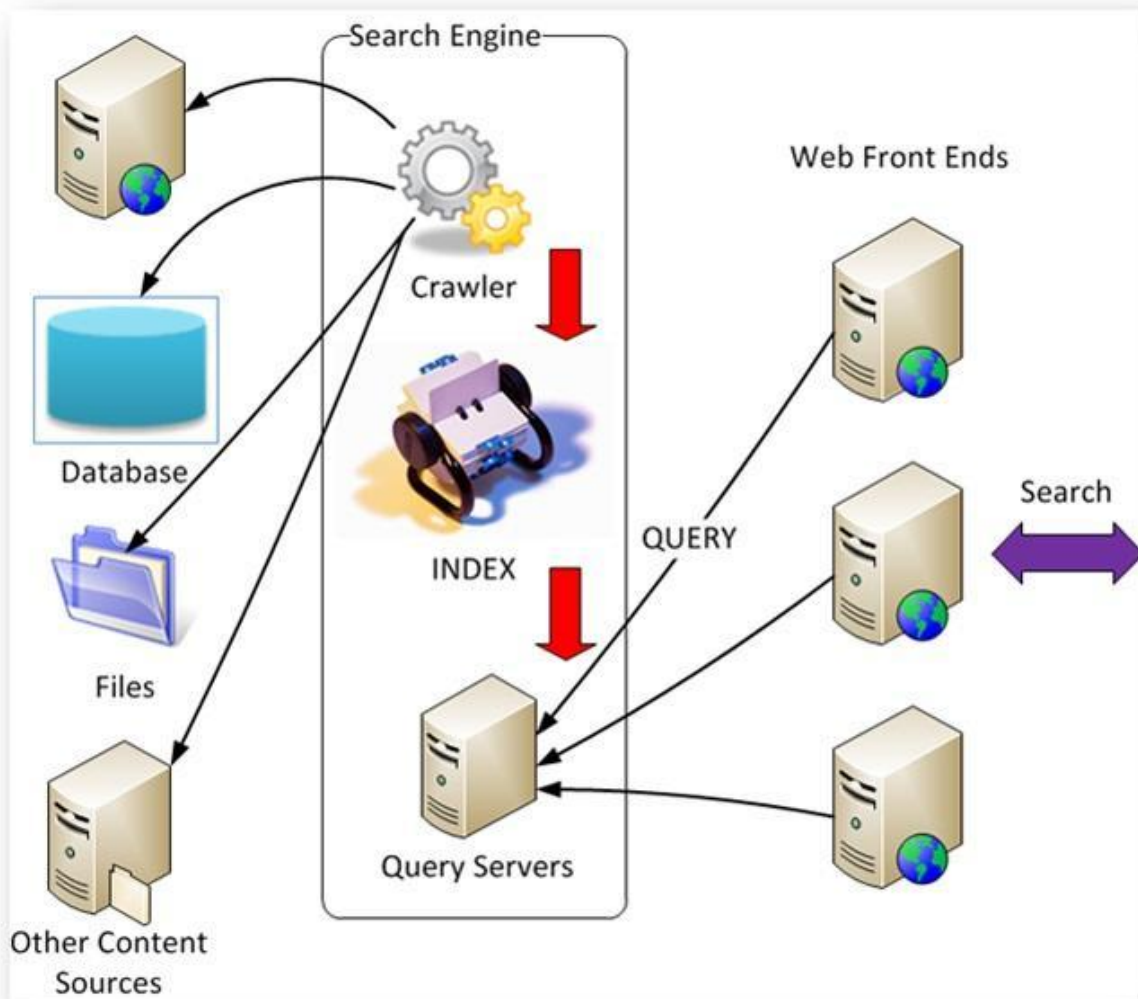
Challenges:

- crawl scale
- indexing speed
- ranking complexity
- hot queries
- shard balancing

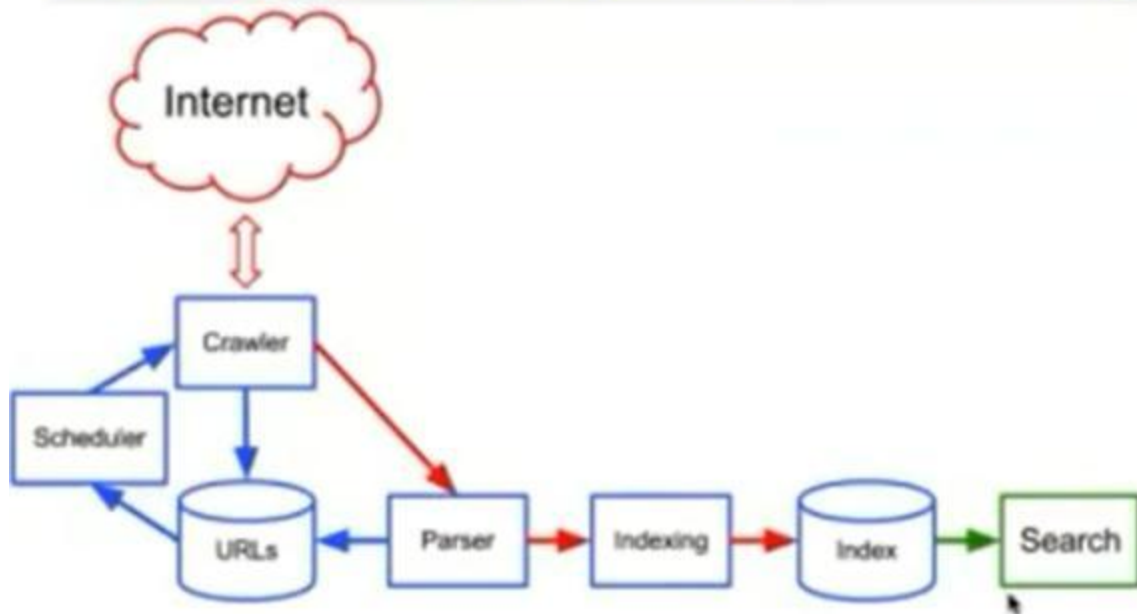
Huge FAANG topic.

---

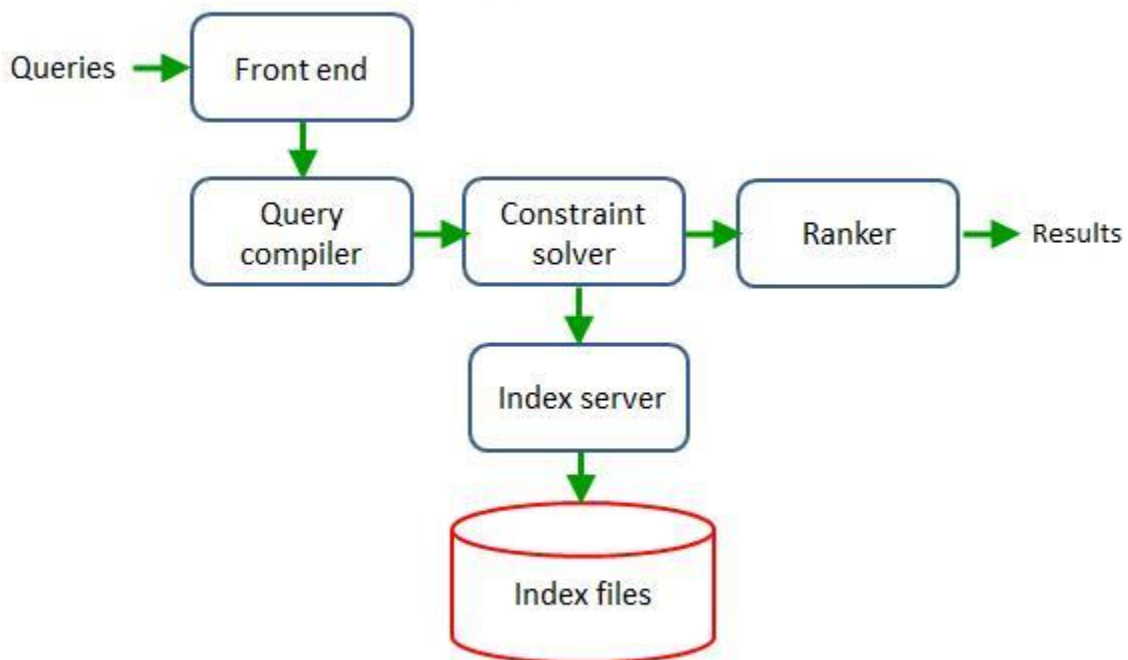
## 2. High-Level Architecture



## The Pipeline (simplified)



## The query serve side



**Q2. Core components?****Answer:**

Usually:

- Crawl Scheduler
  - Crawler Workers
  - Parser
  - Deduplication Service
  - Indexer
  - Inverted Index Store
  - Ranking Engine
  - Query Service
  - Cache Layer
  - Spell Correction Service
  - Autocomplete Service
  - Analytics Service
- 

**3. Crawling****Q3. What is web crawling?****Answer:**

Discovering/fetching pages/documents.

Huge Google topic.

**Q4. Crawl flow?****Answer:**

Seed URLs → Scheduler → Workers → Fetch pages → Parse → Store → Re-crawl.

**Q5. Why Crawl Scheduler?****Answer:**



Controls:

- priority
  - freshness
  - politeness
  - retries
  - partitioning
- 

#### 4. Parsing

**Q6. What parser does?**

**Answer:**

Extracts:

- text
  - title
  - links
  - metadata
  - language
- 

**Q7. Why deduplication?**

**Answer:**

Avoid indexing duplicate/similar content.

Huge scale issue.

---

#### 5. Inverted Index (Critical)

**Q8. What is inverted index?**

**Answer:**

Keyword → list of document IDs.

Core search concept.

Example:

cloud → [doc12, doc90, doc101]

Huge FAANG favorite.

---

### **Q9. Why inverted index?**

**Answer:**

Fast keyword lookup.

Much faster than scanning all docs.

---

### **Q10. Forward index vs inverted index?**

**Answer:**

**Forward Index Inverted Index**

doc → terms    term → docs

Must know.

---

## **6. Query Serving**

### **Q11. Search query flow?**

**Answer:**

Step 1 → User searches

Step 2 → Query parser

Step 3 → spell correction

Step 4 → autocomplete/synonyms

Step 5 → shard lookup

Step 6 → rank results

Step 7 → merge top N

Step 8 → return response

---

### **Q12. Why Query Parser?**

**Answer:**

Normalizes:

- stemming
- tokenization
- filters

- operators
- 

## 7. Ranking Engine

### Q13. How rank search results?

**Answer:**

Signals:

- keyword relevance
- freshness
- popularity
- links
- engagement
- ML ranking

Huge architect topic.

---

### Q14. Why ranking separate?

**Answer:**

Heavy compute logic.

Keeps query service cleaner.

---

## 8. Sharding

### Q15. Why shard index?

**Answer:**

Billions of docs cannot fit in single node.

Huge distributed topic.

---

### Q16. Common sharding?

**Answer:**

- term-based
- document-based

- geo partition
  - hybrid
- 

#### **Q17. Shard merge?**

**Answer:**

Each shard returns top N → aggregator merges best final results.

---

#### **9. Replication / Reliability**

##### **Q18. Why replicate index?**

**Answer:**

Supports:

- HA
  - faster reads
  - failover
  - geo serving
- 

#### **10. Caching**

##### **Q19. What to cache?**

**Answer:**

- hot queries
- trending results
- autocomplete
- ranking metadata

Huge FAANG topic.

---

##### **Q20. Hot query problem?**

**Answer:**

Popular searches create massive load.

Cache mitigates.

---

## 11. Autocomplete

### Q21. How autocomplete works?

**Answer:**

Uses:

- prefix trees (Trie)
- popularity
- history

Very common.

---

## 12. Spell Correction

### Q22. Why spell correction?

**Answer:**

Users mistype queries.

Improves relevance.

---

### Q23. Common methods?

**Answer:**

- edit distance
  - dictionary
  - n-grams
  - ML ranking
- 

## 13. Freshness / Reindexing

### Q24. Why re-crawl?

**Answer:**

Pages change.

Need freshness.

---

#### **Q25. Hot content reindexing?**

**Answer:**

High-priority pages re-crawled faster.

---

#### **14. DB / Storage Design**

##### **Q26. Storage layers?**

**Answer:**

Usually:

- object store → raw docs
  - distributed index store
  - metadata DB
  - cache
  - analytics store
- 

##### **Q27. Why object storage?**

**Answer:**

Cheap long-term storage.

---

#### **15. Analytics**

##### **Q28. Why analytics?**

**Answer:**

Track:

- top queries
  - CTR
  - no-result searches
  - ranking quality
  - abuse
- 

#### **16. Security / Abuse**

### Q29. Risks?

Answer:

- crawl abuse
  - spam pages
  - SEO poisoning
  - DDoS
  - malicious indexing
- 

### Q30. Security controls?

Answer:

- WAF
  - auth (enterprise search)
  - rate limiting
  - bot filtering
  - validation
  - safe crawling policies
- 

## 17. Scale

### Q31. How scale to billions docs?

Answer:

Use:

- distributed crawling
  - shard index
  - replication
  - async pipelines
  - caching
  - geo clusters
- 

## 18. Bottlenecks

### Q32. Common hotspots?

Answer:

- hot queries
  - ranking latency
  - shard imbalance
  - crawl spikes
  - reindex load
  - merge nodes
- 

### 19. Azure Mapping (.NET-focused)

#### Q33. Azure services?

Answer:

| Need           | Azure                    |
|----------------|--------------------------|
| APIs           | AKS / App Service        |
| Queue          | Service Bus / Event Hubs |
| Object Store   | Blob Storage             |
| Search         | Azure AI Search          |
| Cache          | Azure Redis              |
| DB             | Cosmos DB / SQL          |
| Analytics      | Synapse / Data Lake      |
| ML ranking     | Azure ML                 |
| Secrets        | Key Vault                |
| Global routing | Front Door               |

---

### 20. AWS Mapping (.NET-focused)

#### Q34. AWS services?

Answer:



| Need           | AWS                  |
|----------------|----------------------|
| APIs           | EKS / ECS            |
| Queue          | SQS / Kinesis        |
| Object Store   | S3                   |
| Search         | OpenSearch           |
| Cache          | ElastiCache          |
| DB             | DynamoDB / RDS       |
| Analytics      | Athena / EMR         |
| ML             | SageMaker            |
| Secrets        | Secrets Manager      |
| Global routing | Route53 / CloudFront |

---

## 21. Trade-offs

### Q35. Term-based vs document-based sharding?

Answer:

|                   |                       |
|-------------------|-----------------------|
| <b>Term-based</b> | <b>Document-based</b> |
|-------------------|-----------------------|

|                 |                     |
|-----------------|---------------------|
| Query optimized | Easier distribution |
|-----------------|---------------------|

Architect trade-off.

---

### Q36. Strong vs eventual consistency?

Answer:

Strong:

- query correctness per shard

Eventual:

- indexing freshness
- analytics
- suggestions

---

## 22. Anti-Patterns

### Q37. Dangerous anti-patterns?

Answer:

- full scan search
  - no inverted index
  - no cache
  - ranking in sync-heavy DB path
  - no dedupe
  - single index node
- 

## 23. FAANG Trap Questions

### Q38. Can relational DB alone power Google-scale search?

Answer:

No.

---

### Q39. Can cache replace index?

Answer:

No.

---

### Q40. Can indexing be synchronous?

Answer:

Usually no.

---

## Final Architect Summary

Best design:

**Crawler + Scheduler + Parser + Dedup + Inverted Index + Query Service + Ranking Engine + Cache + Trie + Replication + Geo-distributed clusters**

Azure:

AKS + Azure AI Search + Blob + Redis + Cosmos + Synapse + Azure ML

**AWS:**

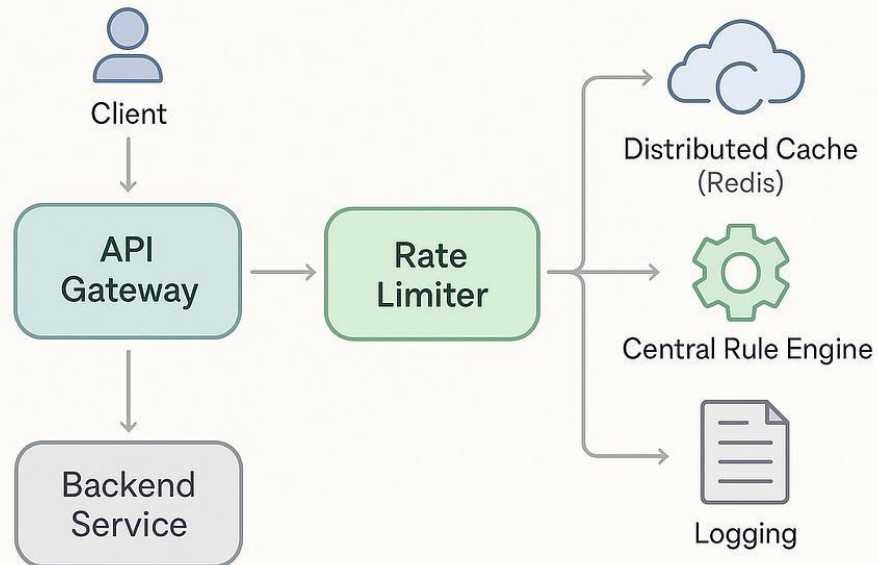
EKS + OpenSearch + S3 + ElastiCache + Dynamo + Athena + SageMaker

This is a strong **Google / Search / FAANG / Cloud Architect** answer.

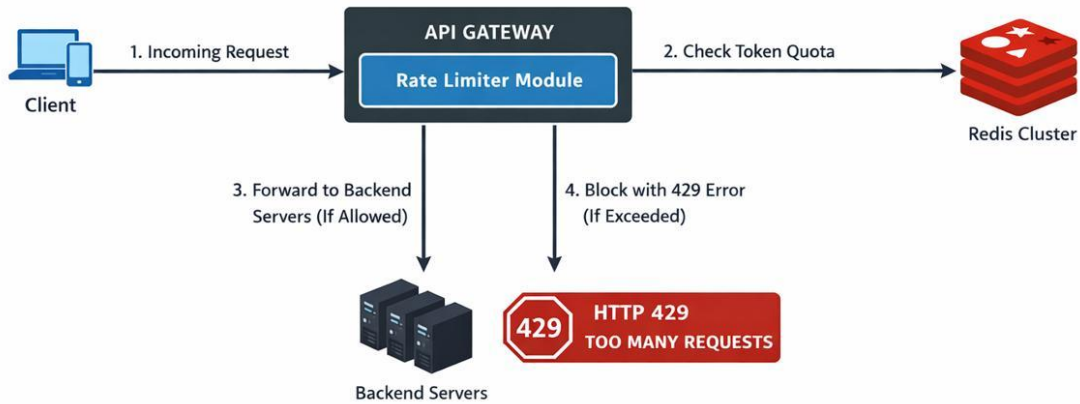
## Design 12 → Distributed Rate Limiter

### Designing a Rate Limiter

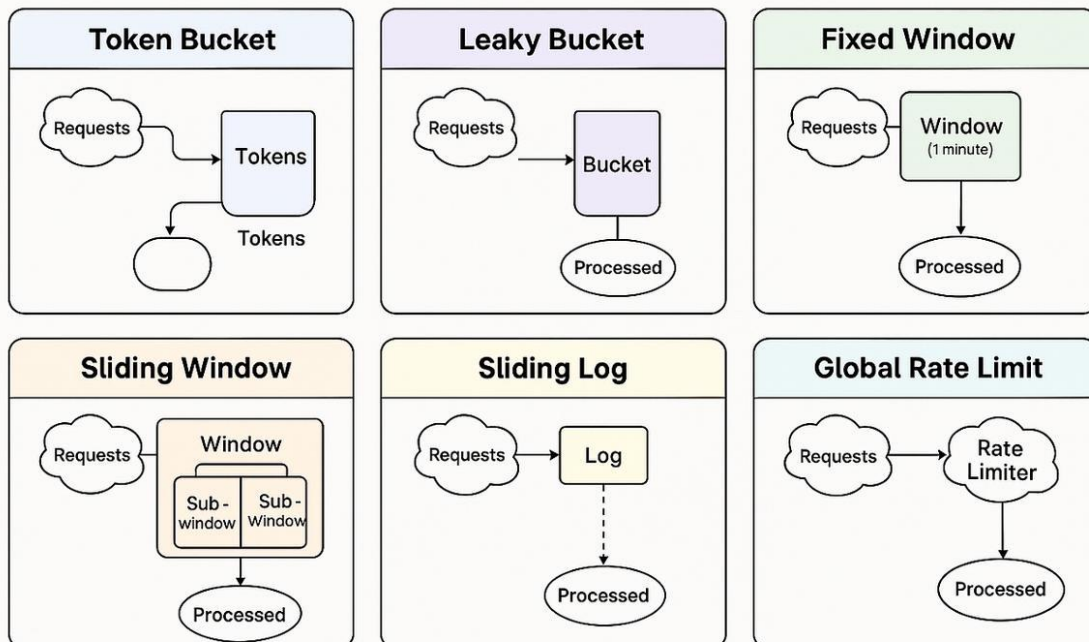
Balancing User Load and Optimizing Security and Fault Tolerance



## Distributed Rate Limiter - High-Level Design



## Rate Limiting & Throttling Algorithms



This answers:

**“How do you design a distributed rate limiter handling millions of requests/sec across multiple nodes?”**

We cover:

- token bucket
- leaky bucket
- fixed window
- sliding window
- Redis
- hot keys
- distributed counters
- consistency
- sharding
- HA/DR
- abuse protection
- Azure + AWS mapping

---

### **Q1. What is Rate Limiter?**

**Answer:**

A control mechanism that limits how many requests a client can make in a given time.

Example:

100 requests / minute / user

Prevents:

- abuse
- DDoS
- brute force
- noisy tenants
- backend overload

Huge architect topic.

---

## **Q2. Where is rate limiter used?**

**Answer:**

Usually:

- API Gateway
  - login endpoints
  - payment APIs
  - OTP systems
  - search APIs
  - notification services
  - SaaS tenant control
- 

## **1. Functional Requirements**

Must support:

- per user limits
  - per IP limits
  - per API limits
  - per tenant limits
  - burst handling
  - distributed nodes
  - global throttling
  - retries
  - dynamic config
- 

## **2. Non-Functional Requirements**

Need:

- low latency (<1–5ms ideal)
- very high throughput

- HA
  - fault tolerance
  - consistency
  - no SPOF
  - scalable
  - low memory
- 

### 3. Capacity Estimation

Assume:

- 10M clients
- 5M req/sec
- multiple regions
- burst traffic

Challenges:

- hot keys
- consistency
- counter accuracy
- Redis bottlenecks

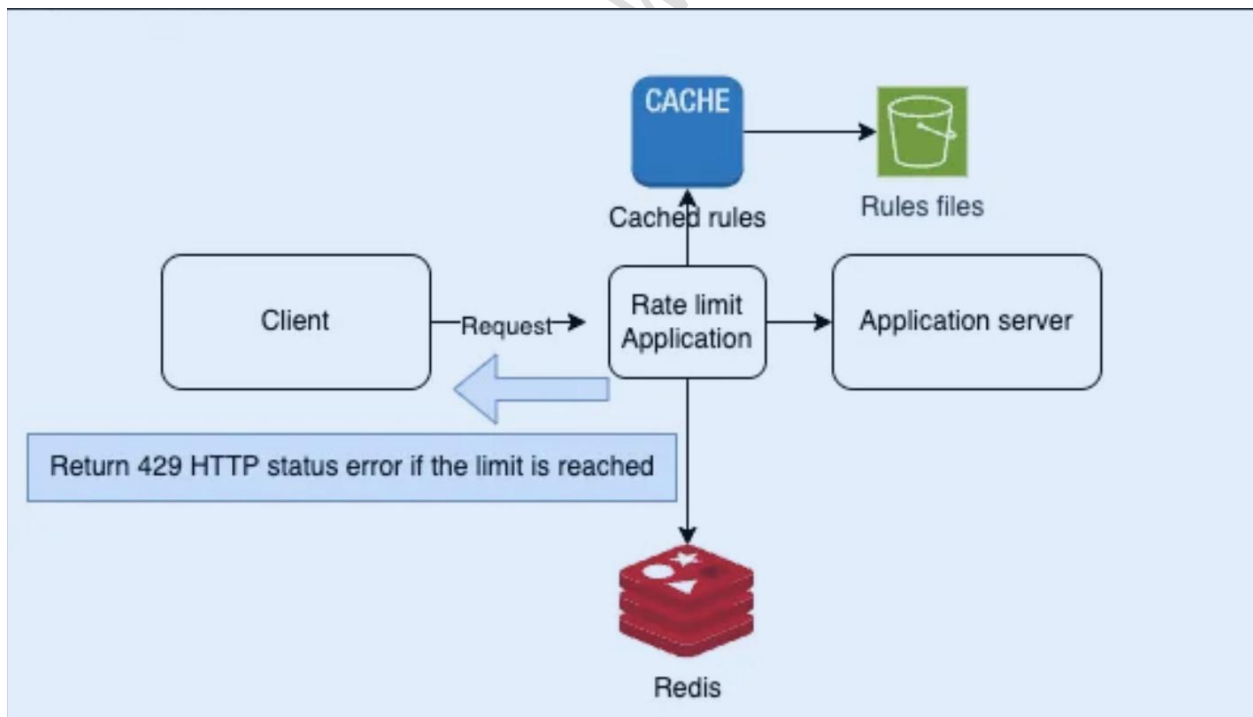
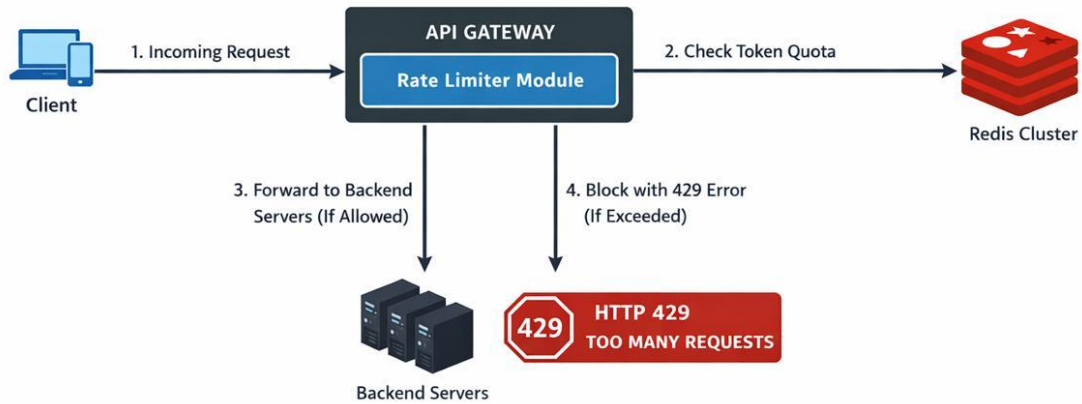
Huge distributed topic.

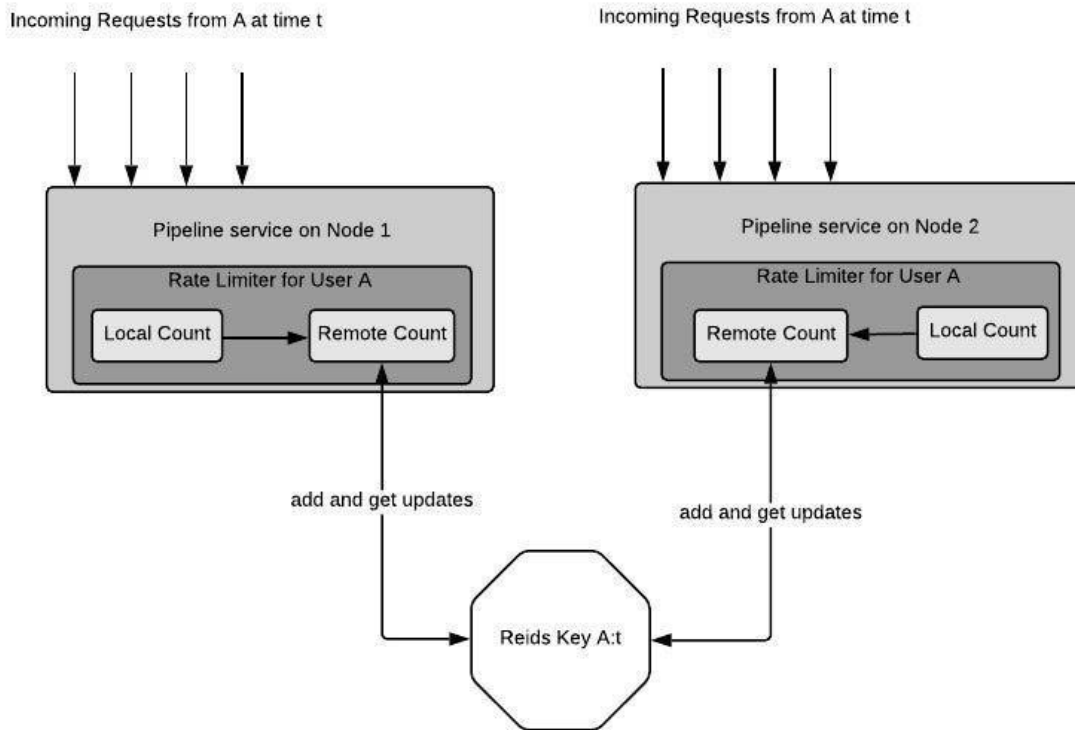
---

### 4. High-Level Architecture



## Distributed Rate Limiter - High-Level Design





7

### Q3. Core components?

**Answer:**

Usually:

- API Gateway / Edge Proxy
- Rate Limit Service
- Policy Store
- Counter Store (Redis)
- Sharding Layer
- Config Service
- Metrics/Observability
- Fallback Logic

## 5. Request Flow

#### Q4. End-to-end rate limit flow?

**Answer:**

Step 1 → Client request arrives  
Step 2 → API Gateway intercepts  
Step 3 → Build key (user/IP/API/tenant)  
Step 4 → Counter check  
Step 5 → Compare against policy  
Step 6 → Allow or reject  
Step 7 → Update metrics/logs

---

### 6. Key Design

#### Q5. What is rate limit key?

**Answer:**

Identifier used for counting.

Examples:

- userId
  - IP
  - API route
  - tenantId
  - token
- 

#### Q6. Composite key examples?

**Answer:**

tenant123:/payment  
user88:/login  
IP\_10.1.1.5

Very common.

---

### 7. Algorithms (Critical)

#### Q7. What is Fixed Window?

**Answer:**

Simple counter per time window.

Example:  
100 req/min.

Easy but bursty at window edges.

---

#### **Q8. Fixed window issue?**

**Answer:**

Boundary burst.

User can hit end + start of next window.

Huge trap.

---

#### **Q9. What is Sliding Window?**

**Answer:**

Tracks rolling time range.

More accurate.

Huge FAANG favorite.

---

#### **Q10. Why sliding window better?**

**Answer:**

Smooth enforcement.

Reduces burst loopholes.

---

#### **Q11. What is Token Bucket?**

**Answer:**

Tokens refill over time.

Request consumes token.

Very common API gateway design.

---

#### **Q12. Why token bucket useful?**

**Answer:**

Allows burst traffic while maintaining average rate.

Excellent for APIs.

---

### **Q13. What is Leaky Bucket?**

**Answer:**

Processes requests at fixed outflow rate.

Smooth traffic.

---

### **Q14. Token Bucket vs Leaky Bucket?**

**Answer:**

**Token Bucket    Leaky Bucket**

Allows bursts    Smooth fixed output

Flexible            Strong smoothing

Must know.

---

### **8. Which Algorithm Best?**

#### **Q15. Best for API Gateway?**

**Answer:**

Usually Token Bucket + Redis.

Very common architect answer.

---

#### **Q16. Best for brute-force/login?**

**Answer:**

Sliding Window often stronger.

---

### **9. Counter Store (Critical)**

#### **Q17. Why Redis common?**

**Answer:**

Fast:

- atomic increments
- TTL
- low latency
- distributed cache

Huge topic.

---

**Q18. Can DB be used?**

**Answer:**

Possible but slower.

High QPS prefers Redis.

---

**Q19. Atomicity requirement?**

**Answer:**

Critical.

Must avoid race conditions.

Use atomic increment/Lua.

---

**10. Distributed System Design**

**Q20. Why centralized counter service?**

**Answer:**

Consistent enforcement across many API nodes.

---

**Q21. Problem if each node tracks local counters?**

**Answer:**

Inconsistent limits.

Huge trap.

---

**Q22. How scale counter store?**

**Answer:**

Use:

- Redis cluster
  - sharding
  - replicas
  - partitioned keys
- 

## **11. Hot Key Problem**

**Q23. What is hot key?**

**Answer:**

One key gets massive traffic.

Example:

shared public API token.

Huge distributed hotspot.

---

**Q24. How solve hot keys?**

**Answer:**

Use:

- sharding
  - key spreading
  - local buffering
  - partitioned counters
- 

## **12. Consistency**

**Q25. Strong vs eventual consistency?**

**Answer:**

Strong:

- login attempts
- payment APIs

Relaxed:

- analytics APIs
- soft quotas

Architect trade-off.

---

### 13. Fallback / Failure

#### Q26. If Redis unavailable?

**Answer:**

Possible strategies:

- fail open
- fail closed
- local fallback cache

Depends on criticality.

Huge architect question.

---

#### Q27. Fail open vs fail closed?

**Answer:**

|                  |                    |
|------------------|--------------------|
| <b>Fail Open</b> | <b>Fail Closed</b> |
|------------------|--------------------|

|               |               |
|---------------|---------------|
| Allow traffic | Block traffic |
|---------------|---------------|

Example:

Login brute-force → fail closed

Analytics API → maybe fail open

---

### 14. Global Rate Limiting

#### Q28. How handle multi-region global limit?

**Answer:**

Use:

- shared distributed counter
- region sync



- token partitioning
- hierarchical limits

Very advanced.

---

## 15. Dynamic Policy

### Q29. Why Config Service?

**Answer:**

Allows runtime updates:

- premium user
  - tenant quota
  - emergency block
  - endpoint-specific rules
- 

## 16. Observability

### Q30. Metrics?

**Answer:**

Track:

- allow/deny rate
  - latency
  - Redis errors
  - hot keys
  - policy hits
  - abuse spikes
- 

## 17. Security

### Q31. Security uses?

**Answer:**

Protect:

- DDoS

- brute force
  - bot abuse
  - noisy tenants
  - scraping
- 

## 18. Scale

**Q32. How scale to millions req/sec?**

**Answer:**

Use:

- stateless gateway nodes
  - Redis cluster
  - sharded counters
  - local caches
  - async metrics
  - geo clusters
- 

## 19. Bottlenecks

**Q33. Common hotspots?**

**Answer:**

- Redis hot keys
  - central config
  - global sync
  - auth coupling
  - large tenant traffic
- 

## 20. Azure Mapping (.NET-focused)

**Q34. Azure services?**

**Answer:**

| <b>Need</b>   | <b>Azure</b>                 |
|---------------|------------------------------|
| API edge      | API Management / Front Door  |
| App nodes     | AKS / App Service            |
| Counter store | Azure Cache for Redis        |
| Config        | App Configuration            |
| DB            | Cosmos / SQL                 |
| Metrics       | App Insights / Azure Monitor |
| Security      | WAF                          |
| Secrets       | Key Vault                    |

---

## 21. AWS Mapping (.NET-focused)

### Q35. AWS services?

**Answer:**

| <b>Need</b>   | <b>AWS</b>               |
|---------------|--------------------------|
| API edge      | API Gateway / CloudFront |
| App nodes     | ECS / EKS                |
| Counter store | ElastiCache Redis        |
| Config        | AppConfig                |
| DB            | DynamoDB                 |
| Metrics       | CloudWatch               |
| Security      | WAF / Shield             |
| Secrets       | Secrets Manager          |

---

## 22. Trade-offs

### Q36. Redis vs local memory?

**Answer:**

| Redis                   | Local Memory       |
|-------------------------|--------------------|
| Distributed consistency | Fastest local      |
| Higher reliability      | Wrong across nodes |

---

### Q37. Global vs local rate limit?

Answer:

| Global                  | Local     |
|-------------------------|-----------|
| Accurate across cluster | Faster    |
| More complex            | May drift |

---

## 23. Anti-Patterns

### Q38. Dangerous anti-patterns?

Answer:

- per-node local counters only
  - non-atomic updates
  - no TTL cleanup
  - no hot-key mitigation
  - no fallback
  - DB-only counters at huge scale
- 

## 24. FAANG Trap Questions

### Q39. Can Redis be source of truth?

Answer:

Usually no (for business truth), but yes for limiter counters.

Subtle trap.

---

### Q40. Which is best: Token bucket or sliding window?

Answer:

Depends on use case.

APIs → Token Bucket

Security/login → Sliding Window

---

#### **Q41. Can fail-open be dangerous?**

**Answer:**

Yes. Security-sensitive paths may need fail-closed.

---

#### **Final Architect Summary**

Best design:

**API Gateway + Rate Limit Service + Redis Cluster + Token Bucket/Sliding Window + Sharded Counters + Config Service + Metrics + Fallback + Multi-region Strategy**

**Azure:**

APIM + Redis + AKS + Front Door + App Config + Monitor

**AWS:**

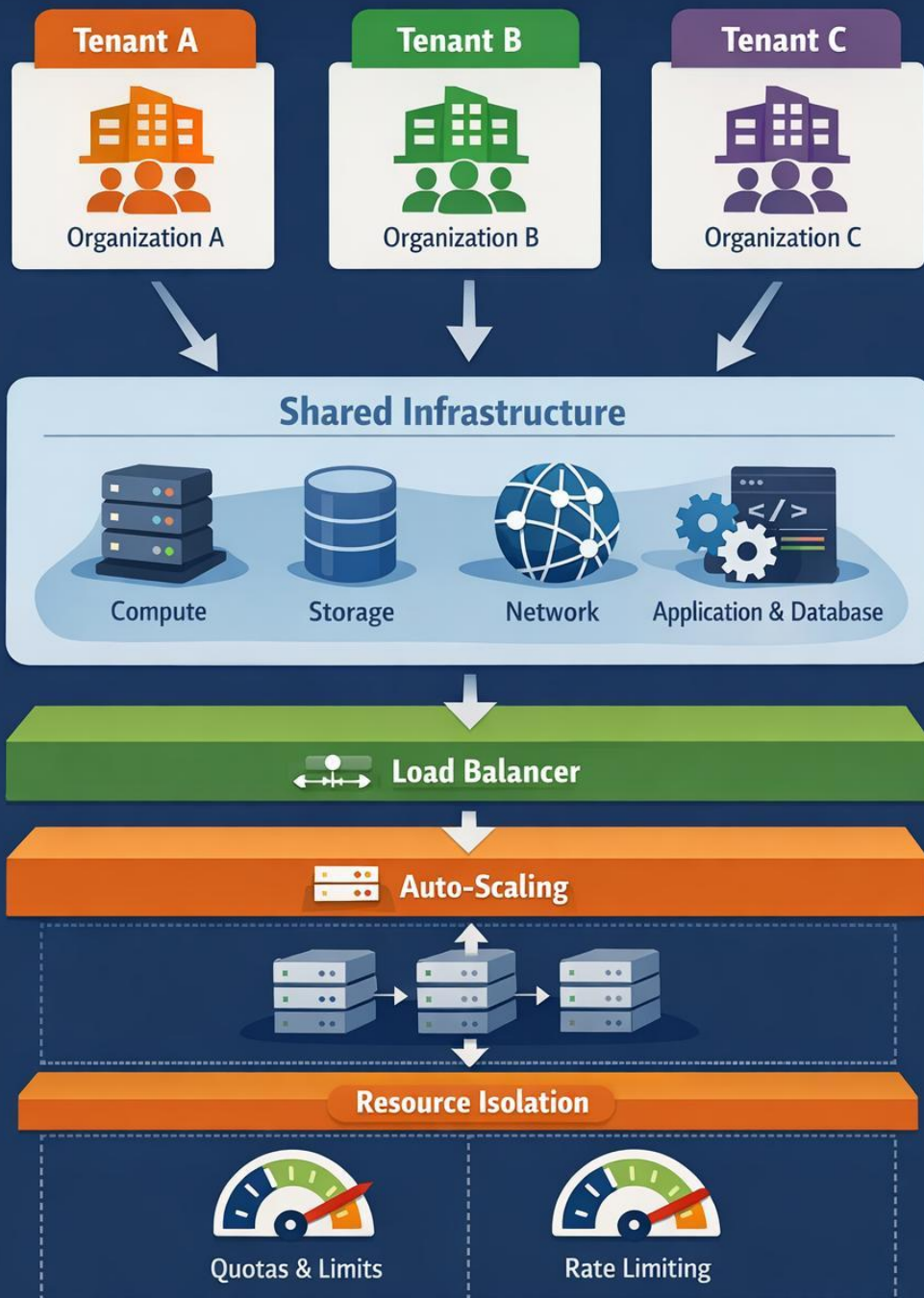
API Gateway + ElastiCache + EKS + CloudFront + AppConfig + CloudWatch

This is a strong **Cloud / Security / FAANG / Distributed Systems Architect** answer.

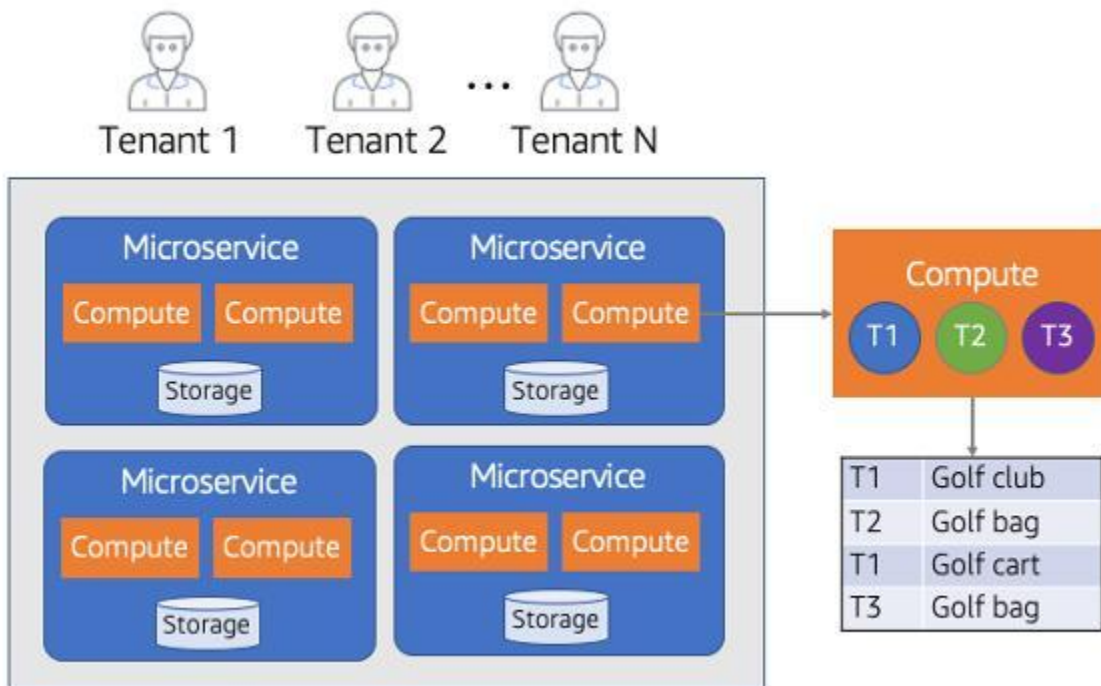
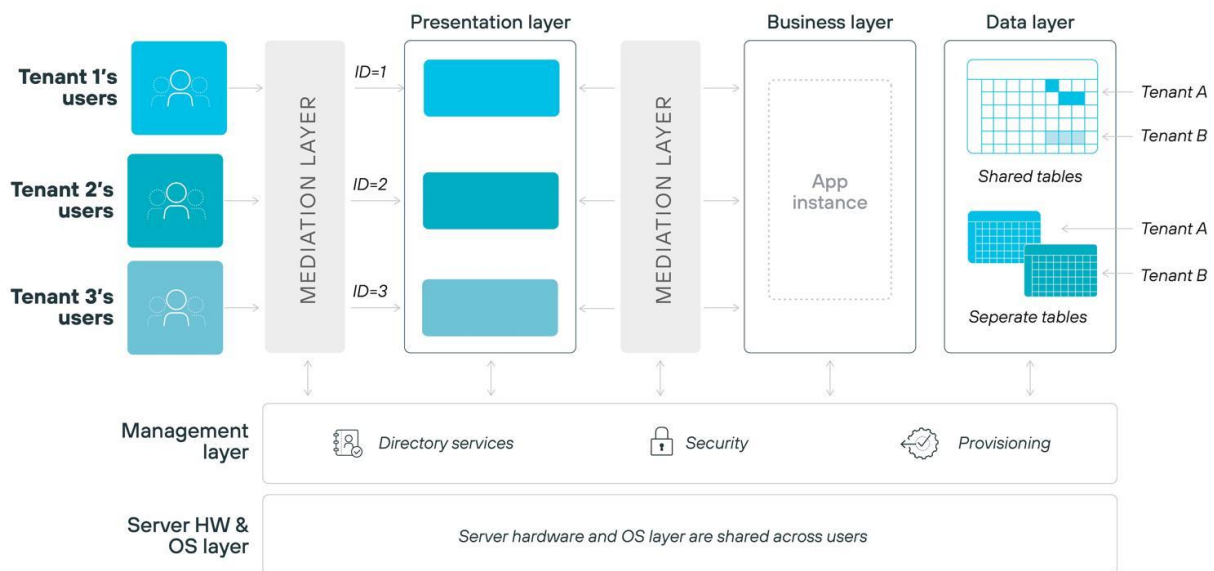
## Design 13 → Multi-Tenant SaaS Platform

mailto:kirank\_tekawade@yahoo.co.in

# Multi-Tenant Architecture



## Multi-tenant architecture



7

This answers:

**"How do you design a SaaS platform serving thousands of enterprise tenants securely and independently?"**



---

## Q1. What is Multi-Tenant SaaS Platform?

### Answer:

A platform where **multiple customers (tenants)** share infrastructure, while keeping data/config/security logically isolated.

Examples:

- CRM
- HR SaaS
- ERP
- Banking portal
- Collaboration platforms

Huge enterprise topic.

---

## Q2. Core requirements?

### Answer:

#### Functional Requirements

- tenant onboarding
- authentication
- RBAC
- tenant-specific config
- billing
- quotas
- usage metering
- reporting
- backups
- tenant customization
- tenant suspension/deletion

#### Non-Functional Requirements

- strict isolation

- scalability
  - HA
  - fault tolerance
  - zero trust
  - cost optimization
  - compliance
  - observability
- 

## 1. Capacity Estimation

Assume:

- 10K enterprise tenants
- 100M users
- 1M concurrent sessions
- highly uneven tenant load

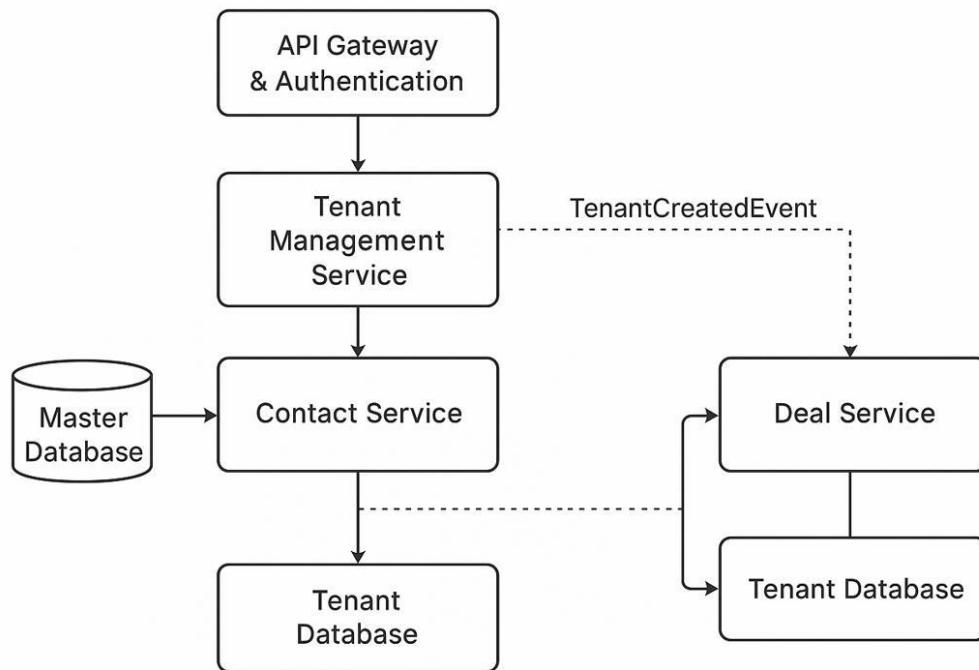
Challenges:

- tenant isolation
- noisy neighbors
- custom configs
- billing
- data residency

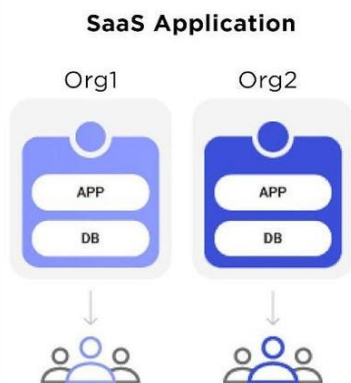
Huge SaaS architect topic.

---

## 2. High-Level Architecture

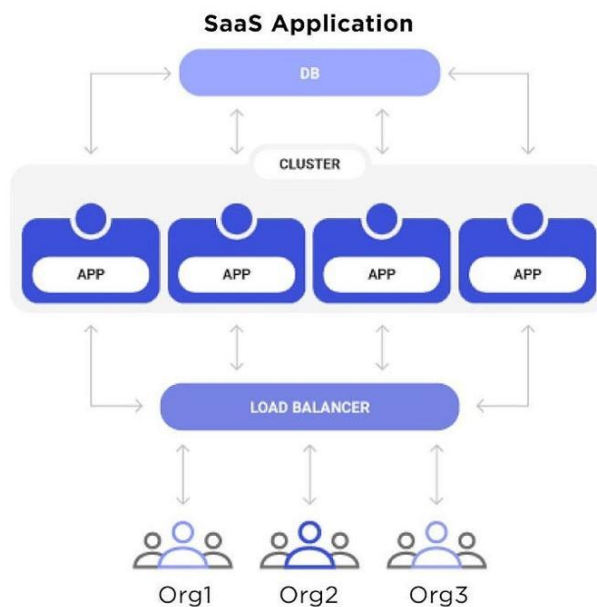


## SINGLE-TENANT



VS

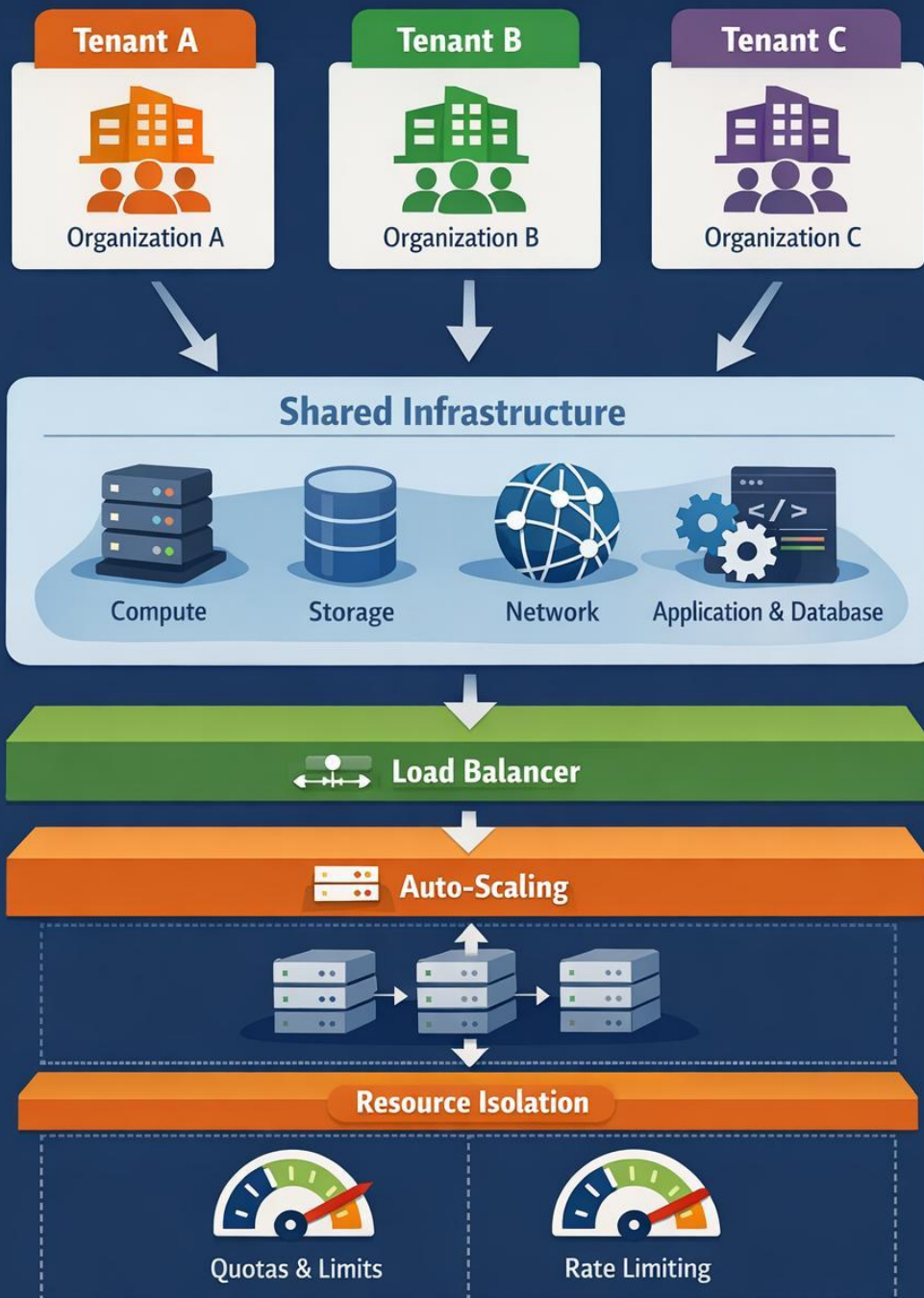
## MULTI-TENANT



RELEVANT

relevant.software

# Multi-Tenant Architecture



**Q3. Core components?****Answer:**

Usually:

- API Gateway
  - Identity/Auth Service
  - Tenant Resolution Service
  - User/Profile Service
  - Config Service
  - Billing Service
  - Quota Service
  - Usage Metering Service
  - Notification Service
  - Tenant Admin Portal
  - Data Isolation Layer
  - Event Bus
  - Analytics
- 

**3. Tenant Resolution****Q4. What is Tenant Resolution Service?****Answer:**

Identifies which tenant request belongs to.

Common methods:

- subdomain → abc.company.com
- JWT claim
- header
- custom domain

Very common interview topic.

#### **Q5. Why separate tenant resolution?**

**Answer:**

Centralizes:

- routing
  - policy
  - isolation
  - config lookup
- 

#### **4. Authentication / Authorization**

##### **Q6. Auth model?**

**Answer:**

Usually:

- OAuth2
- OIDC
- SSO
- SAML
- JWT
- RBAC

Enterprise-heavy.

---

##### **Q7. Why tenant-aware auth?**

**Answer:**

Same user may belong to multiple tenants.

Must scope identity.

Critical.

---

#### **5. Tenant Isolation (Critical)**

##### **Q8. Types of tenant isolation?**

**Answer:**

Three common:

1. Shared App + Shared DB
2. Shared App + Separate DB per tenant
3. Dedicated stack per tenant

Huge architect favorite.

---

#### **Q9. Shared DB model?**

**Answer:**

Single DB, tenantId in tables.

Pros:

- cheap
- easy scale

Cons:

- weaker isolation
  - noisy queries
- 

#### **Q10. DB-per-tenant model?**

**Answer:**

Separate DB per tenant.

Pros:

- stronger isolation
- easier restore
- compliance

Cons:

- operational overhead

Very common SaaS choice.

---

#### **Q11. Dedicated stack model?**

**Answer:**

Tenant gets dedicated app + DB + infra.

Used for:

- premium tenants
- regulated banking
- strict compliance

Highest isolation.

---

#### **Q12. Best model?**

**Answer:**

Usually hybrid.

Small tenants → shared

Large/regulated → dedicated

Strong architect answer.

---

#### **6. Noisy Neighbor Problem**

##### **Q13. What is noisy neighbor?**

**Answer:**

One tenant consumes excessive shared resources.

Huge SaaS topic.

---

##### **Q14. How solve noisy neighbors?**

**Answer:**

Use:

- quotas
- rate limiting
- workload isolation
- sharding
- autoscale
- tenant throttling



---

## 7. Config / Customization

### Q15. Why Config Service?

**Answer:**

Each tenant may have:

- branding
- limits
- feature flags
- workflows
- locale

---

### Q16. Hard-code tenant logic?

**Answer:**

No.

Use config-driven design.

Huge trap.

---

## 8. Quotas / Metering / Billing

### Q17. Why Quota Service?

**Answer:**

Enforce:

- API calls
- storage
- users
- compute
- premium features

---

### Q18. Why Usage Metering?

**Answer:**

Needed for:

- billing
  - analytics
  - plan enforcement
- 

#### Q19. Billing model examples?

Answer:

- per user
  - per API
  - storage-based
  - compute-based
  - tiered plans
- 

### 9. Data Layer

#### Q20. DB choices?

Answer:

Usually polyglot:

- Relational → users/billing/core
  - NoSQL → config/metadata
  - Redis → cache/session/quota
  - Data lake → analytics
  - Object storage → files
- 

#### Q21. Why include tenantId everywhere?

Answer:

Isolation + auditing + partitioning.

Critical.

---

### 10. Event Bus

## Q22. Why Event Bus?

**Answer:**

Used for:

- onboarding
  - billing
  - quota
  - notifications
  - audit
  - analytics
- 

## 11. Tenant Onboarding

### Q23. Onboarding flow?

**Answer:**

Step 1 → Register tenant

Step 2 → Provision config

Step 3 → create auth realm

Step 4 → assign plan

Step 5 → DB provisioning

Step 6 → admin creation

Step 7 → activation

Huge enterprise workflow.

---

## 12. Security / Zero Trust

### Q24. Security risks?

**Answer:**

- tenant data leak
- privilege escalation
- shared-secret abuse
- misrouting
- RBAC bugs

Huge architect topic.

---

**Q25. Security controls?**

**Answer:**

Need:

- zero trust
- tenant-scoped JWT
- RBAC
- encryption
- audit
- key isolation
- secret vault
- rate limiting
- WAF

---

**13. Data Residency / Compliance****Q26. Why data residency important?**

**Answer:**

Some tenants require region-specific storage.

Example:

EU, India, regulated banking.

Enterprise-heavy.

---

**14. HA / DR****Q27. HA design?**

**Answer:**

Use:

- stateless apps
- tenant-aware routing
- multi-region

- DB replication
  - failover
- 

#### **Q28. DR strategy?**

**Answer:**

- per-tenant backup
  - restore
  - geo failover
  - tested recovery
- 

#### **15. Scale**

#### **Q29. How scale to 10K+ tenants?**

**Answer:**

Use:

- shard by tenant
  - stateless APIs
  - config cache
  - autoscaling
  - queue-driven workloads
- 

#### **Q30. Hot tenant problem?**

**Answer:**

One enterprise tenant becomes huge.

Need:

- dedicated infra
  - shard split
  - throttling
  - isolation
-

## 16. Observability

### Q31. Why tenant-aware observability?

**Answer:**

Need per-tenant:

- latency
- cost
- failures
- usage
- abuse

Very important.

---

## 17. Bottlenecks

### Q32. Common hotspots?

**Answer:**

- hot tenants
  - shared DB contention
  - billing spikes
  - config cache misses
  - onboarding storms
- 

## 18. Azure Mapping (.NET-focused)

### Q33. Azure services?

**Answer:**

| Need  | Azure                 |
|-------|-----------------------|
| APIs  | App Service / AKS     |
| Auth  | Entra ID / B2C        |
| DB    | Azure SQL / Cosmos DB |
| Cache | Azure Redis           |

| <b>Need</b>       | <b>Azure</b>             |
|-------------------|--------------------------|
| Event bus         | Service Bus / Event Hubs |
| Files             | Blob Storage             |
| Config            | App Configuration        |
| Billing analytics | Synapse                  |
| Monitoring        | App Insights             |
| Global routing    | Front Door               |
| Secrets           | Key Vault                |

---

## 19. AWS Mapping (.NET-focused)

### Q34. AWS services?

**Answer:**

| <b>Need</b>    | <b>AWS</b>          |
|----------------|---------------------|
| APIs           | ECS / EKS           |
| Auth           | Cognito             |
| DB             | RDS / DynamoDB      |
| Cache          | ElastiCache         |
| Event bus      | SNS / SQS / Kinesis |
| Files          | S3                  |
| Config         | AppConfig           |
| Analytics      | Athena / Redshift   |
| Monitoring     | CloudWatch          |
| Global routing | Route53             |
| Secrets        | Secrets Manager     |

---

## 20. Trade-offs

### Q35. Shared DB vs DB-per-tenant vs dedicated?

**Answer:**

| Model         | Best For             |
|---------------|----------------------|
| Shared        | small tenants / cost |
| DB-per-tenant | enterprise balance   |
| Dedicated     | regulated / premium  |
| Must know.    |                      |

---

## 21. Anti-Patterns

### Q36. Dangerous anti-patterns?

**Answer:**

- missing tenantId filters
- shared secrets across tenants
- hard-coded tenant behavior
- no quota isolation
- no per-tenant audit
- shared admin privileges

Huge SaaS traps.

---

## 22. FAANG / Architect Trap Questions

### Q37. Can cache be shared?

**Answer:**

Yes, but isolate keys.

---

### Q38. Can one tenant bring whole platform down?

**Answer:**

Should not. Solve noisy neighbor.

---



**Q39. Is DB-per-tenant always best?**

**Answer:**

No. Depends on scale/cost/compliance.

---

### **Final Architect Summary**

Best design:

**API Gateway + Tenant Resolution + Tenant-aware Auth + Config + Quotas + Billing + Event Bus + Polyglot DB + Isolation Strategy + Zero Trust + Multi-region HA**

**Azure:**

AKS + Entra + Azure SQL/Cosmos + Redis + Service Bus + Key Vault + Front Door

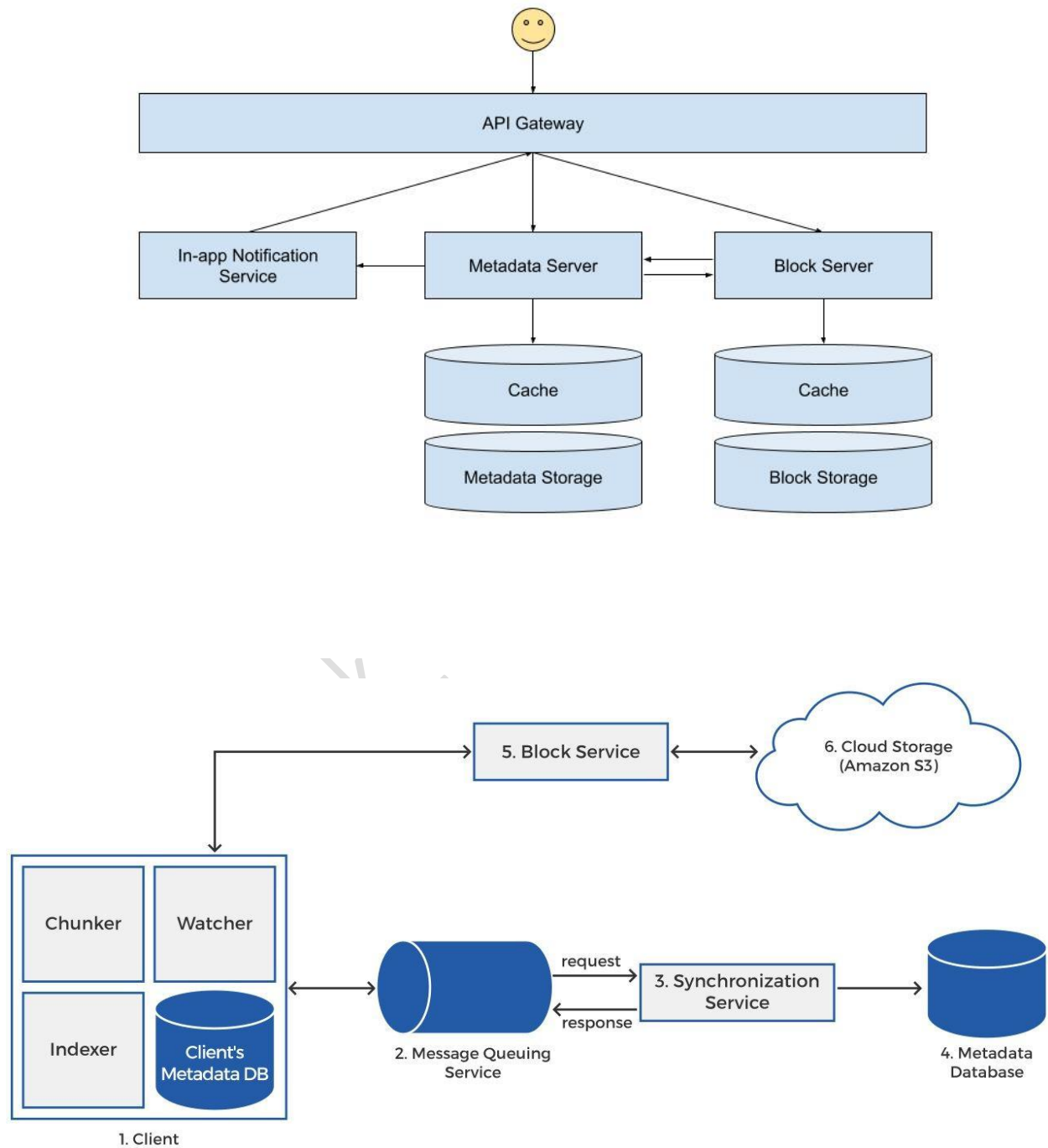
**AWS:**

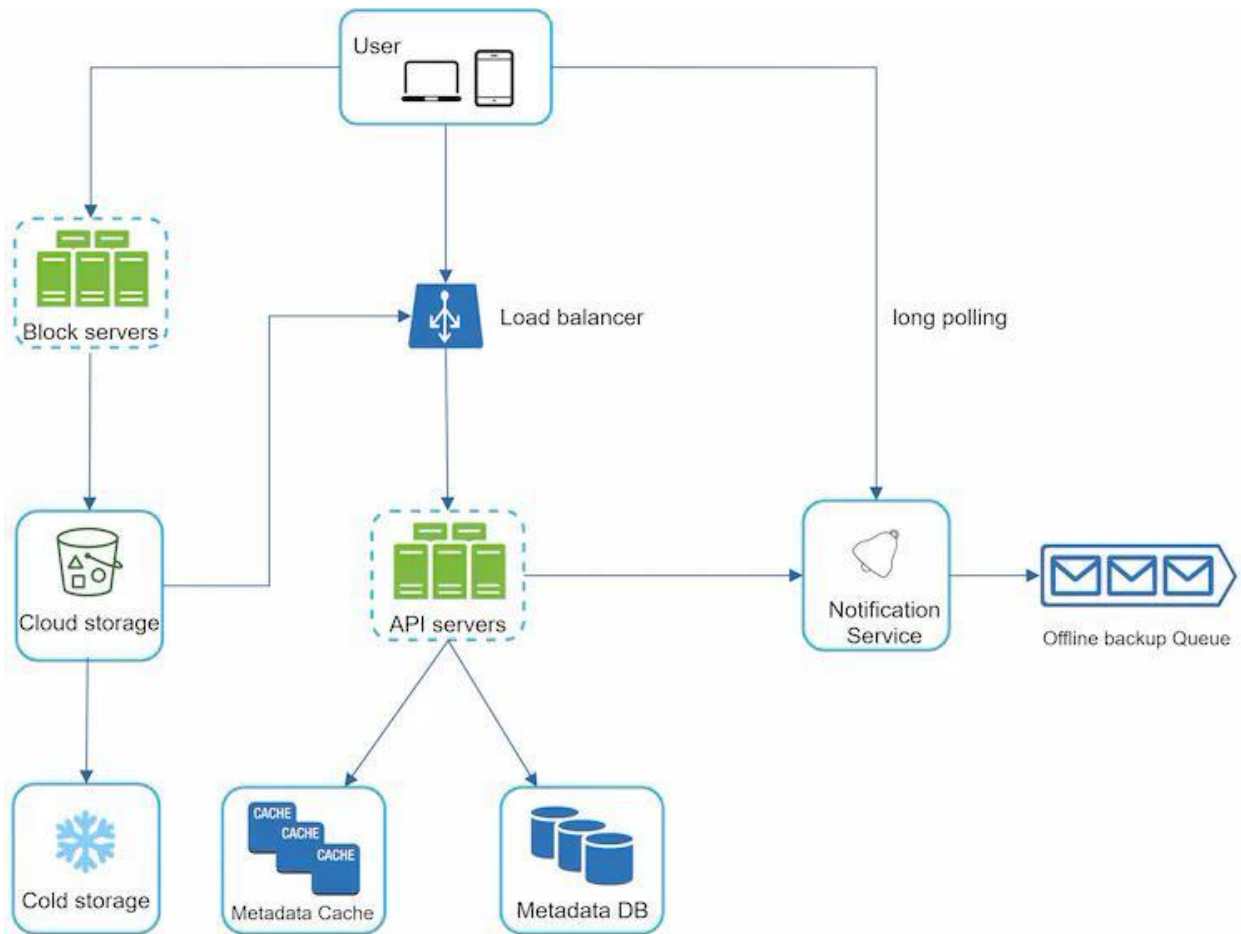
EKS + Cognito + RDS/Dynamo + ElastiCache + SNS/SQS + Secrets Manager + Route53

This is a strong **SaaS / Cloud / Enterprise / Accenture Architect answer.**

## Design 14 → File Storage System (Dropbox / Google Drive)

High level design of a personal cloud storage system





7

This answers:

**“How do you design a cloud file storage system like Dropbox / Google Drive handling billions of files, syncing across devices, and ensuring durability?”**

**Q1. What are core requirements of File Storage System?**

**Answer:**

**Functional Requirements**

- upload file
- download file
- sync across devices
- versioning
- sharing

- permissions
- folder hierarchy
- delete/restore
- deduplication
- preview
- audit
- notifications

### **Non-Functional Requirements**

- billions of files
- durability
- HA
- fault tolerance
- low latency
- large file handling
- global access
- consistency
- scalability

---

### **1. Capacity Estimation**

Assume:

- 1B users
- 100B+ files
- PB/EB scale storage
- millions uploads/day
- multi-device sync

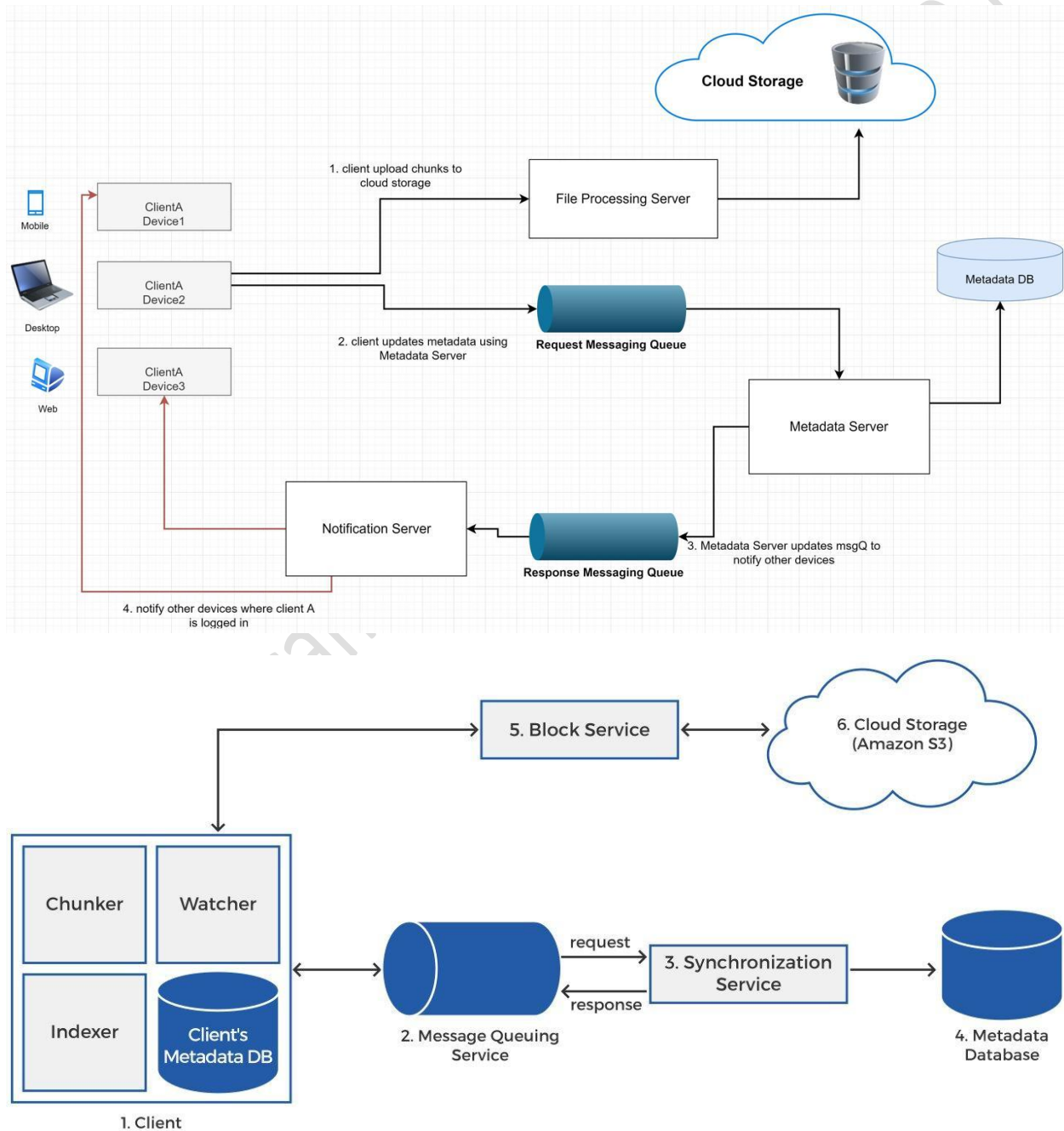
Challenges:

- chunking
- metadata scale
- duplicate files

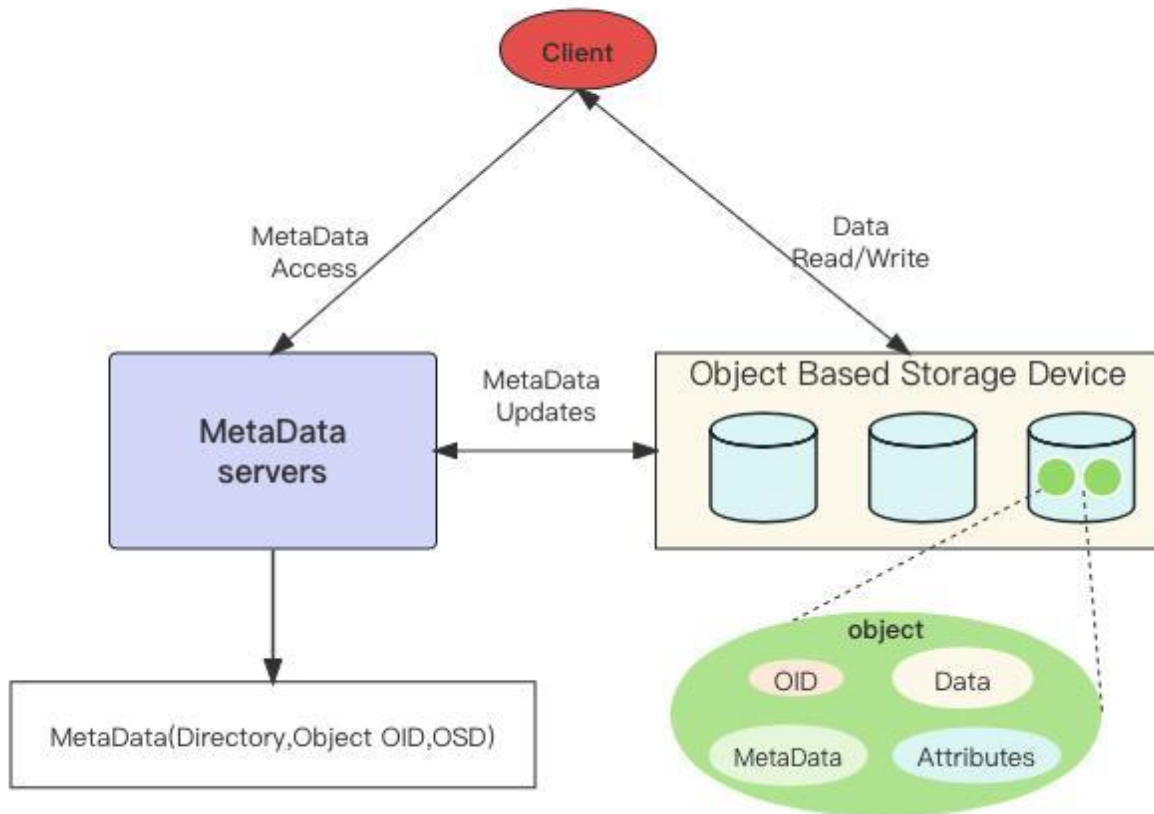
- hot files
- consistency
- conflict resolution

Huge FAANG storage topic.

## 2. High-Level Architecture



## S3 Architecture



6

### Q2. Core components?

**Answer:**

Usually:

- API Gateway
- Upload Service
- Download Service
- Chunking Service
- Metadata Service
- Sync Engine
- Sharing/Permission Service

- Versioning Service
  - Deduplication Service
  - Notification Service
  - Search Service
  - Cache Layer
  - Object Storage
  - Event Bus
- 

### 3. Upload Flow

#### Q3. File upload flow?

**Answer:**

Step 1 → User uploads file  
Step 2 → Auth check  
Step 3 → Split into chunks  
Step 4 → Hash chunks  
Step 5 → Dedup check  
Step 6 → Store chunks  
Step 7 → Metadata write  
Step 8 → Publish sync event  
Step 9 → Notify devices

Huge architect flow.

---

### 4. Why Chunking? (Critical)

#### Q4. What is file chunking?

**Answer:**

Breaking file into smaller pieces.

Example:

1GB file → multiple chunks.

Huge storage topic.

---

#### Q5. Why chunking useful?

**Answer:**

Benefits:

- resumable uploads
- parallel upload/download
- dedupe
- better retry
- large-file scale

Very common FAANG question.

---

## 5. Deduplication

### Q6. What is deduplication?

**Answer:**

Avoid storing identical data multiple times.

Huge storage topic.

---

### Q7. How dedupe works?

**Answer:**

Usually:

- hash chunk/file
  - compare existing hash
  - reuse chunk pointer
- 

### Q8. File-level vs chunk-level dedupe?

**Answer:**

| File-level | Chunk-level |
|------------|-------------|
|------------|-------------|

|                      |                     |
|----------------------|---------------------|
| Whole file duplicate | Partial chunk reuse |
|----------------------|---------------------|

Chunk-level usually stronger.

---

## 6. Metadata



### Q9. What is Metadata Service?

**Answer:**

Stores:

- fileId
- owner
- size
- path
- version
- checksum
- permissions
- timestamps

Critical.

---

### Q10. Why separate metadata from binary?

**Answer:**

Binary files huge.

Metadata query-heavy.

Best separation.

---

## 7. Download Flow

### Q11. File download flow?

**Answer:**

Step 1 → Request file

Step 2 → Auth/permission check

Step 3 → Metadata lookup

Step 4 → Chunk retrieval

Step 5 → Reassemble/stream

Step 6 → CDN/cache support

---

## 8. Sync Engine (Very Important)

### Q12. What is Sync Engine?

**Answer:**

Keeps file state aligned across devices.

Huge Dropbox topic.

---

**Q13. Sync flow?**

**Answer:**

Change → Event → Sync Engine → Target devices → Pull/update.

---

**Q14. Why event-driven sync?**

**Answer:**

Supports:

- decoupling
  - scale
  - async updates
  - multi-device fan-out
- 

## **9. Conflict Resolution**

**Q15. What if two devices edit same file?**

**Answer:**

Conflict.

Huge distributed systems topic.

---

**Q16. How solve sync conflict?**

**Answer:**

Common:

- version check
- last-write-wins
- merge
- create conflict copy

Very common.

---

## 10. Versioning

### Q17. Why file versioning?

**Answer:**

Needed for:

- restore
  - audit
  - rollback
  - accidental delete
- 

### Q18. Soft delete vs hard delete?

**Answer:**

Usually soft delete first.

---

## 11. Sharing / Permissions

### Q19. Sharing models?

**Answer:**

- private
  - shared user
  - group share
  - public link
  - expiring link
- 

### Q20. Why separate Permission Service?

**Answer:**

Complex ACL/RBAC.

---

## 12. DB Design

### Q21. Data stores?

**Answer:**

Usually polyglot:

- Object Storage → file chunks
  - Relational/NoSQL → metadata
  - Redis → cache
  - Search Index → file search
  - Event Bus → sync events
- 

### Q22. Why object storage?

**Answer:**

Large binary durability + scale.

Huge cloud topic.

---

### 13. Search

#### Q23. What can be searched?

**Answer:**

- file name
  - tags
  - metadata
  - content index
  - owner
- 

### 14. Caching / CDN

#### Q24. What to cache?

**Answer:**

- hot files
- metadata
- permissions

- previews
  - download links
- 

#### **Q25. Hot file problem?**

**Answer:**

Viral/shared file → huge reads.

CDN + cache.

---

#### **15. Reliability / Durability**

##### **Q26. Durability strategy?**

**Answer:**

Use:

- multi-copy replication
- erasure coding
- checksums
- restore

Huge storage architect topic.

---

##### **Q27. Why checksums?**

**Answer:**

Detect corruption.

---

#### **16. HA / DR**

##### **Q28. HA design?**

**Answer:**

Use:

- stateless APIs
- replicated metadata DB
- geo object replication

- cache clusters
  - failover
- 

**Q29. DR strategy?**

**Answer:**

- cross-region replication
  - backups
  - restore test
  - metadata replay
- 

**17. Scale**

**Q30. How scale to billions of files?**

**Answer:**

Use:

- chunking
  - sharding
  - metadata partitioning
  - object storage
  - CDN
  - event-driven sync
- 

**Q31. Hotspots?**

**Answer:**

- metadata DB
  - hot files
  - permission checks
  - sync storms
  - large uploads
-

## 18. Security

### Q32. Security controls?

**Answer:**

Need:

- auth
- RBAC/ACL
- encryption at rest
- TLS
- signed URLs
- malware scan
- DLP
- audit trail

Huge enterprise topic.

---

## 19. Azure Mapping (.NET-focused)

### Q33. Azure services?

**Answer:**

| Need               | Azure                    |
|--------------------|--------------------------|
| API                | App Service / AKS        |
| Object Storage     | Blob Storage             |
| Event bus          | Service Bus / Event Hubs |
| Metadata DB        | Cosmos DB / SQL          |
| Cache              | Azure Redis              |
| Search             | Azure AI Search          |
| Auth               | Entra ID                 |
| File scan/workflow | Functions                |
| Monitoring         | App Insights             |

| Need           | Azure      |
|----------------|------------|
| Global routing | Front Door |
| Secrets        | Key Vault  |

---

## 20. AWS Mapping (.NET-focused)

### Q34. AWS services?

Answer:

| Need           | AWS                  |
|----------------|----------------------|
| API            | ECS / EKS            |
| Object Storage | S3                   |
| Event bus      | SNS / SQS / Kinesis  |
| Metadata DB    | DynamoDB / RDS       |
| Cache          | ElastiCache          |
| Search         | OpenSearch           |
| Auth           | Cognito              |
| File workflows | Lambda               |
| Monitoring     | CloudWatch           |
| Global routing | Route53 / CloudFront |
| Secrets        | Secrets Manager      |

---

## 21. Trade-offs

### Q35. Chunking vs full-file upload?

Answer:

#### Chunking Full File

resilient   simple

parallel   weak retry



## Chunking Full File

dedupe no partial reuse

Chunking wins at scale.

---

### Q36. Strong vs eventual consistency?

**Answer:**

Strong:

- metadata write
- permissions

Eventual:

- sync propagation
  - search indexing
  - analytics
- 

## 22. Anti-Patterns

### Q37. Dangerous anti-patterns?

**Answer:**

- storing files in DB
  - no chunking
  - no dedupe
  - no versioning
  - no conflict handling
  - sync in tight blocking path
- 

## 23. FAANG Trap Questions

### Q38. Can DB store binaries?

**Answer:**

Usually no.

---

**Q39. Can metadata and file store be same?**

**Answer:**

Possible, but usually separated.

---

**Q40. Is last-write-wins always safe?**

**Answer:**

No.

Can lose edits.

Huge trap.

---

### **Final Architect Summary**

Best design:

**API Gateway + Upload/Download + Chunking + Dedup + Metadata + Sync Engine + Versioning + Permissions + Object Storage + Event Bus + CDN + Multi-region HA**

**Azure:**

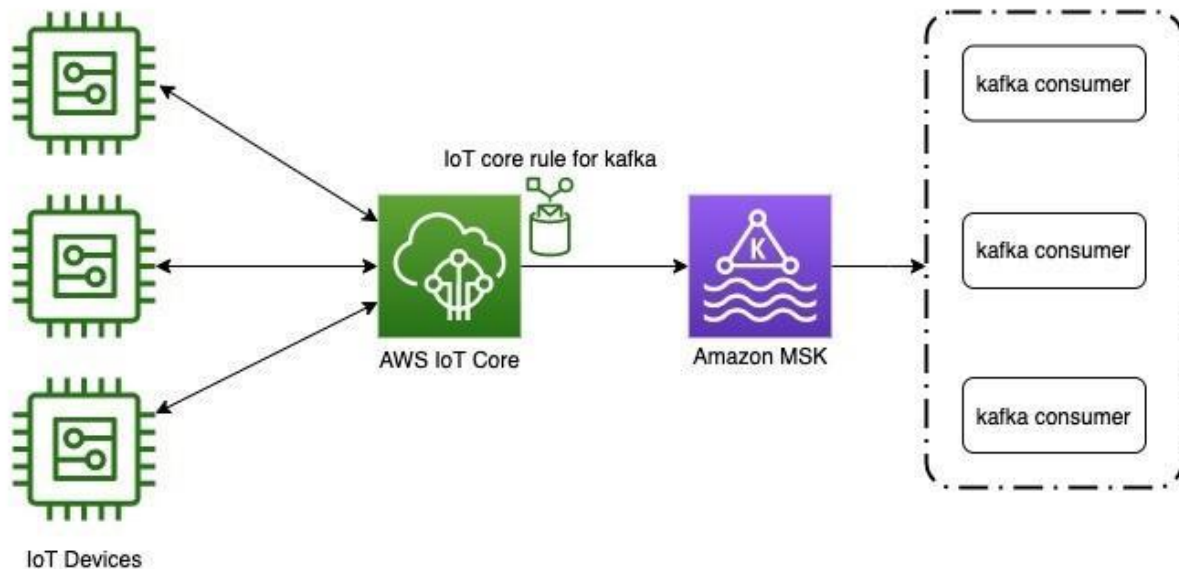
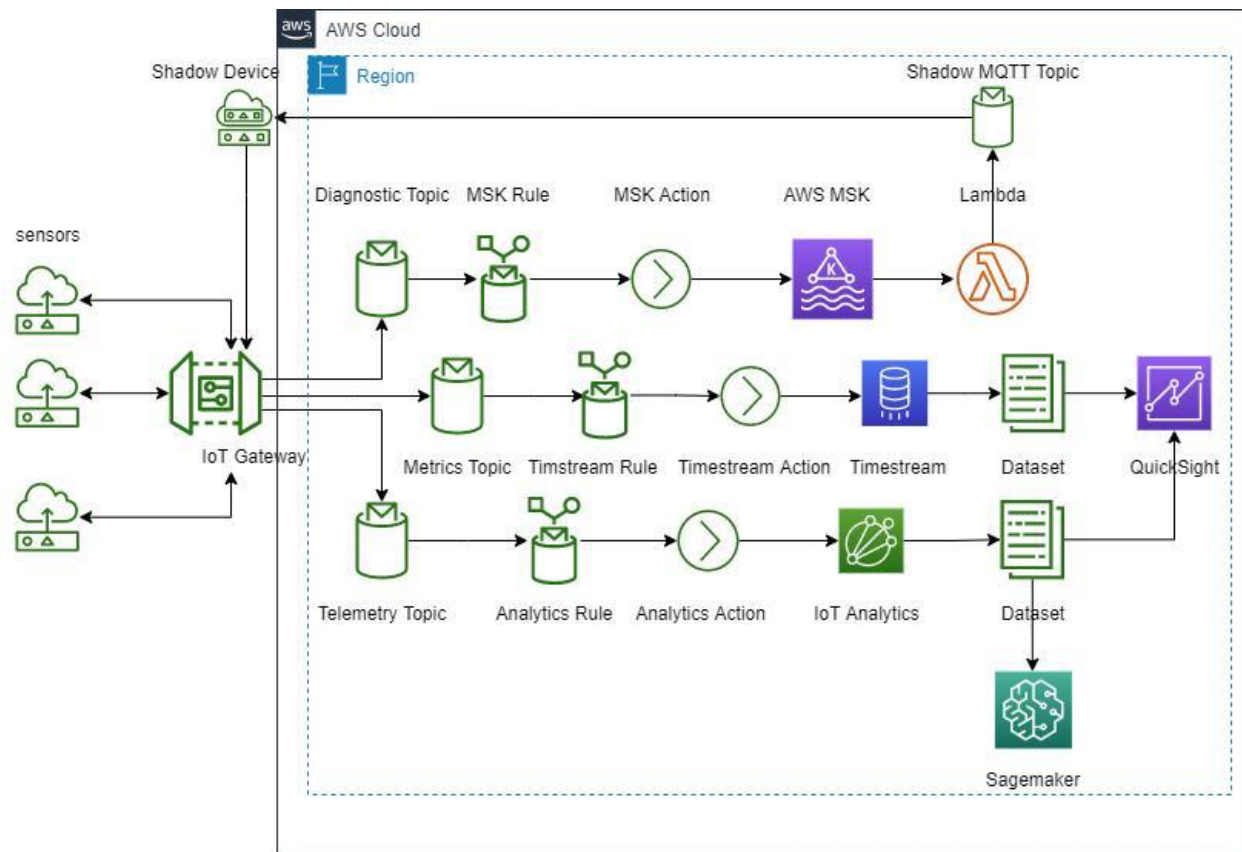
AKS + Blob + Cosmos + Redis + Service Bus + AI Search + Key Vault

**AWS:**

EKS + S3 + DynamoDB + ElastiCache + SNS/SQS + OpenSearch + Secrets Manager

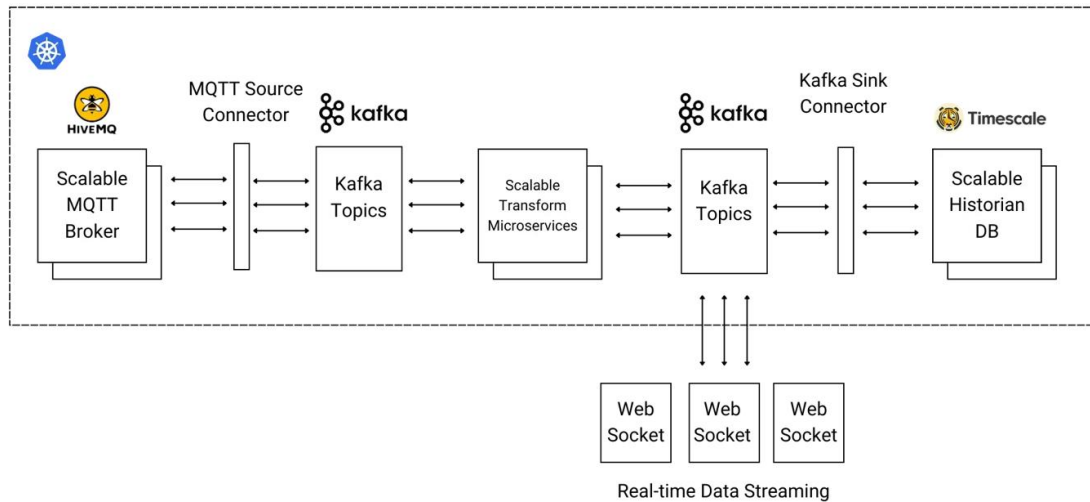
This is a strong **Dropbox / Google Drive / Cloud / Storage / FAANG Architect answer.**

## Design 15 → IoT Event Platform (10M–1B events/day)



## Architecture

Cloud



6

This answers:

**“How do you design an IoT Event Platform handling millions of devices and 10M–1B events/day?”**

### Q1. What is IoT Event Platform?

**Answer:**

A system that ingests, processes, stores, and reacts to telemetry/events from connected devices.

Examples:

- GPS trackers
- smart meters
- vehicles
- industrial sensors
- healthcare devices
- temperature sensors

Huge architect topic.

### Q2. Core requirements?

**Answer:**

### **Functional Requirements**

- device registration
- telemetry ingestion
- real-time stream processing
- alerting
- rules engine
- dashboards
- command/control
- digital twin
- time-series storage
- replay
- retention
- analytics

### **Non-Functional Requirements**

- 10M–1B events/day
- HA
- low latency
- fault tolerance
- ordering (where needed)
- secure ingestion
- geo scale
- horizontal scale

---

## **1. Capacity Estimation**

Assume:

- 10M devices
- avg 1 msg / 5 sec
- spikes during reconnects

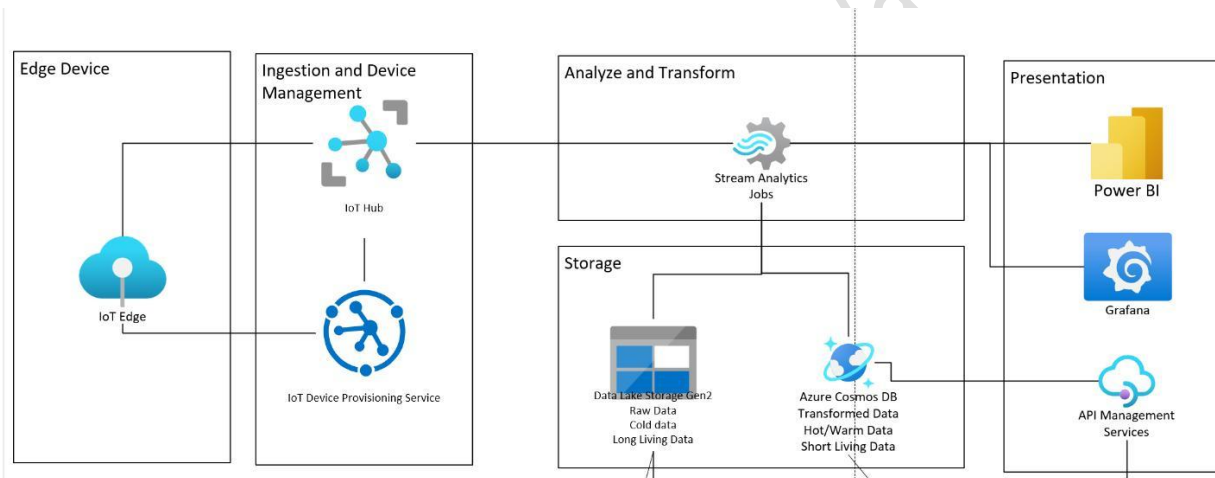
- TB/PB yearly data

Challenges:

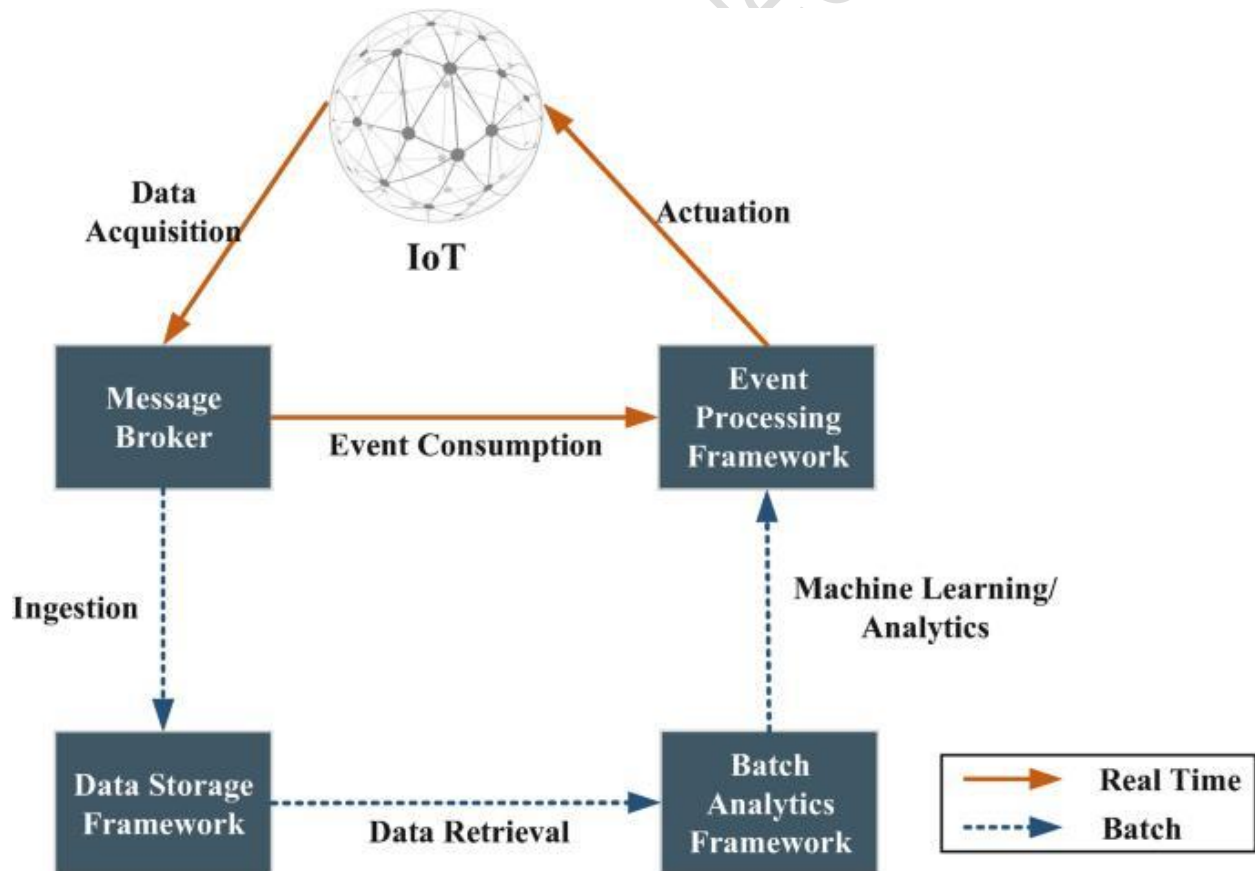
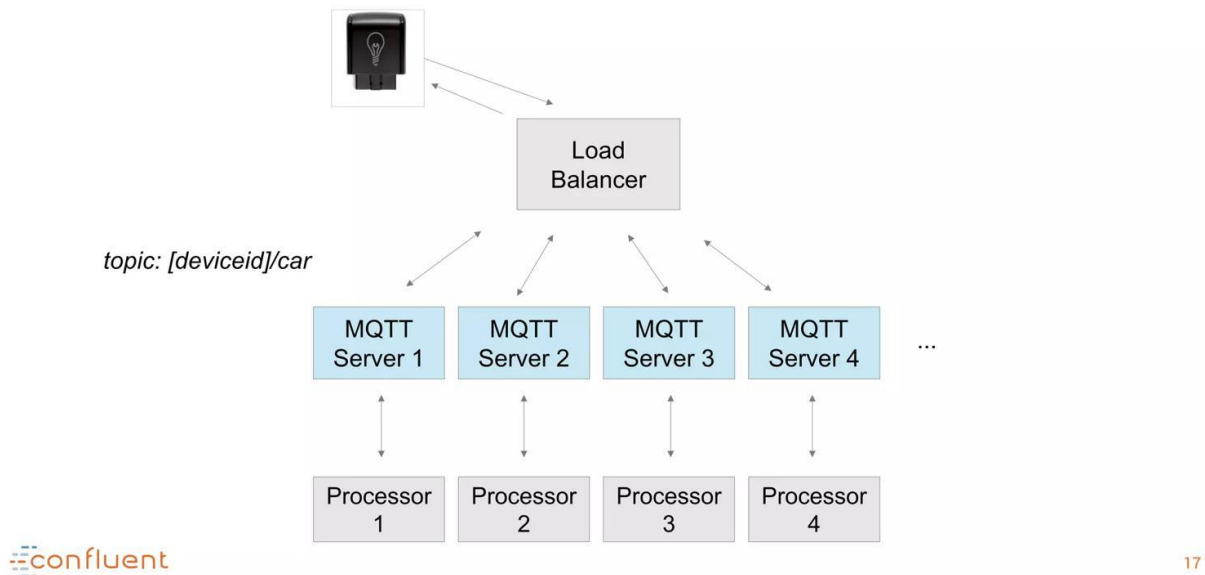
- burst reconnect storms
- partition imbalance
- time-series writes
- hot devices
- retention cost

Huge distributed system topic.

## 2. High-Level Architecture



## MQTT Architecture (large scale)



7

Q3. Core components?

**Answer:**

Usually:

- Device Gateway / MQTT Broker
  - Device Registry
  - Auth/Certificate Service
  - Ingestion Service
  - Event Bus
  - Stream Processing Engine
  - Rules Engine
  - Alert Service
  - Digital Twin Service
  - Time-Series DB
  - Cold Storage / Data Lake
  - Command Service
  - Analytics / Dashboard
- 

### **3. Device Connectivity**

#### **Q4. Why MQTT common in IoT?**

**Answer:**

Lightweight pub/sub protocol.

Great for:

- low bandwidth
- unstable networks
- small devices

Huge IoT favorite.

---

#### **Q5. MQTT vs HTTP?**

**Answer:**



## MQTT

## HTTP

lightweight persistent pub/sub request-response

better for devices

simpler for APIs

Must know.

---

### Q6. What if device offline?

**Answer:**

Queue/retry/store-forward depending on device type.

---

## 4. Device Registry

### Q7. What is Device Registry?

**Answer:**

Stores:

- deviceId
- firmware
- tenant
- cert
- status
- metadata

Critical.

---

## 5. Ingestion Flow

### Q8. End-to-end event flow?

**Answer:**

Step 1 → Device sends telemetry  
Step 2 → MQTT Broker receives  
Step 3 → Auth/cert validation  
Step 4 → Ingestion Service  
Step 5 → Event Bus publish  
Step 6 → Stream processing

Step 7 → Rules evaluation

Step 8 → alert/storage/dashboard

Huge architect flow.

---

## **6. Event Bus (Critical)**

### **Q9. Why Kafka/Event Bus?**

**Answer:**

Needed for:

- decoupling
- replay
- scaling
- buffering
- durability

Huge event-driven topic.

---

### **Q10. Partition strategy?**

**Answer:**

Usually:

- deviceId
- region
- tenant
- sensor group

Very important.

---

### **Q11. Why deviceId partition?**

**Answer:**

Preserves per-device ordering.

Huge trap.

---

## 7. Stream Processing

### Q12. Why Stream Processing Engine?

**Answer:**

Real-time transforms:

- aggregate
  - enrich
  - detect anomalies
  - route
  - filter
- 

### Q13. Examples?

**Answer:**

- avg temperature
  - GPS anomaly
  - threshold breach
  - fraud/spoof detection
- 

## 8. Rules Engine

### Q14. What is Rules Engine?

**Answer:**

Evaluates business logic.

Examples:

IF temp > 90 → alert

IF battery < 10% → notify

Very common enterprise pattern.

---

### Q15. Why separate rules engine?

**Answer:**

Config-driven policies, not hard-coded logic.

---

## 9. Alerting

### Q16. Alert types?

**Answer:**

- threshold
- anomaly
- offline
- battery
- security
- region-wide alerts

---

## 10. Digital Twin

### Q17. What is Digital Twin?

**Answer:**

Virtual model/state of physical device.

Huge Azure IoT topic.

---

### Q18. Why Digital Twin useful?

**Answer:**

Tracks:

- current state
- config
- shadow state
- command sync

Very common architect concept.

---

## 11. Command / Control

### Q19. What is command path?

**Answer:**

Platform → Broker → Device.

Examples:

- reboot
  - firmware update
  - config change
- 

## 12. Time-Series DB

### Q20. Why time-series DB?

**Answer:**

Optimized for:

- timestamp writes
- retention
- rollups
- range queries

Huge data topic.

---

### Q21. What data stores?

**Answer:**

Usually:

- Time-series DB → telemetry
  - Relational → registry/config
  - Redis → hot state
  - Data lake → cold history
  - Event bus → streaming
- 

## 13. Retention / Cold Storage

### Q22. Why cold storage?

**Answer:**

IoT data grows huge.

Move old data cheaper.

Hot/Warm/Cold strategy.

---

### **Q23. Retention strategy?**

**Answer:**

Example:

- 7 days hot
- 90 days warm
- years cold archive

Strong architect answer.

---

### **14. Hot Partition / Hot Device**

#### **Q24. What is hot partition?**

**Answer:**

One partition overloaded.

Huge distributed topic.

---

#### **Q25. How solve?**

**Answer:**

Use:

- rebalance
  - key strategy
  - shard spreading
  - buffering
- 

### **15. Reconnect Storm**

#### **Q26. What is reconnect storm?**

**Answer:**

Millions of devices reconnect after outage.

Huge IoT trap.

---

**Q27. How handle?**

**Answer:**

- backoff
- jitter
- throttling
- queue buffering
- autoscale

Very strong answer.

---

**16. Security**

**Q28. Security risks?**

**Answer:**

- spoofed devices
  - firmware abuse
  - stolen certs
  - DDoS
  - replay attacks
- 

**Q29. Security controls?**

**Answer:**

Need:

- mTLS
- device certificates
- per-device identity
- secure boot
- encryption
- secret rotation

- OTA signing
- RBAC

Huge architect answer.

---

## 17. HA / DR

### Q30. HA design?

**Answer:**

Use:

- multi-region brokers
  - replicated event streams
  - stateless services
  - DB replication
  - failover
- 

### Q31. DR?

**Answer:**

- replay events
  - backup
  - geo failover
  - restore tests
- 

## 18. Scale

### Q32. How scale to 1B events/day?

**Answer:**

Use:

- partitioned brokers
- Kafka scale
- stream workers
- shard DB



- hot/cold storage
- autoscale

---

## 19. Bottlenecks

### Q33. Common hotspots?

**Answer:**

- hot devices
  - partition skew
  - reconnect storms
  - rule spikes
  - TSDB writes
  - dashboard load
- 

## 20. Azure Mapping (.NET-focused)

### Q34. Azure services?

**Answer:**

| Need               | Azure                  |
|--------------------|------------------------|
| Device ingress     | IoT Hub                |
| MQTT / device mgmt | IoT Hub                |
| Event streaming    | Event Hubs             |
| Stream processing  | Stream Analytics / AKS |
| Rules/compute      | Functions              |
| Digital Twin       | Azure Digital Twins    |
| Time-series        | Cosmos / Data Explorer |
| Data lake          | ADLS                   |
| Cache              | Redis                  |
| Monitoring         | Azure Monitor          |

| Need       | Azure     |
|------------|-----------|
| Secrets    | Key Vault |
| Containers | AKS       |

Very strong Azure topic.

---

## 21. AWS Mapping (.NET-focused)

### Q35. AWS services?

**Answer:**

| Need               | AWS                     |
|--------------------|-------------------------|
| Device ingress     | AWS IoT Core            |
| MQTT               | AWS IoT Core            |
| Event stream       | Kinesis                 |
| Stream processing  | Kinesis Analytics / EKS |
| Rules/compute      | Lambda                  |
| Digital twin/state | IoT Device Shadow       |
| Time-series        | Timestream              |
| Data lake          | S3                      |
| Cache              | ElastiCache             |
| Monitoring         | CloudWatch              |
| Secrets            | Secrets Manager         |
| Containers         | EKS                     |

---

## 22. Trade-offs

### Q36. MQTT vs HTTP?

**Answer:**

**MQTT**            **HTTP**

device friendly easier integration

persistent        request/response

---

### **Q37. Strong vs eventual consistency?**

**Answer:**

Strong:

- command state
- critical device action

Eventual:

- analytics
  - dashboards
  - long-term reporting
- 

### **23. Anti-Patterns**

#### **Q38. Dangerous anti-patterns?**

**Answer:**

- no partition strategy
  - no replay
  - hard-coded rules
  - no device auth
  - no cold storage
  - single broker cluster
- 

### **24. FAANG / Architect Trap Questions**

#### **Q39. Can Kafka be source of truth?**

**Answer:**

No. Stream transport, not primary business truth.

---

**Q40. Can Redis store telemetry truth?**

**Answer:**

No.

---

**Q41. Can HTTP replace MQTT?**

**Answer:**

Possible, but often weaker for device-scale real-time.

---

### **Final Architect Summary**

Best design:

**MQTT Broker + Device Registry + Event Bus + Stream Processing + Rules Engine + Alerting + Digital Twin + TSDB + Data Lake + Multi-region HA**

**Azure:**

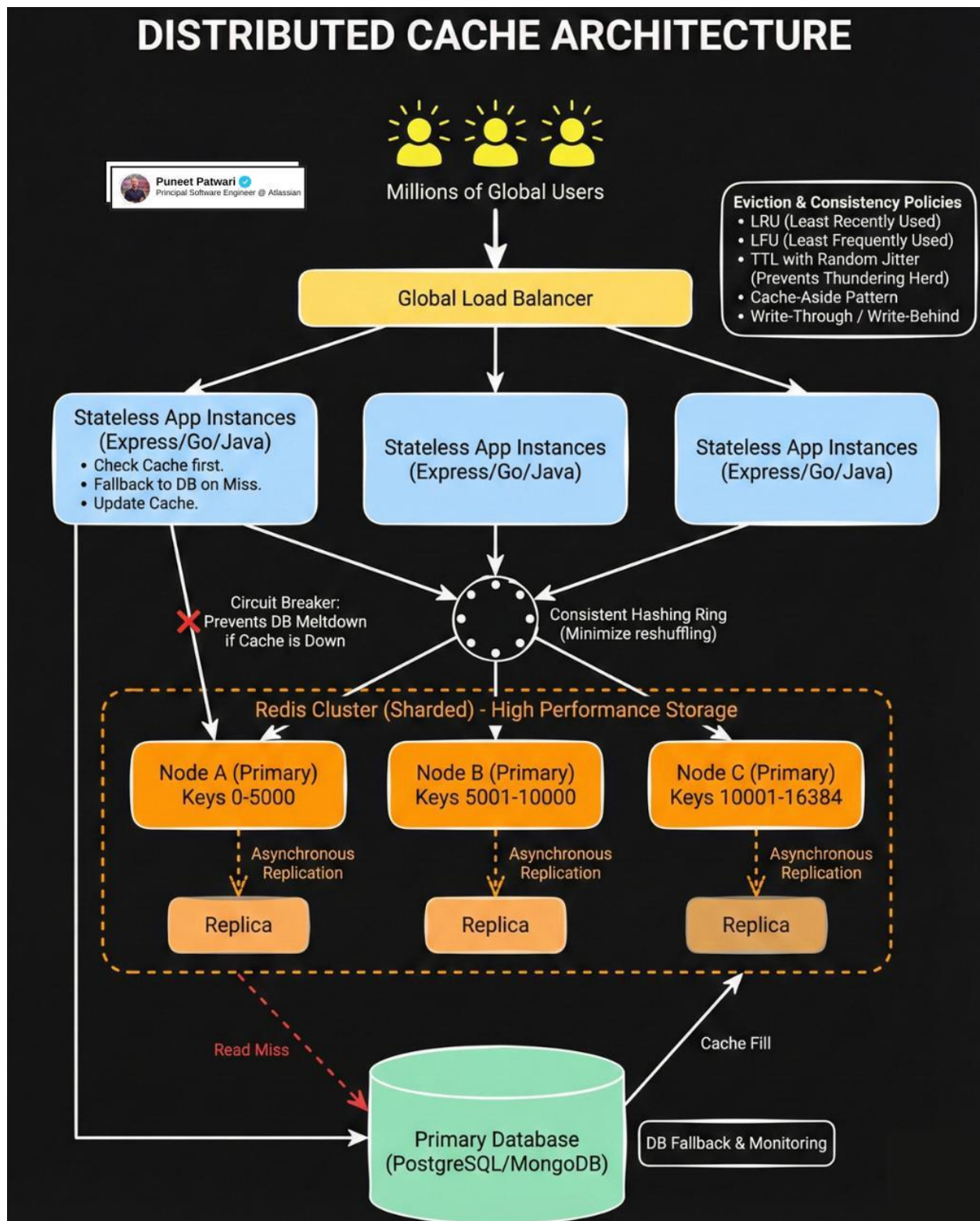
IoT Hub + Event Hubs + Stream Analytics + Digital Twins + Data Explorer + ADLS + AKS

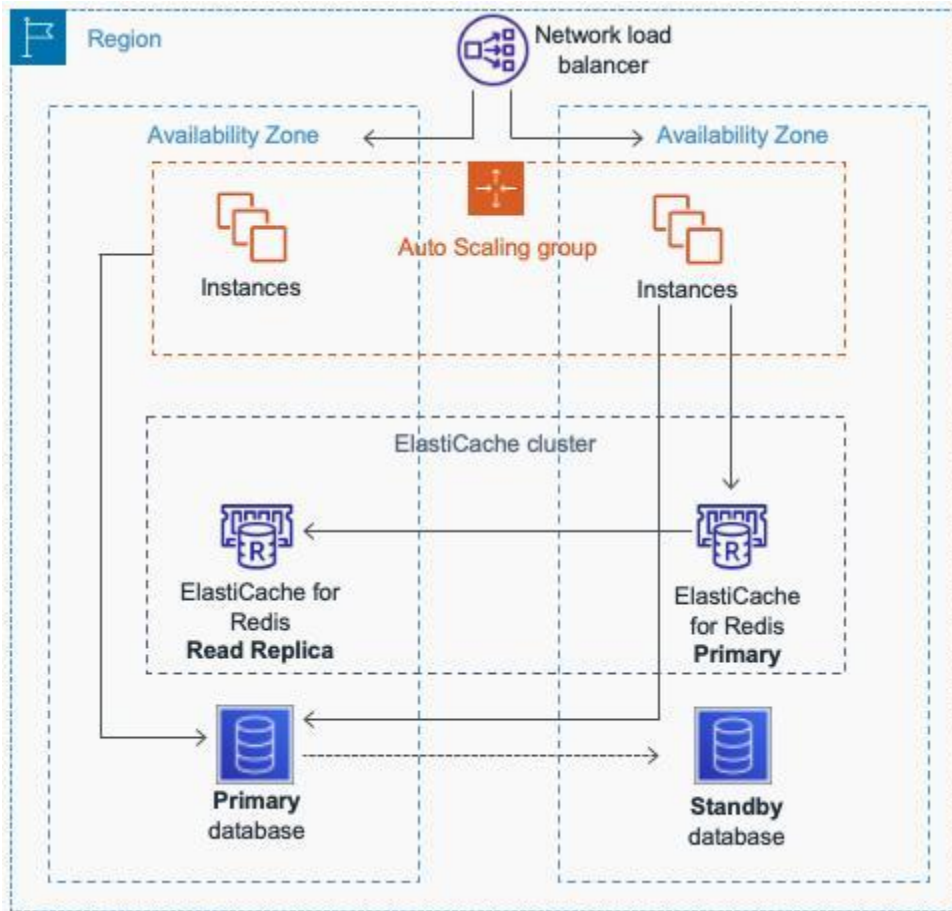
**AWS:**

IoT Core + Kinesis + Lambda + Device Shadow + Timestream + S3 + EKS

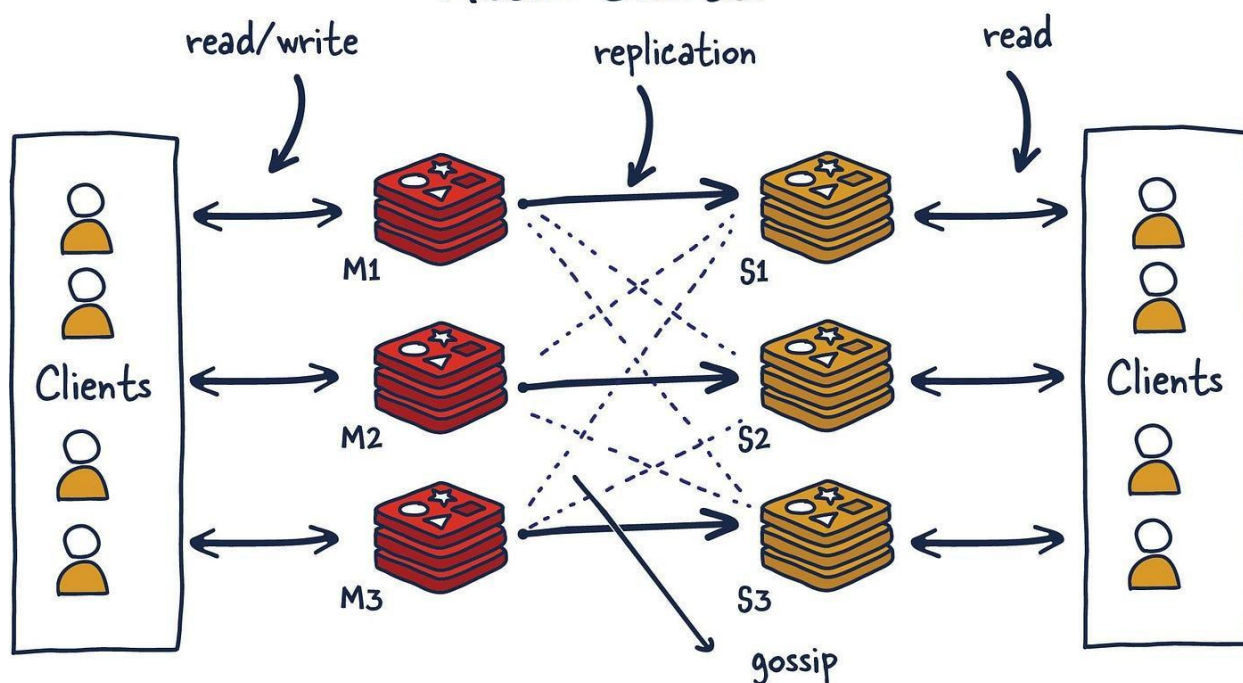
This is a strong **IoT / Cloud / Distributed / Data Architect** answer.

## Design 16 → Global Distributed Cache Platform (Redis-like)





## Redis Cluster



This answers:

**“How do you design a Redis-like distributed cache platform handling millions of requests/sec globally?”**

---

### **Q1. What is Distributed Cache Platform?**

**Answer:**

A high-speed in-memory system used for fast data retrieval across multiple nodes/regions.

Used for:

- API caching
- session storage
- auth tokens
- hot product cache
- feed cache
- rate limiting
- leaderboards

Huge infra topic.

---

### **Q2. Core requirements?**

**Answer:**

#### **Functional Requirements**

- GET/SET
- TTL
- replication
- partitioning
- failover
- pub/sub
- counters
- atomic ops

- eviction
- persistence (optional)
- cache invalidation

### **Non-Functional Requirements**

- sub-ms to low-ms latency
  - millions req/sec
  - HA
  - fault tolerance
  - horizontal scale
  - consistency trade-offs
  - multi-region support
- 

### **1. Capacity Estimation**

Assume:

- 50M active clients
- 5–20M req/sec
- TB-scale hot data
- burst traffic

Challenges:

- hot keys
- replication lag
- memory pressure
- failover
- cache stampede

Huge distributed topic.

---

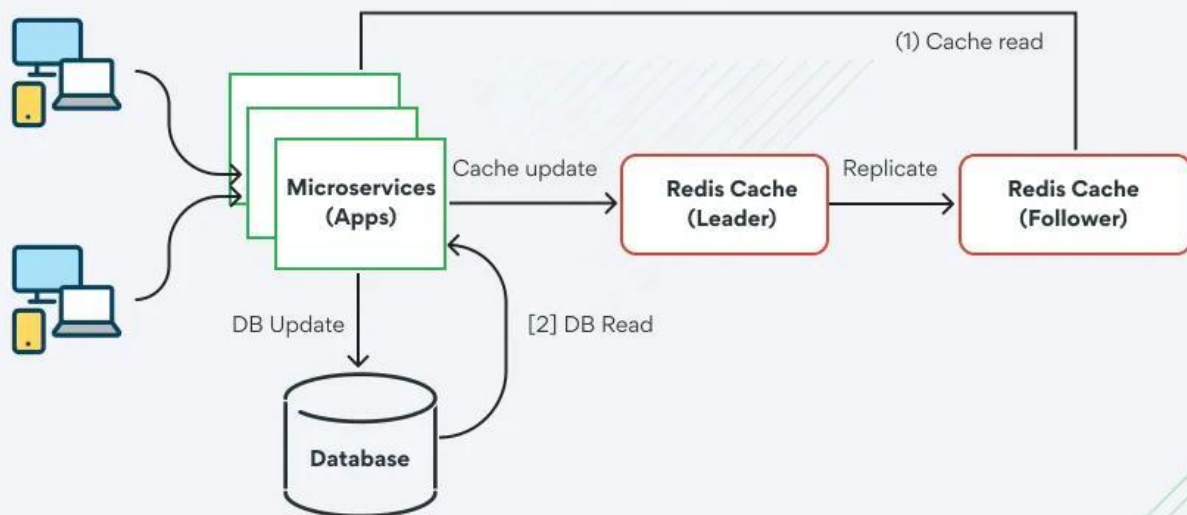
### **2. High-Level Architecture**

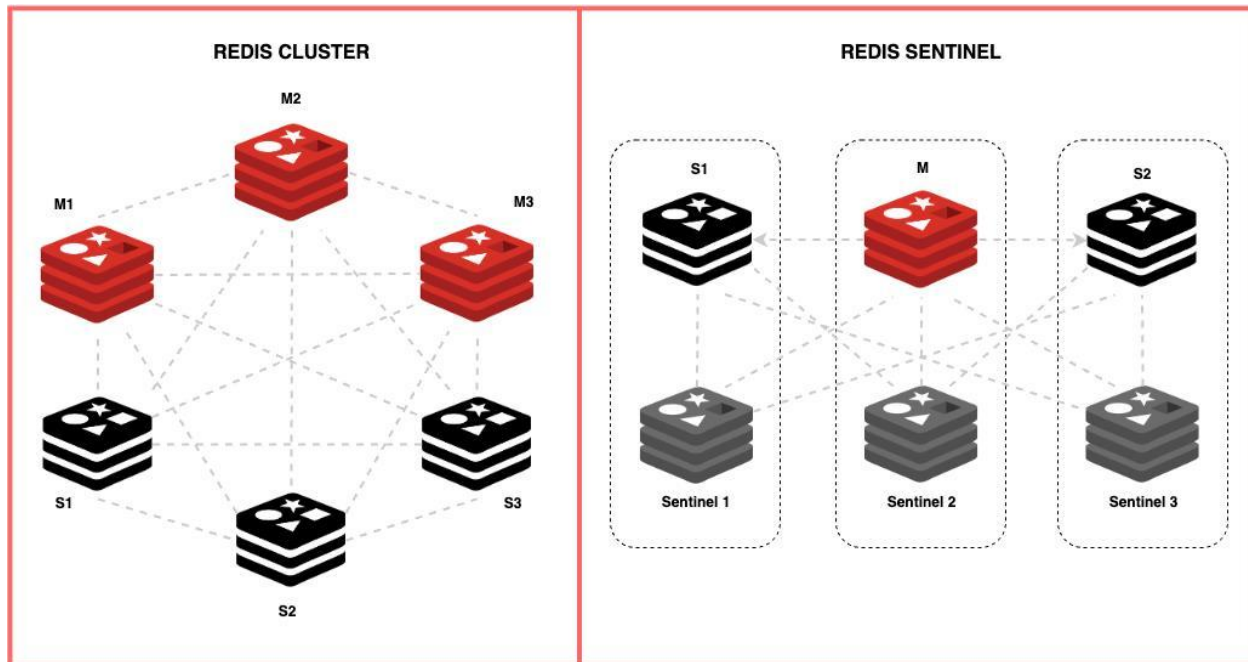


## Redis Cluster Architecture



## Distributed Caching with Redis





8

### Q3. Core components?

Answer:

Usually:

- Client SDK / Cache Proxy
- Router / Partition Resolver
- Cache Cluster
- Primary Nodes
- Replica Nodes
- Sentinel / Failover Coordinator
- Config Service
- Pub/Sub Engine
- Persistence Layer (optional)
- Metrics/Observability

### 3. Cache Access Flow

#### Q4. GET flow?

**Answer:**

Step 1 → Client requests key  
Step 2 → Hash key  
Step 3 → Route to shard  
Step 4 → Read from memory  
Step 5 → Return response

Ultra-low latency path.

---

**Q5. SET flow?**

**Answer:**

Step 1 → Key/value request  
Step 2 → Hash key  
Step 3 → Route to primary  
Step 4 → Write memory  
Step 5 → replicate  
Step 6 → ACK

---

#### **4. Partitioning / Sharding (Critical)**

**Q6. Why partition cache?**

**Answer:**

One node cannot hold all hot data.

Huge scale topic.

---

**Q7. Common partition strategies?**

**Answer:**

- consistent hashing
  - hash slot partitioning
  - range partitioning
- 

**Q8. Why consistent hashing popular?**

**Answer:**

Minimal remapping when nodes added/removed.

Huge FAANG favorite.

---

## 5. Replication

### Q9. Why replication?

**Answer:**

Supports:

- HA
  - failover
  - read scale
  - durability (limited)
- 

### Q10. Primary-replica model?

**Answer:**

Writes → primary

Reads → replica/primary

Very common.

---

### Q11. Replication risks?

**Answer:**

- lag
  - stale reads
  - split brain
- 

## 6. Failover

### Q12. What if primary node dies?

**Answer:**

Promote replica.

Huge Redis topic.

---

### Q13. Why failover coordinator?

**Answer:**

Manages:

- health checks
- leader election
- promotion
- topology updates

Redis Sentinel-like concept.

---

## 7. Consistency

### Q14. Strong vs eventual consistency?

**Answer:**

Strong-ish:

- counters
- locks
- rate limiter

Relaxed/eventual:

- product cache
- feed cache
- recommendations

Very common trade-off.

---

### Q15. Can cache be source of truth?

**Answer:**

Usually no.

Huge architect trap.

---

## 8. Eviction Policies

### Q16. Why eviction?

**Answer:**

Memory is limited.

Need removal strategy.

---

#### **Q17. Common eviction strategies?**

**Answer:**

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- TTL expiry
- Random
- FIFO

Must know.

---

#### **Q18. LRU vs LFU?**

**Answer:**

| LRU | LFU |
|-----|-----|
|-----|-----|

|               |                 |
|---------------|-----------------|
| recency-based | frequency-based |
|---------------|-----------------|

---

#### **9. TTL / Expiration**

##### **Q19. Why TTL?**

**Answer:**

Prevent stale data + memory cleanup.

Huge caching topic.

---

##### **Q20. TTL challenges?**

**Answer:**

Mass expiration storms.

Huge hotspot.

---

---

## 10. Cache Patterns

### Q21. What is Cache-Aside?

**Answer:**

App checks cache → DB on miss → update cache.

Most common.

---

### Q22. What is Write-Through?

**Answer:**

Write DB + cache together.

---

### Q23. What is Write-Back?

**Answer:**

Write cache first, DB later.

Faster but riskier.

---

### Q24. Best pattern?

**Answer:**

Usually Cache-Aside.

Strong architect answer.

---

## 11. Cache Stampede (Critical)

### Q25. What is Cache Stampede?

**Answer:**

Hot key expires → many clients hit DB simultaneously.

Huge FAANG topic.

---

### Q26. How solve?

**Answer:**

Use:

- jitter TTL
- request coalescing
- lock/single-flight
- refresh-ahead

Very common.

---

## 12. Hot Key Problem

**Q27. What is hot key?**

**Answer:**

One key receives massive reads.

Example:

viral product / auth token / homepage.

---

**Q28. How solve?**

**Answer:**

- replication
  - local cache
  - key spreading
  - edge cache
  - partitioning
- 

## 13. Pub/Sub

**Q29. Why pub/sub useful?**

**Answer:**

Supports:

- invalidation
- notifications
- event fan-out



- distributed coordination
- 

## 14. Persistence

### Q30. Why persistence in cache?

**Answer:**

Optional for:

- restart recovery
- snapshots
- logs

Redis-like feature.

---

## 15. Multi-Region

### Q31. How support global cache?

**Answer:**

Use:

- geo replication
- region-local caches
- eventual sync
- active-active patterns

Huge cloud topic.

---

### Q32. Risks?

**Answer:**

- stale data
  - conflict
  - split brain
  - network partition
- 

## 16. Security

### Q33. Security risks?

Answer:

- exposed cache
  - data leakage
  - poisoned cache
  - insecure pub/sub
  - unauthorized access
- 

### Q34. Security controls?

Answer:

- auth
  - TLS
  - network isolation
  - RBAC
  - encryption
  - secret rotation
  - private networking
- 

## 17. Reliability / HA / DR

### Q35. HA design?

Answer:

Use:

- replicas
  - failover
  - cluster health
  - quorum
  - autoscaling
- 

### Q36. DR strategy?

**Answer:**

- snapshots
  - restore
  - geo replication
  - rebuild from source-of-truth
- 

## **18. Scale**

**Q37. How scale to millions req/sec?**

**Answer:**

Use:

- sharded cluster
  - consistent hashing
  - replicas
  - local caches
  - async replication
  - proxy routing
- 

## **19. Bottlenecks**

**Q38. Common hotspots?**

**Answer:**

- hot keys
  - memory fragmentation
  - failover storms
  - replication lag
  - large values
  - TTL storms
- 

## **20. Azure Mapping (.NET-focused)**

**Q39. Azure services?**

**Answer:**

**Need          Azure**

Cache          Azure Cache for Redis

App nodes    AKS / App Service

Config        App Configuration

Monitoring   Azure Monitor

Security      VNet + Key Vault

Multi-region Front Door + geo patterns

---

## **21. AWS Mapping (.NET-focused)**

**Q40. AWS services?**

**Answer:**

**Need          AWS**

Cache        ElastiCache Redis

App nodes    EKS / ECS

Config        AppConfig

Monitoring   CloudWatch

Security      VPC + Secrets Manager

Multi-region Route53 + replication

---

## **22. Trade-offs**

**Q41. Redis vs Memcached?**

**Answer:**

**Redis                  Memcached**

rich features          simpler

persistence, pub/sub lighter

Must know.

---

#### Q42. Local cache vs distributed cache?

Answer:

| Local        | Distributed   |
|--------------|---------------|
| fastest      | shared/global |
| inconsistent | scalable      |

---

#### 23. Anti-Patterns

##### Q43. Dangerous anti-patterns?

Answer:

- cache as source of truth
  - no TTL strategy
  - no hot-key mitigation
  - huge objects in cache
  - no failover
  - shared public cache access
- 

#### 24. FAANG / Architect Trap Questions

##### Q44. Can Redis replace DB?

Answer:

Usually no.

---

##### Q45. Can replication guarantee zero stale reads?

Answer:

No.

---

##### Q46. Is cache always faster?

**Answer:**

Usually yes, but wrong usage can hurt.

---

**Final Architect Summary**

Best design:

**Client SDK + Router + Sharded Cache Cluster + Primaries + Replicas + Failover Coordinator + TTL + Eviction + Cache-Aside + Multi-region HA**

**Azure:**

Azure Cache for Redis + AKS + App Config + Monitor + Front Door

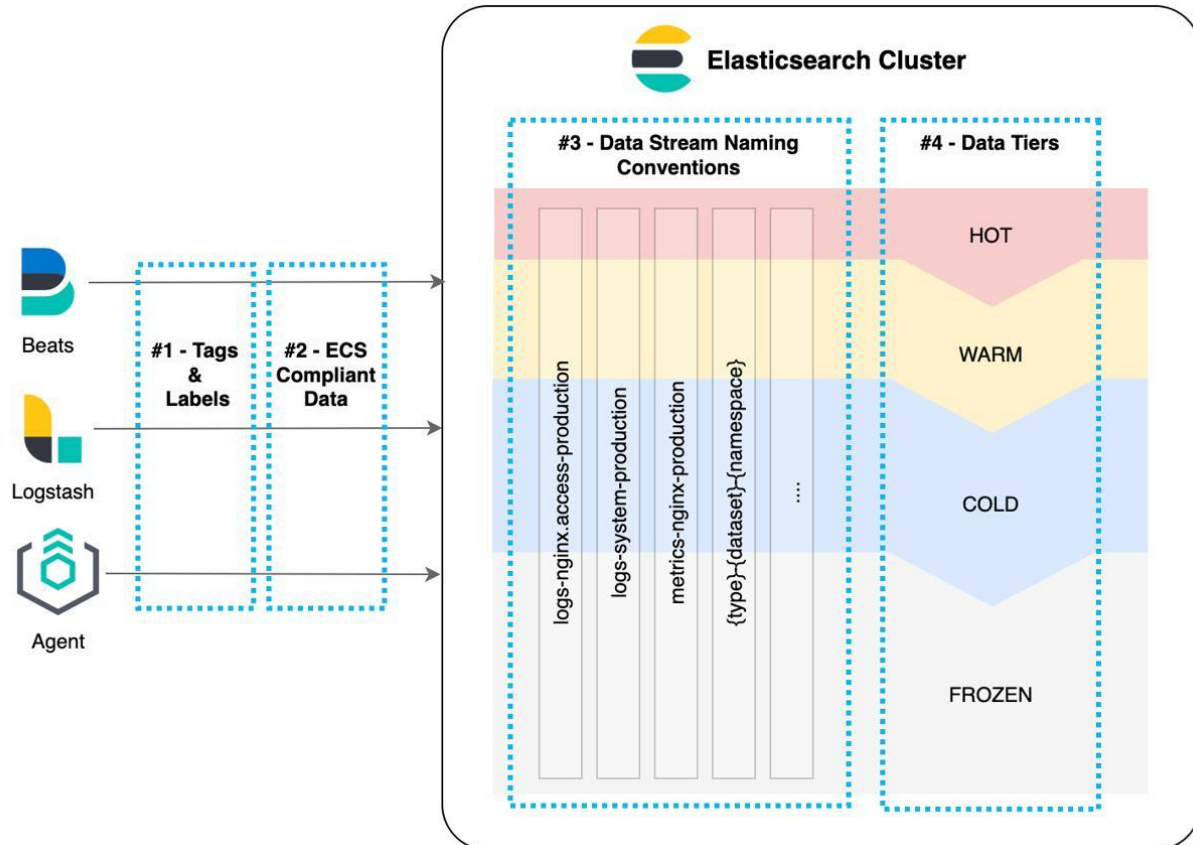
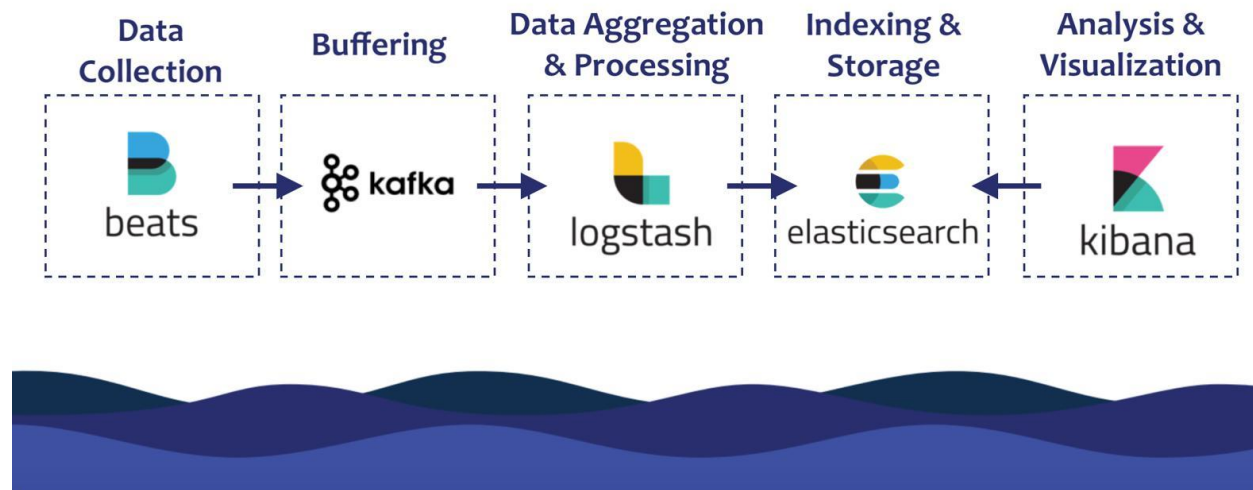
**AWS:**

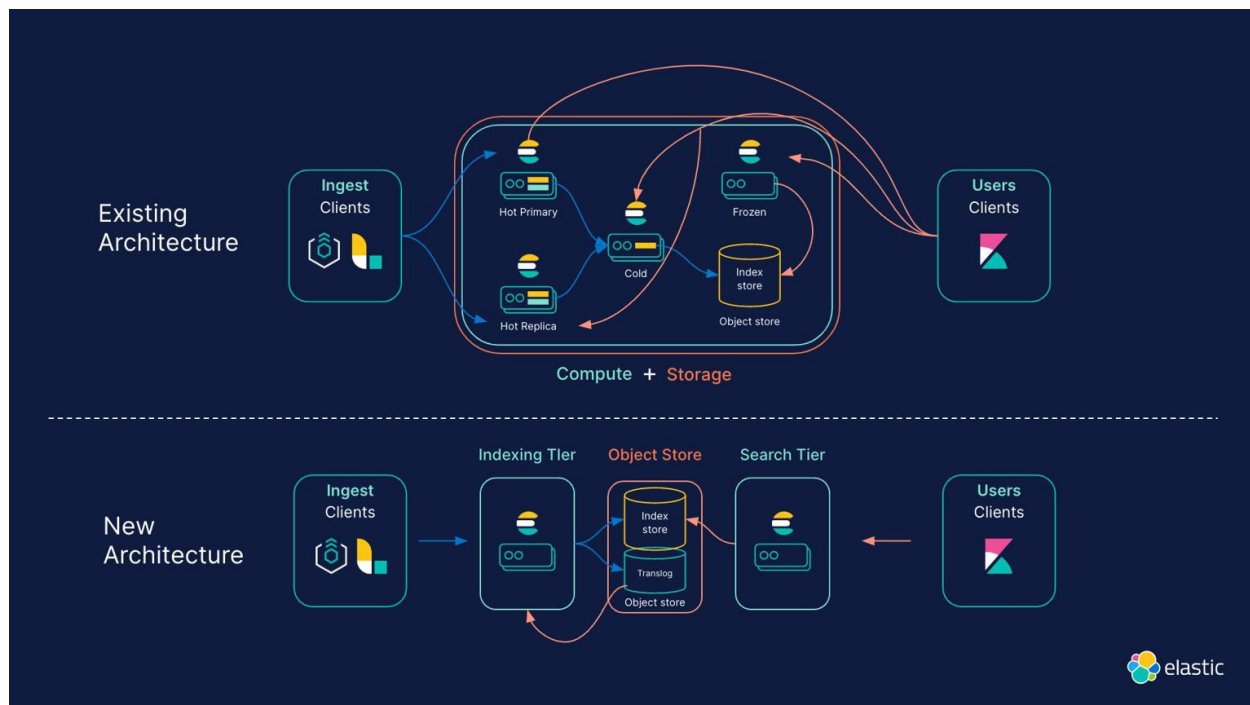
ElastiCache + EKS + AppConfig + CloudWatch + Route53

This is a strong **Redis / Performance / Cloud / Distributed Systems Architect** answer.

## Design 17 → Searchable Logging Platform (ELK / Splunk-like)

### Example of ELK Stack Architecture for Massive Amounts of Data





7

This answers:

**“How do you design a centralized searchable logging platform handling TB/PB logs/day with fast querying and retention?”**

### Q1. What is Searchable Logging Platform?

**Answer:**

A centralized system that collects, stores, indexes, searches, and analyzes logs from distributed systems.

Examples:

- application logs
- API logs
- infra logs
- audit logs
- security logs
- Kubernetes logs
- cloud logs

Huge observability topic.



---

## Q2. Core requirements?

Answer:

### Functional Requirements

- log ingestion
- parsing
- enrichment
- indexing
- full-text search
- filtering
- dashboards
- alerting
- anomaly detection
- retention
- multi-tenant isolation
- replay/export

### Non-Functional Requirements

- TB/PB per day
- low-latency search
- HA
- horizontal scale
- fault tolerance
- durability
- cost optimization
- geo-distribution

---

## 1. Capacity Estimation

Assume:

- 500K servers / services

- 100B log events/day
- TB/PB data/day
- millions of queries/day

Challenges:

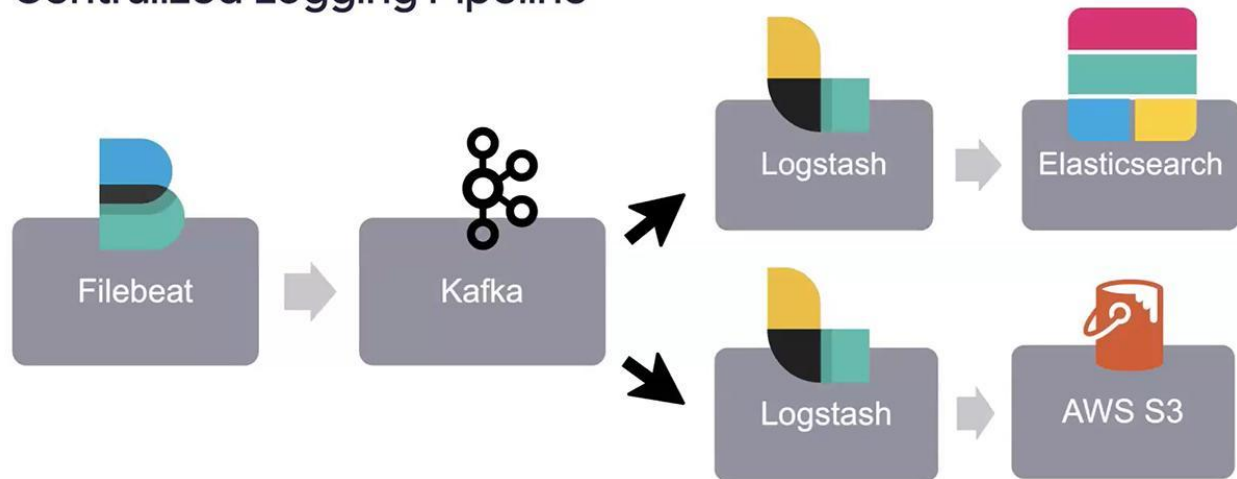
- write-heavy ingestion
- hot shards
- high-cardinality fields
- retention cost
- noisy tenants

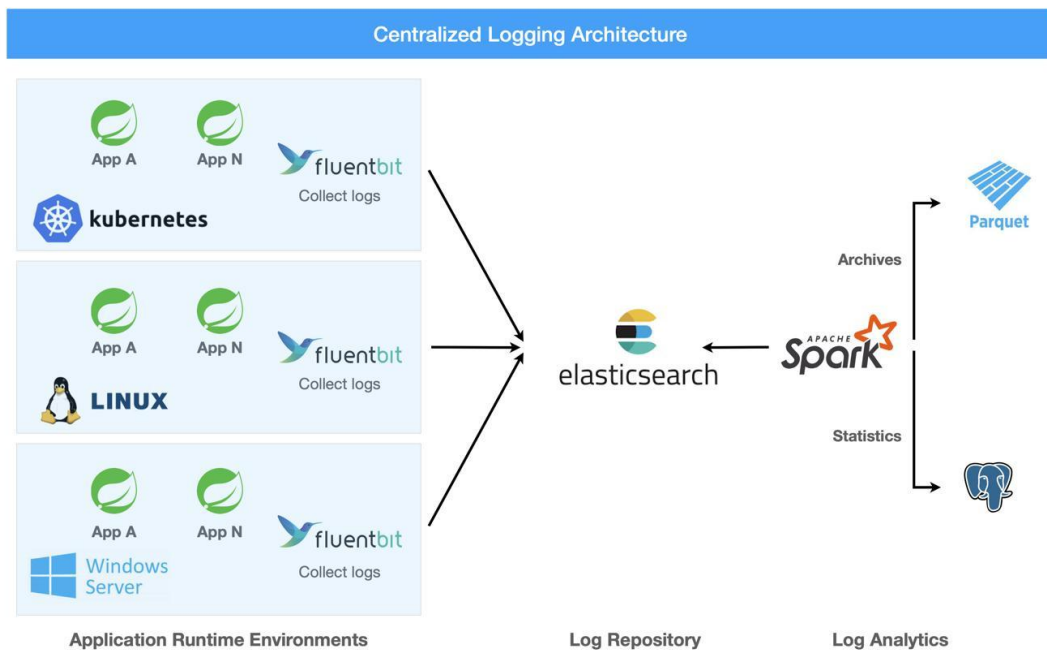
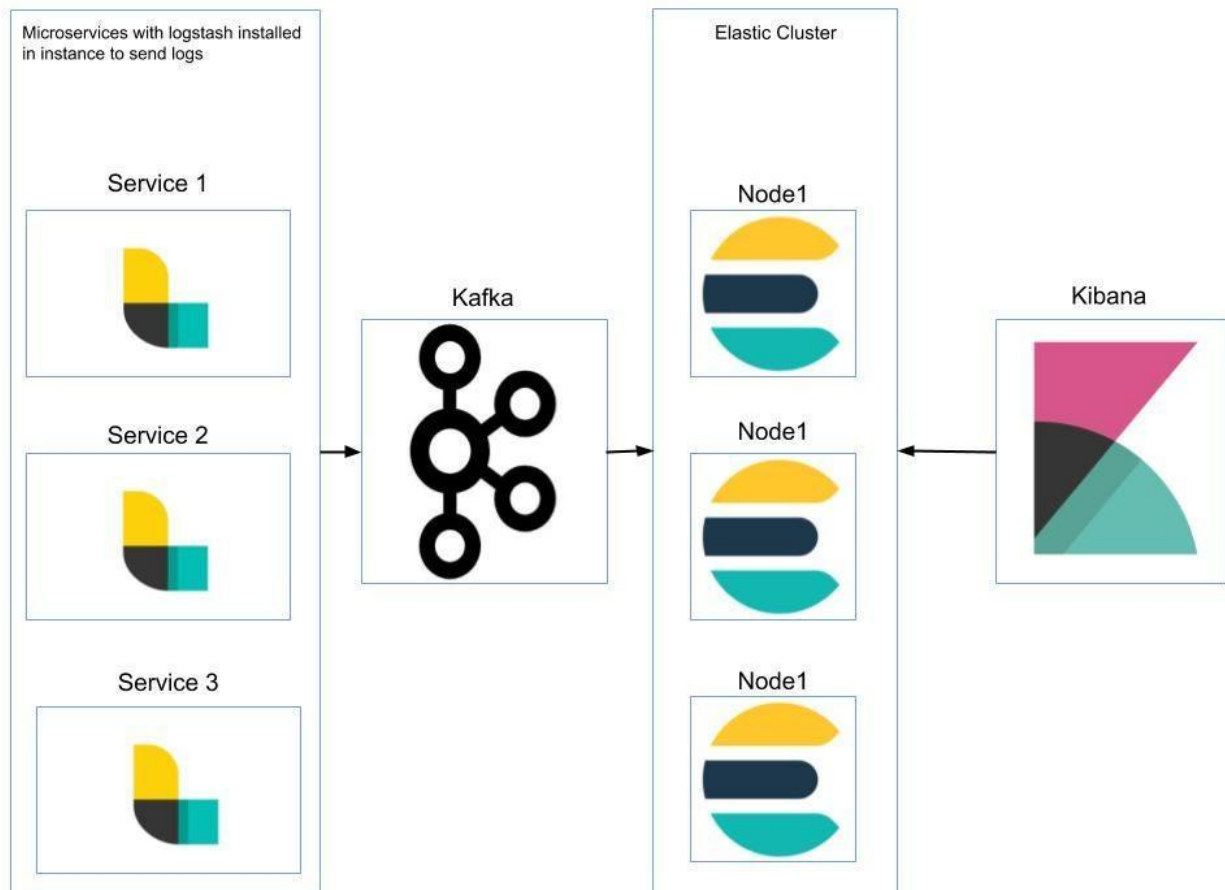
Huge platform engineering topic.

---

## 2. High-Level Architecture

### Centralized Logging Pipeline





### Q3. Core components?

**Answer:**

Usually:

- Log Agents / Collectors
  - Ingestion Gateway
  - Queue / Event Bus
  - Parser / Enrichment Service
  - Indexing Service
  - Search Cluster
  - Query API
  - Dashboard / Visualization
  - Alert Engine
  - Retention Manager
  - Archive Storage
  - Tenant Isolation Layer
- 

### 3. Log Collection

#### Q4. How logs are collected?

**Answer:**

Common sources:

- app logs
- containers
- K8s nodes
- DB logs
- audit trails
- OS logs

Flow:

App/Node → Agent → Gateway.

---

#### Q5. Why log agents?

**Answer:**

Agents handle:

- buffering
- batching
- retries
- compression
- local resiliency

Examples:

Fluent Bit / Logstash / Vector.

---

#### 4. Ingestion Pipeline

##### Q6. End-to-end log flow?

**Answer:**

Step 1 → App emits log

Step 2 → Agent collects

Step 3 → Batch/compress

Step 4 → Ingestion gateway

Step 5 → Queue/Event bus

Step 6 → Parse/enrich

Step 7 → Index

Step 8 → Store

Step 9 → Search/alert/dashboard

Huge architect flow.

---

#### 5. Queue / Event Bus

##### Q7. Why Kafka/Event Bus?

**Answer:**

Needed for:

- buffering spikes
- decoupling
- replay

- durability
- scaling consumers

Huge distributed topic.

---

#### **Q8. Partition strategy?**

**Answer:**

Usually by:

- service
  - tenant
  - region
  - appld
  - log type
- 

#### **6. Parsing / Enrichment**

##### **Q9. Why parse logs?**

**Answer:**

Converts raw logs into structured searchable fields.

Example:

timestamp, level, traceId, userId.

---

##### **Q10. Why enrichment?**

**Answer:**

Adds:

- hostname
- region
- env
- service
- pod
- tenantId

Very important.

---

## **7. Indexing (Critical)**

### **Q11. Why indexing needed?**

**Answer:**

Fast search at scale.

Cannot scan PB logs.

Huge ELK concept.

---

### **Q12. Common indexing model?**

**Answer:**

Usually inverted index.

Same search concept as Elastic/OpenSearch.

---

### **Q13. Sharding strategy?**

**Answer:**

Usually:

- time-based
- tenant-based
- service-based

Most common:  
time-based.

---

## **8. Search Query Flow**

### **Q14. Search flow?**

**Answer:**

Step 1 → Query API

Step 2 → Query parser

Step 3 → shard routing

Step 4 → parallel search

Step 5 → aggregate results

Step 6 → return

---

## 9. High Cardinality Problem

**Q15. What is high-cardinality field?**

**Answer:**

Field with huge unique values.

Examples:

- requestId
- userId
- traceId

Huge logging trap.

---

**Q16. Why dangerous?**

**Answer:**

Index bloat + memory pressure + slow queries.

Very common SRE question.

---

## 10. Hot/Warm/Cold Storage (Critical)

**Q17. Why storage tiers?**

**Answer:**

Logs grow huge.

Need cost optimization.

---

**Q18. Typical retention strategy?**

**Answer:**

Example:

- Hot → 7 days (fast SSD search)
- Warm → 30–90 days



- Cold → months/years archive

Very common architect answer.

---

## 11. Search / Query Performance

### Q19. How optimize search?

**Answer:**

Use:

- shard sizing
  - index lifecycle
  - compression
  - cache
  - filter-first queries
  - time-based pruning
- 

### Q20. Hot shard problem?

**Answer:**

One shard overloaded.

Huge distributed topic.

---

## 12. Alerting

### Q21. What alerts can be built?

**Answer:**

- error spikes
  - latency increase
  - auth failures
  - suspicious access
  - disk issues
  - pod crashes
-

### 13. Correlation / Observability

#### Q22. Why traceId correlation?

**Answer:**

Connect logs across microservices.

Huge observability topic.

---

#### Q23. Logs vs Metrics vs Traces?

**Answer:**

| Logs | Metrics | Traces |
|------|---------|--------|
|------|---------|--------|

|                 |            |              |
|-----------------|------------|--------------|
| detailed events | aggregates | request path |
|-----------------|------------|--------------|

Must know for architect/SRE.

---

### 14. Multi-Tenant Isolation

#### Q24. Why tenant isolation?

**Answer:**

Shared platform but isolated:

- search
  - quotas
  - billing
  - retention
  - security
- 

### 15. Security / Compliance

#### Q25. Security risks?

**Answer:**

- PII leakage
- secret exposure
- log poisoning

- unauthorized search
- audit tampering

Huge enterprise topic.

---

#### **Q26. Security controls?**

**Answer:**

Need:

- RBAC
  - encryption
  - PII masking
  - immutable audit
  - WORM retention
  - tenant isolation
  - secret redaction
- 

#### **16. Reliability / HA / DR**

##### **Q27. HA design?**

**Answer:**

Use:

- replicated queue
  - search replicas
  - shard redundancy
  - stateless APIs
  - multi-zone
- 

##### **Q28. DR?**

**Answer:**

- snapshot restore
- cross-region archive

- replay queue
  - geo failover
- 

## 17. Scale

### Q29. How scale to PB/day?

**Answer:**

Use:

- partitioned ingestion
  - Kafka scaling
  - distributed search cluster
  - shard balancing
  - lifecycle tiers
- 

### Q30. Common bottlenecks?

**Answer:**

- hot shards
  - high-cardinality indexes
  - replay storms
  - query fan-out
  - storage IO
  - noisy tenants
- 

## 18. DB / Storage Layer

### Q31. What storage systems?

**Answer:**

Usually:

- Search cluster → indexed logs
- Object store → archive
- Queue → replay

- Redis → hot query cache
- Relational/NoSQL → configs

---

## 19. Azure Mapping (.NET-focused)

### Q32. Azure services?

**Answer:**

| Need                 | Azure                                         |
|----------------------|-----------------------------------------------|
| Log collection       | Azure Monitor Agent                           |
| Queue                | Event Hubs                                    |
| Processing           | Functions / AKS                               |
| Search/log analytics | Azure Monitor / Log Analytics / Data Explorer |
| Archive              | Blob / ADLS                                   |
| Cache                | Azure Redis                                   |
| Security             | Key Vault                                     |
| Monitoring           | App Insights                                  |
| Containers           | AKS                                           |

Very strong Azure topic.

---

## 20. AWS Mapping (.NET-focused)

### Q33. AWS services?

**Answer:**

| Need           | AWS                           |
|----------------|-------------------------------|
| Log collection | CloudWatch Agent / Fluent Bit |
| Queue          | Kinesis                       |
| Processing     | Lambda / EKS                  |
| Search         | OpenSearch                    |

| Need       | AWS                |
|------------|--------------------|
| Archive    | S3 Glacier         |
| Cache      | ElastiCache        |
| Security   | Secrets Manager    |
| Monitoring | CloudWatch / X-Ray |
| Containers | EKS                |

---

## 21. Trade-offs

### Q34. ELK/OpenSearch vs object-store-only?

Answer:

**Search Cluster**   **Object-store-only**

fast search      cheap archive

expensive      weak query speed

---

### Q35. Real-time vs batch indexing?

Answer:

Real-time → alerts / ops

Batch → cheaper

---

## 22. Anti-Patterns

### Q36. Dangerous anti-patterns?

Answer:

- no retention lifecycle
- indexing every field
- no tenant isolation
- storing secrets in logs
- no shard strategy
- no replay capability

---

### 23. FAANG / Architect Trap Questions

**Q37. Can DB replace search cluster?**

**Answer:**

Usually no.

---

**Q38. Can all logs stay hot forever?**

**Answer:**

Too expensive.

---

**Q39. Can logs be source of truth?**

**Answer:**

Usually no.

---

### Final Architect Summary

Best design:

**Agents + Gateway + Event Bus + Parser/Enrichment + Indexing + Search Cluster + Query API + Alerting  
+ Hot/Warm/Cold Storage + Tenant Isolation + Multi-region HA**

**Azure:**

Azure Monitor Agent + Event Hubs + AKS + Log Analytics + Data Explorer + ADLS

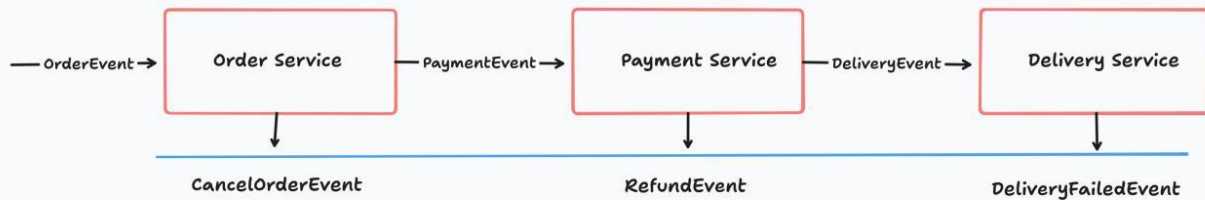
**AWS:**

CloudWatch Agent + Kinesis + EKS + OpenSearch + S3 Glacier

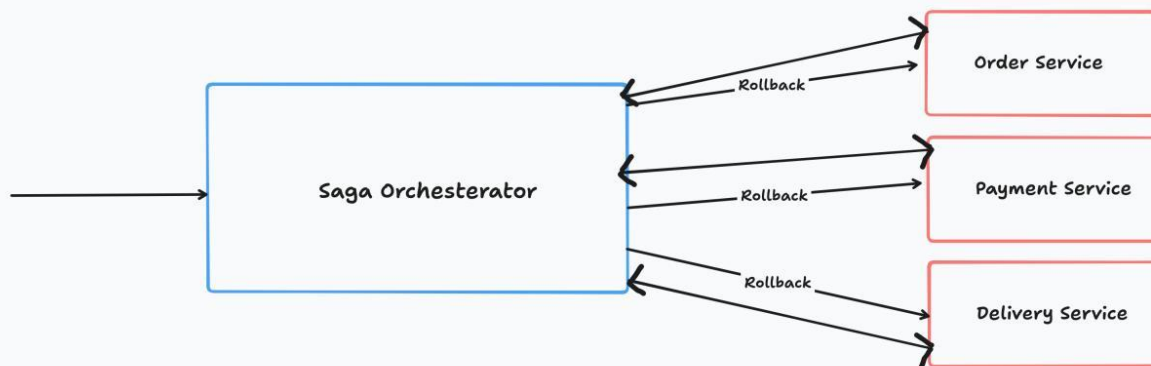
This is a strong **SRE / Observability / ELK / Cloud Architect** answer.

## Design 18 → Order Management System (Enterprise Workflow + Saga)

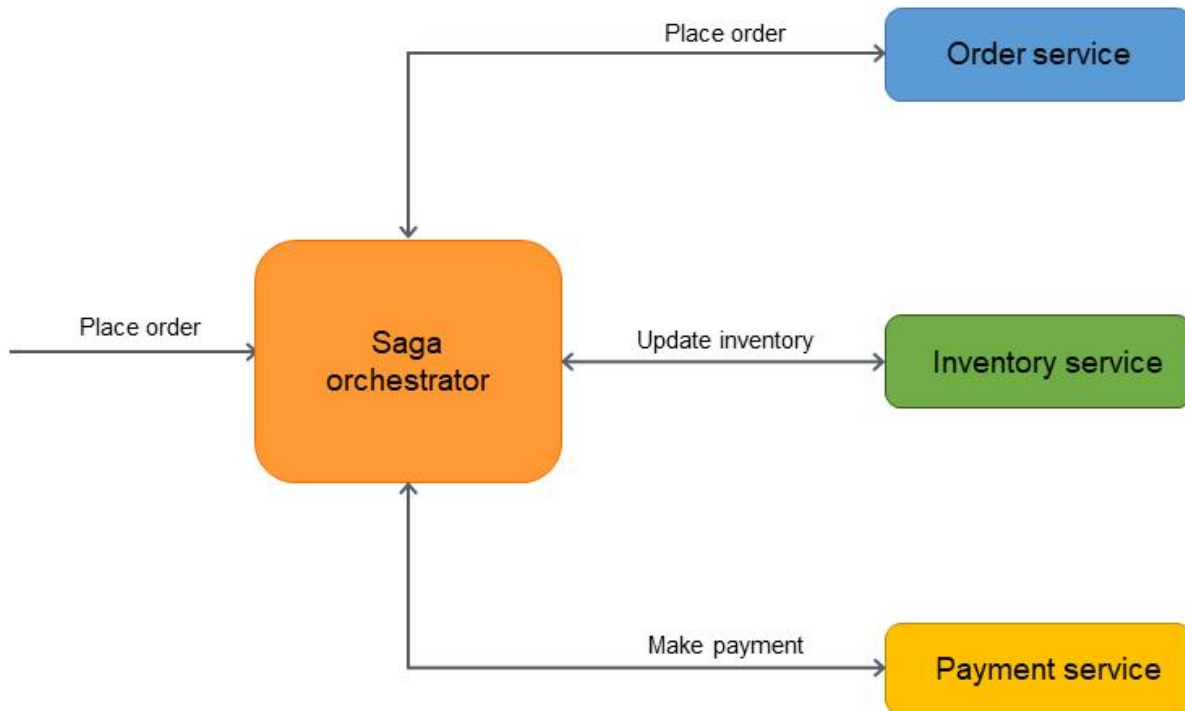
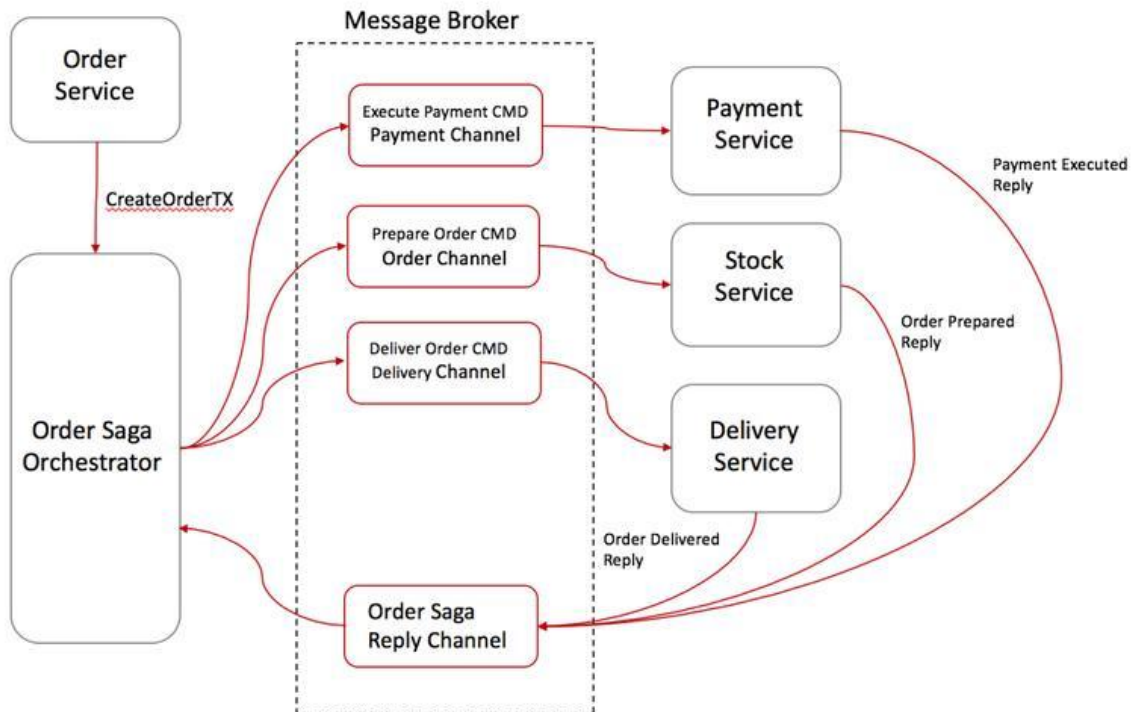
### Saga Choreography Pattern



### Saga Orchestration Pattern







6

This answers:

**“How do you design a large-scale enterprise Order Management System handling millions of orders, distributed workflows, inventory, payment, and eventual consistency?”**

---

### **Q1. What is Order Management System (OMS)?**

**Answer:**

A platform that manages full lifecycle of order processing from creation to fulfillment.

Examples:

- e-commerce orders
- retail fulfillment
- supply chain
- B2B procurement
- banking transaction workflow
- insurance claims

Huge enterprise topic.

---

### **Q2. Core requirements?**

**Answer:**

#### **Functional Requirements**

- create order
- validate inventory
- reserve stock
- process payment
- shipping workflow
- invoice
- cancel/refund
- retries
- compensation
- notifications
- audit trail

- order tracking

### **Non-Functional Requirements**

- millions orders/day
  - HA
  - fault tolerance
  - eventual consistency
  - no double payment
  - idempotency
  - scale
  - auditability
- 

### **1. Capacity Estimation**

Assume:

- 100M users
- 10M orders/day
- spikes (sales/events)
- multi-region

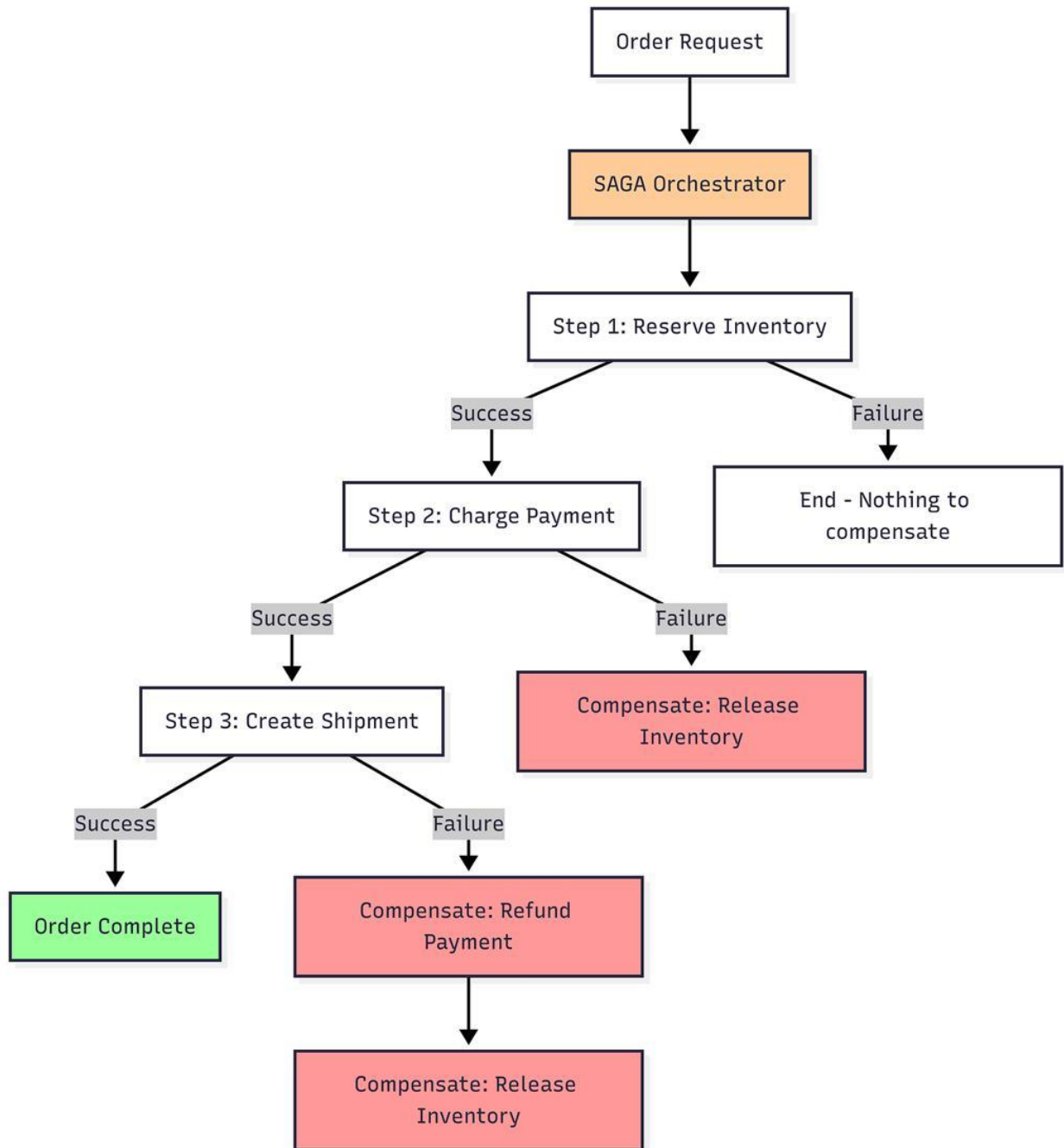
Challenges:

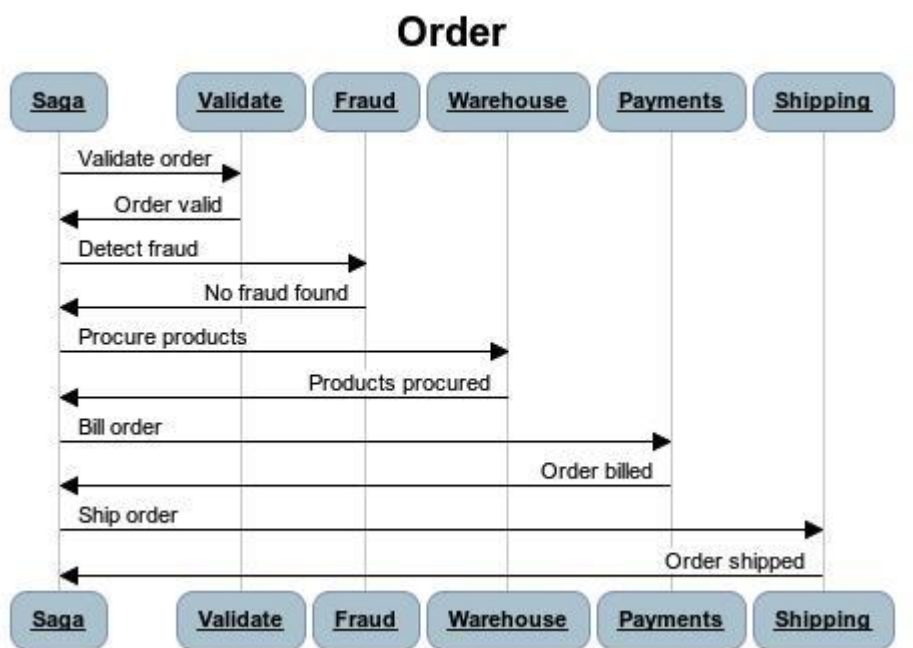
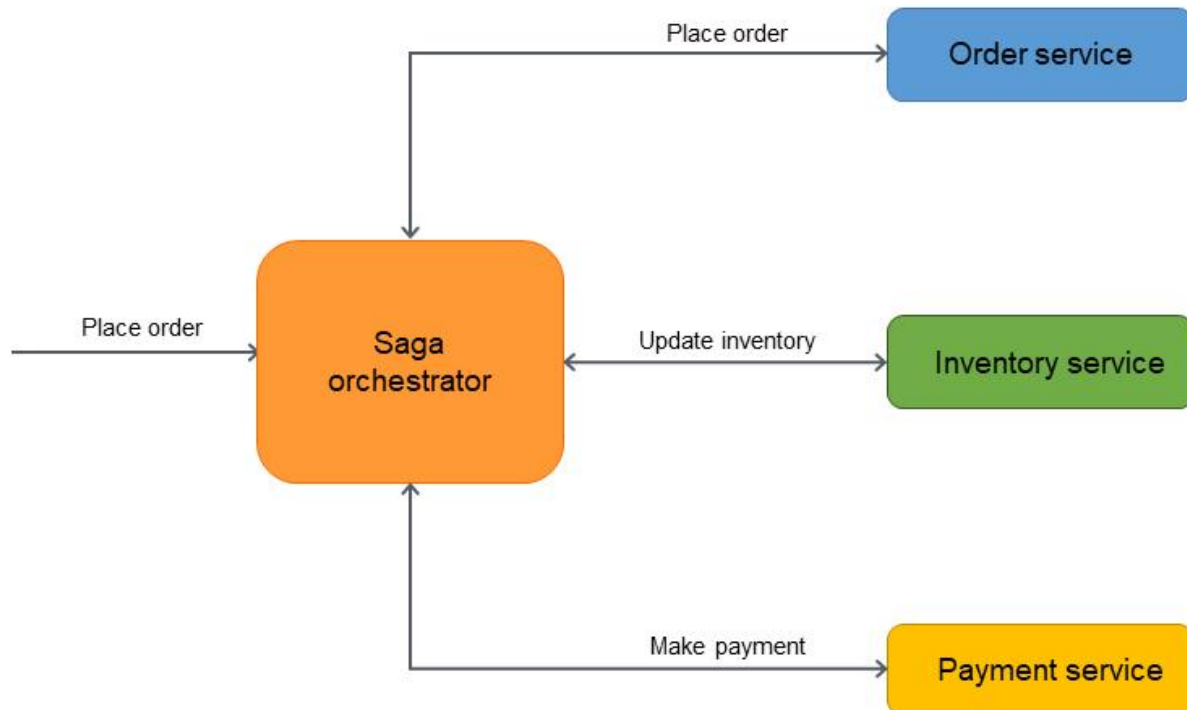
- distributed transactions
- retries
- duplicate orders
- stock race conditions
- compensation

Huge architect topic.

---

### **2. High-Level Architecture**





www.websequencediagrams.com

6

**Q3. Core services?**

**Answer:**

Usually:

- API Gateway
  - Order Service
  - Inventory Service
  - Payment Service
  - Pricing Service
  - Shipping/Fulfillment Service
  - Notification Service
  - Saga Coordinator
  - Event Bus
  - Outbox Processor
  - Audit Service
  - Customer Service
  - Billing/Invoice Service
- 

### **3. Order Lifecycle (Critical)**

#### **Q4. Typical order states?**

**Answer:**

Usually:

- Created
- Pending Validation
- Inventory Reserved
- Payment Authorized
- Confirmed
- Packed
- Shipped
- Delivered
- Cancelled
- Failed

- Refunded

Huge OMS topic.

---

#### **Q5. Why state machine?**

**Answer:**

Orders are workflow-based.

Helps:

- traceability
- retries
- consistency
- audit

Very common architect answer.

---

#### **4. End-to-End Flow**

##### **Q6. Order flow?**

**Answer:**

Step 1 → Customer places order

Step 2 → Order Service creates order

Step 3 → Inventory reserve

Step 4 → Payment auth

Step 5 → Shipping creation

Step 6 → Invoice

Step 7 → Notify customer

Step 8 → Complete

Huge enterprise workflow.

---

#### **5. Inventory**

##### **Q7. Why separate Inventory Service?**

**Answer:**

Handles:

- stock count

- reservation
- release
- warehouse mapping

Independent domain.

---

#### **Q8. Stock race condition?**

**Answer:**

Two users buy last item.

Huge distributed systems problem.

---

#### **Q9. How solve?**

**Answer:**

Use:

- reservation
- optimistic locking
- atomic updates
- short-lived holds

Must know.

---

### **6. Payment**

#### **Q10. Why Payment Service separate?**

**Answer:**

Complex domain:

- authorization
  - capture
  - refund
  - settlement
  - PCI compliance
-



### Q11. Payment states?

Answer:

- initiated
  - authorized
  - captured
  - failed
  - refunded
  - reversed
- 

### 7. Shipping / Fulfillment

#### Q12. Shipping responsibilities?

Answer:

- warehouse allocation
  - packing
  - dispatch
  - tracking
  - delivery confirmation
- 

### 8. Saga Pattern (Very Critical)

#### Q13. Why Saga needed?

Answer:

Distributed transactions across services.

Avoids 2PC.

Huge architect favorite.

---

#### Q14. What is Saga?

Answer:

Sequence of local transactions + compensations.

---

### **Q15. Example OMS Saga?**

**Answer:**

Order → Reserve stock → Charge payment → Ship.

If payment fails:  
release stock.

Classic answer.

---

## **9. Orchestration vs Choreography**

### **Q16. Orchestrated Saga?**

**Answer:**

Central coordinator controls workflow.

Good for enterprise visibility.

---

### **Q17. Choreography Saga?**

**Answer:**

Services react via events.

Looser coupling.

---

### **Q18. Which better?**

**Answer:**

OMS usually orchestration.

Complex workflows easier.

Very strong answer.

---

## **10. Compensation**

### **Q19. What is compensation?**

**Answer:**

Undo previous steps.

Examples:

- release stock
  - refund payment
  - cancel shipment
- 

## **Q20. Why compensation critical?**

### **Answer:**

Distributed failures happen.

Need eventual consistency.

---

## **11. Event Bus + Outbox**

### **Q21. Why Event Bus?**

#### **Answer:**

Supports:

- async workflow
  - decoupling
  - retries
  - scaling
- 

### **Q22. Why Outbox Pattern?**

#### **Answer:**

Avoid DB write + message publish inconsistency.

Huge architect topic.

---

### **Q23. Outbox flow?**

#### **Answer:**

Transaction:

write order + outbox record

Then:

outbox processor → publish event

Must know.

---

## 12. Idempotency

### Q24. Why idempotency?

**Answer:**

Avoid:

- double orders
- duplicate payment
- repeated shipping

Huge enterprise favorite.

---

### Q25. Common idempotent areas?

**Answer:**

- order creation
  - payment
  - notifications
  - refunds
- 

## 13. Data Layer

### Q26. What DBs?

**Answer:**

Usually polyglot:

- Relational → orders/payments
  - Redis → cache/locks
  - Event bus → workflow
  - Search index → order lookup
  - Data lake → analytics
- 

### Q27. Why relational DB strong here?

**Answer:**

Transactional consistency.

---

## **14. Caching**

**Q28. What to cache?**

**Answer:**

- order summary
  - inventory
  - product pricing
  - customer profile
- 

## **15. Reliability / HA / DR**

**Q29. HA design?**

**Answer:**

Use:

- stateless APIs
  - replicated DB
  - event replay
  - queue durability
  - failover
- 

**Q30. DR?**

**Answer:**

- backups
  - event replay
  - restore
  - geo failover
- 

## **16. Scale**

**Q31. How scale to millions orders/day?**

**Answer:**

Use:

- partition by order/customer
  - async workflows
  - queues
  - autoscaling
  - shard reads
- 

**Q32. Common bottlenecks?**

**Answer:**

- payment latency
  - inventory locks
  - retry storms
  - hot products
  - shipment spikes
- 

**17. Security**

**Q33. Security risks?**

**Answer:**

- payment abuse
  - replay
  - fraud
  - PII leakage
  - order tampering
- 

**Q34. Security controls?**

**Answer:**

Need:

- auth
- RBAC
- encryption
- audit
- PCI controls
- rate limiting
- tokenization

---

## 18. Multi-Tenant / Enterprise Workflow

### Q35. If B2B SaaS OMS?

**Answer:**

Need:

- tenant isolation
- workflow config
- approval chains
- SLA-based routing

Very enterprise-heavy.

---

## 19. Azure Mapping (.NET-focused)

### Q36. Azure services?

**Answer:**

| Need            | Azure                    |
|-----------------|--------------------------|
| APIs            | AKS / App Service        |
| Queue/Event bus | Service Bus / Event Hubs |
| Saga/workflows  | Durable Functions / AKS  |
| DB              | Azure SQL / Cosmos       |
| Cache           | Azure Redis              |

| Need           | Azure           |
|----------------|-----------------|
| Search         | Azure AI Search |
| Files/invoices | Blob Storage    |
| Monitoring     | App Insights    |
| Secrets        | Key Vault       |
| Global routing | Front Door      |

Very strong Azure topic.

---

## 20. AWS Mapping (.NET-focused)

### Q37. AWS services?

**Answer:**

| Need            | AWS                     |
|-----------------|-------------------------|
| APIs            | EKS / ECS               |
| Queue/Event bus | SQS / SNS / EventBridge |
| Saga/workflows  | Step Functions / EKS    |
| DB              | RDS / DynamoDB          |
| Cache           | ElastiCache             |
| Search          | OpenSearch              |
| Files           | S3                      |
| Monitoring      | CloudWatch              |
| Secrets         | Secrets Manager         |
| Global routing  | Route53                 |

---

## 21. Trade-offs

### Q38. 2PC vs Saga?

**Answer:**



## 2PC

## Saga

strong consistency    eventual consistency

slow/tightly coupled    scalable

OMS usually Saga.

---

### Q39. Sync vs async workflow?

**Answer:**

Critical:

- payment auth → maybe sync

Non-critical:

- invoice
  - notifications
  - analytics → async
- 

## 22. Anti-Patterns

### Q40. Dangerous anti-patterns?

**Answer:**

- distributed DB transactions everywhere
  - no compensation
  - no idempotency
  - no outbox
  - inventory in shared global lock
  - sync-heavy OMS
- 

## 23. FAANG / Architect Trap Questions

### Q41. Can Saga guarantee atomicity?

**Answer:**

No. Eventual consistency.

---

**Q42. Can event bus be source of truth?**

**Answer:**

No.

---

**Q43. Is choreography always better?**

**Answer:**

No. Complex OMS often favors orchestration.

---

### **Final Architect Summary**

Best design:

**API Gateway + Order Service + Inventory + Payment + Shipping + Saga Coordinator + Event Bus + Outbox + Compensation + Search + Multi-region HA**

**Azure:**

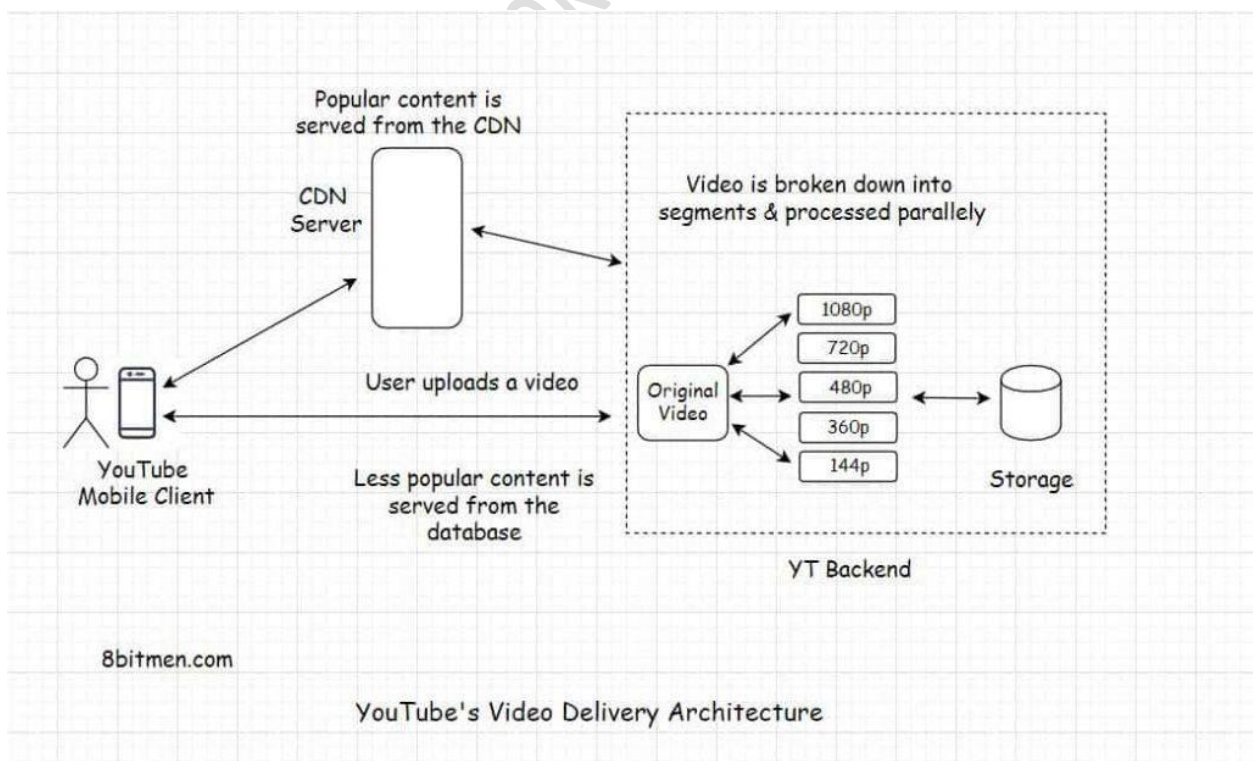
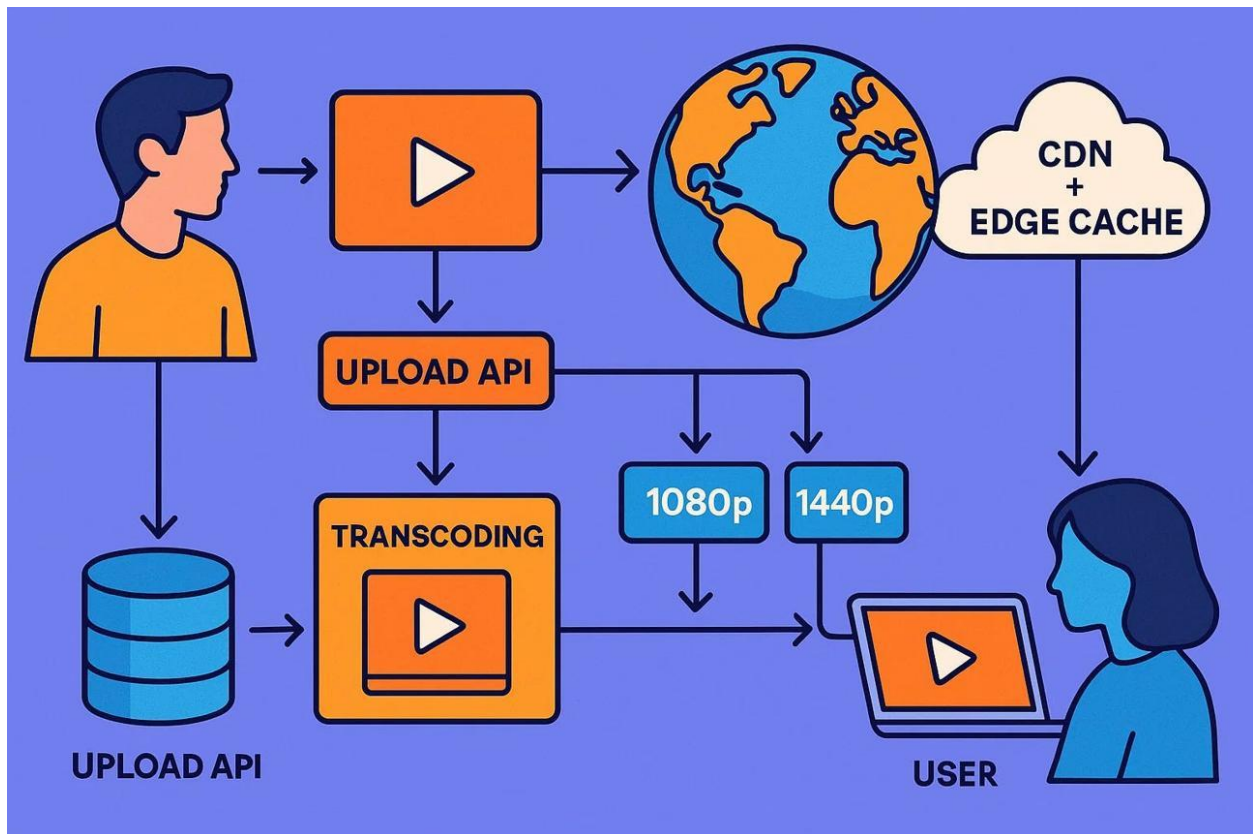
AKS + Service Bus + Durable Functions + SQL + Redis + Front Door

**AWS:**

EKS + EventBridge/SQS + Step Functions + RDS + ElastiCache + Route53

This is a strong **Enterprise Workflow / OMS / Saga / Cloud Architect** answer.

## Design 19 → YouTube-like Video Platform



This answers:

**“How do you design a YouTube-like video platform handling billions of videos, global streaming, uploads, search, and recommendations?”**

---

### **Q1. What is YouTube-like Video Platform?**

**Answer:**

A platform that supports:

- video upload
- encoding
- global streaming
- subscriptions
- likes/comments
- recommendations
- search
- analytics
- monetization

Huge FAANG media-system topic.

---

### **Q2. Core requirements?**

**Answer:**

#### **Functional Requirements**

- upload video
- transcoding
- playback
- adaptive bitrate
- comments
- likes/dislikes
- subscriptions
- notifications

- search
- recommendations
- watch history
- monetization/ads
- live stats

### **Non-Functional Requirements**

- billions of videos
- PB/EB storage
- low startup latency
- HA
- geo-distribution
- extreme read scale
- durability
- cost optimization

---

### **1. Capacity Estimation**

Assume:

- 2B users
- 100M uploads/day
- billions of play requests
- PB/EB storage

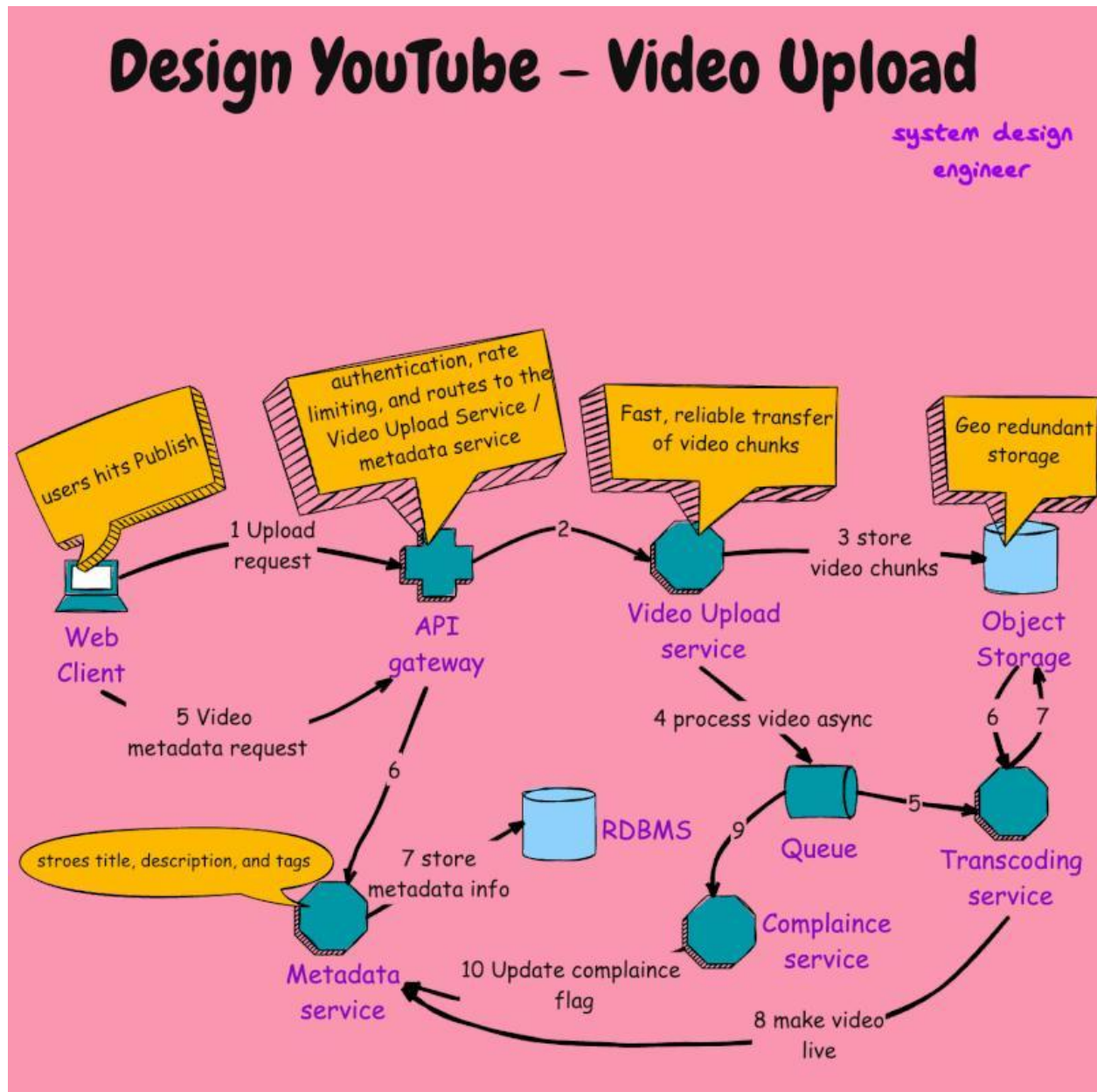
Challenges:

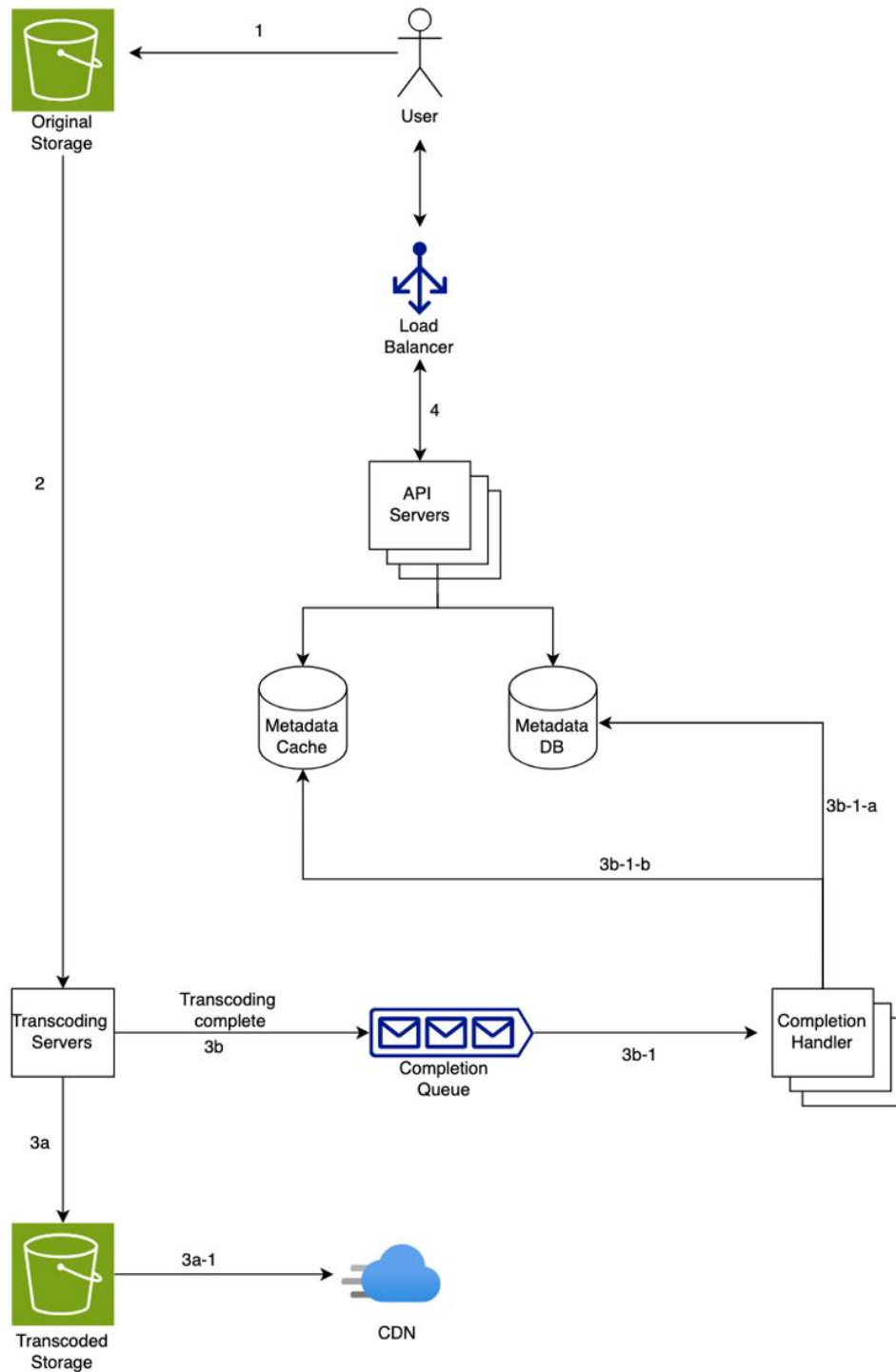
- transcoding scale
- hot viral videos
- metadata scale
- recommendation latency
- global CDN cost

Huge distributed topic.

---

## 2. High-Level Architecture





**Q3. Core components?**

**Answer:**

Usually:

- API Gateway
  - Upload Service
  - Transcoding Pipeline
  - Chunking Service
  - Metadata Service
  - Streaming Service
  - CDN
  - Recommendation Engine
  - Search Service
  - Comment Service
  - Subscription Service
  - Notification Service
  - Analytics Pipeline
  - Ad Service
  - Event Bus
- 

### 3. Upload Flow

#### Q4. Video upload flow?

**Answer:**

Step 1 → User uploads video  
Step 2 → Auth/validation  
Step 3 → Chunk upload  
Step 4 → Object storage  
Step 5 → Event publish  
Step 6 → Transcoding  
Step 7 → Thumbnail generation  
Step 8 → Metadata update  
Step 9 → CDN distribution  
Step 10 → Search indexing

Huge FAANG workflow.

---

#### 4. Why Chunking?



### **Q5. Why chunk video uploads?**

**Answer:**

Benefits:

- resumable uploads
  - retry partial failures
  - parallel upload
  - large-file scale
  - better reliability
- 

## **5. Transcoding (Critical)**

### **Q6. Why Transcoding Pipeline?**

**Answer:**

Convert raw video to multiple formats/resolutions.

Examples:

- 240p
- 480p
- 720p
- 1080p
- 4K

Huge architect topic.

---

### **Q7. Why async transcoding?**

**Answer:**

Heavy compute.

Never block request path.

Huge trap.

---

## **6. Adaptive Bitrate Streaming**

### **Q8. What is ABR?**

**Answer:**

Video quality dynamically adjusts based on bandwidth.

Improves:

- less buffering
  - stable playback
  - better UX
- 

## **7. Storage**

### **Q9. Storage layers?**

**Answer:**

Usually:

- Raw upload store
  - Encoded media store
  - Thumbnail store
  - CDN edge cache
  - Metadata DB
  - Cold archive
- 

### **Q10. Why object storage?**

**Answer:**

Large immutable media + durability + scale.

---

## **8. Streaming Flow**

### **Q11. Playback flow?**

**Answer:**

Step 1 → Request video  
Step 2 → Auth/region/DRM check  
Step 3 → Manifest lookup  
Step 4 → CDN edge fetch  
Step 5 → Segment stream

Step 6 → ABR adjustment

Step 7 → analytics emit

---

## 9. CDN (Critical)

### Q12. Why CDN?

**Answer:**

Moves media near users.

Reduces:

- latency
- origin load
- bandwidth pressure

Huge media topic.

---

### Q13. Viral video problem?

**Answer:**

Hot video gets massive reads.

Need:

- CDN
  - cache
  - edge replication
- 

## 10. Metadata

### Q14. Metadata service stores?

**Answer:**

- title
- description
- tags
- language
- category

- owner
  - duration
  - privacy
  - thumbnail
- 

## **11. Comments / Likes**

### **Q15. Why separate Comment Service?**

**Answer:**

Very high-write/read domain.

Independent scaling.

---

### **Q16. Like/dislike challenges?**

**Answer:**

Huge counters + abuse/spam + concurrency.

---

## **12. Subscription System**

### **Q17. What is Subscription Service?**

**Answer:**

Tracks creator followers.

Used for feed + notifications.

---

## **13. Search**

### **Q18. Search requirements?**

**Answer:**

- title
- tags
- transcript
- creator
- category

- ranking

Usually inverted index.

---

## **14. Recommendation Engine (Very Critical)**

### **Q19. Recommendation inputs?**

**Answer:**

- watch history
- click-through
- watch time
- subscriptions
- trending
- collaborative filtering
- ML ranking

Huge YouTube topic.

---

### **Q20. Why async recommendations?**

**Answer:**

Heavy ranking/ML compute.

Offline + near-real-time pipeline.

---

## **15. Notifications**

### **Q21. What notifications?**

**Answer:**

- new upload
  - live stream
  - comment reply
  - subscription updates
  - recommendation alerts
-

## 16. Analytics Pipeline

### Q22. What events captured?

Answer:

- views
- watch time
- drop-off
- likes
- CTR
- playback errors
- ad clicks

Huge platform topic.

---

## 17. Monetization / Ads

### Q23. Why separate Ad Service?

Answer:

Independent:

- bidding
  - targeting
  - ad tracking
  - fraud
  - billing
- 

## 18. Caching

### Q24. What to cache?

Answer:

- metadata
- recommendations
- hot videos
- thumbnails

- subscriptions
  - auth/session
- 

## **19. Reliability / Durability**

### **Q25. Durability strategy?**

**Answer:**

Use:

- object replication
  - checksums
  - multi-copy storage
  - cold backup
- 

### **Q26. What if transcoder fails?**

**Answer:**

Retry queue + idempotent jobs + replay.

---

## **20. Security / DRM**

### **Q27. Security risks?**

**Answer:**

- piracy
  - malicious upload
  - abuse
  - copyright violation
  - DDoS
  - bot spam
- 

### **Q28. Security controls?**

**Answer:**

- auth

- signed URLs
  - DRM
  - malware scan
  - copyright scan
  - moderation
  - rate limiting
  - WAF
- 

## 21. HA / DR

### Q29. HA design?

**Answer:**

Use:

- stateless APIs
  - geo CDN
  - replicated metadata DB
  - queue durability
  - multi-region
- 

### Q30. DR?

**Answer:**

- cross-region media replication
  - metadata restore
  - event replay
  - backup
- 

## 22. Scale

### Q31. How scale to billions?

**Answer:**

Use:



- chunking
  - async pipelines
  - CDN
  - sharded metadata
  - event-driven processing
  - autoscaling
- 

### **Q32. Common bottlenecks?**

**Answer:**

- transcoding farm
  - viral videos
  - recommendation latency
  - metadata hot rows
  - comment spikes
  - CDN misses
- 

## **23. Data Stores**

### **Q33. What DBs?**

**Answer:**

Usually polyglot:

- Object store → videos
  - Relational/NoSQL → metadata
  - Search cluster → indexing
  - Redis → cache
  - Event bus → analytics
  - Data lake → ML/reporting
- 

## **24. Azure Mapping (.NET-focused)**

### **Q34. Azure services?**

**Answer:**

| <b>Need</b>      | <b>Azure</b>              |
|------------------|---------------------------|
| APIs             | AKS / App Service         |
| Video processing | Media Services / AKS jobs |
| Object storage   | Blob Storage              |
| CDN              | Azure CDN / Front Door    |
| Event bus        | Event Hubs / Service Bus  |
| DB               | Cosmos DB / SQL           |
| Search           | Azure AI Search           |
| ML               | Azure ML                  |
| Cache            | Azure Redis               |
| Analytics        | Synapse / ADLS            |
| Monitoring       | App Insights              |
| Secrets          | Key Vault                 |

---

**25. AWS Mapping (.NET-focused)****Q35. AWS services?****Answer:**

| <b>Need</b>      | <b>AWS</b>              |
|------------------|-------------------------|
| APIs             | EKS / ECS               |
| Video processing | MediaConvert / EKS jobs |
| Object storage   | S3                      |
| CDN              | CloudFront              |
| Event bus        | Kinesis / SQS / SNS     |
| DB               | DynamoDB / RDS          |

| Need       | AWS             |
|------------|-----------------|
| Search     | OpenSearch      |
| ML         | SageMaker       |
| Cache      | ElastiCache     |
| Analytics  | Athena / S3     |
| Monitoring | CloudWatch      |
| Secrets    | Secrets Manager |

---

## 26. Trade-offs

### Q36. Pre-transcode vs on-demand transcode?

Answer:

#### Pre-transcode On-demand

fast playback cheaper initially

storage heavy latency risk

Usually pre-transcode.

---

### Q37. Strong vs eventual consistency?

Answer:

Strong:

- metadata updates
- permissions
- billing

Eventual:

- recommendations
- analytics
- comments indexing
- search

---

## 27. Anti-Patterns

### Q38. Dangerous anti-patterns?

Answer:

- serving video from DB
  - sync transcoding
  - no CDN
  - no ABR
  - no dedupe/chunking
  - no async analytics
- 

## 28. FAANG / Architect Trap Questions

### Q39. Can CDN replace origin?

Answer:

No.

---

### Q40. Can Redis store videos?

Answer:

No.

---

### Q41. Can recommendation be strongly consistent?

Answer:

Usually no.

---

## Final Architect Summary

Best design:

**API Gateway + Upload + Chunking + Object Storage + Async Transcoding + Streaming + CDN + Metadata + Search + Recommendation + Comments + Analytics + Multi-region HA**

Azure:

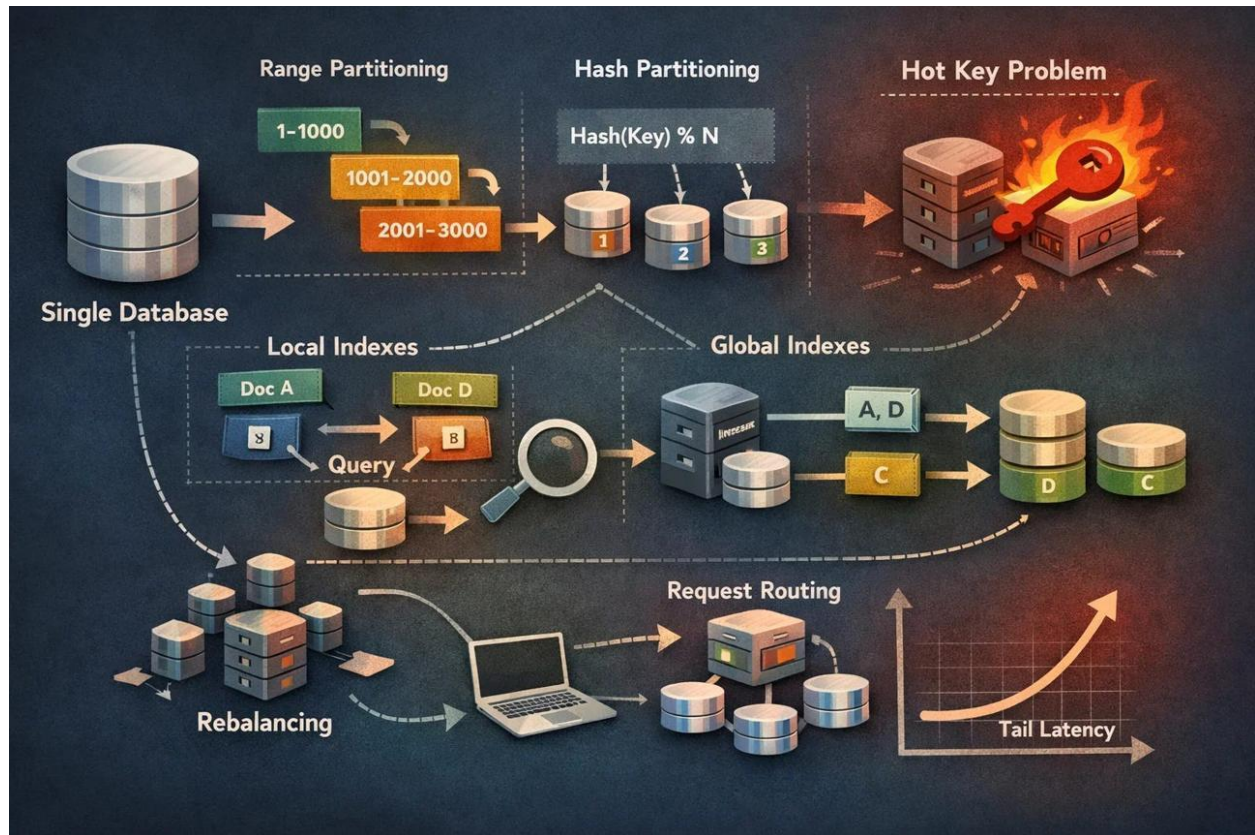
AKS + Media Services + Blob + CDN + Cosmos + Event Hubs + Azure ML

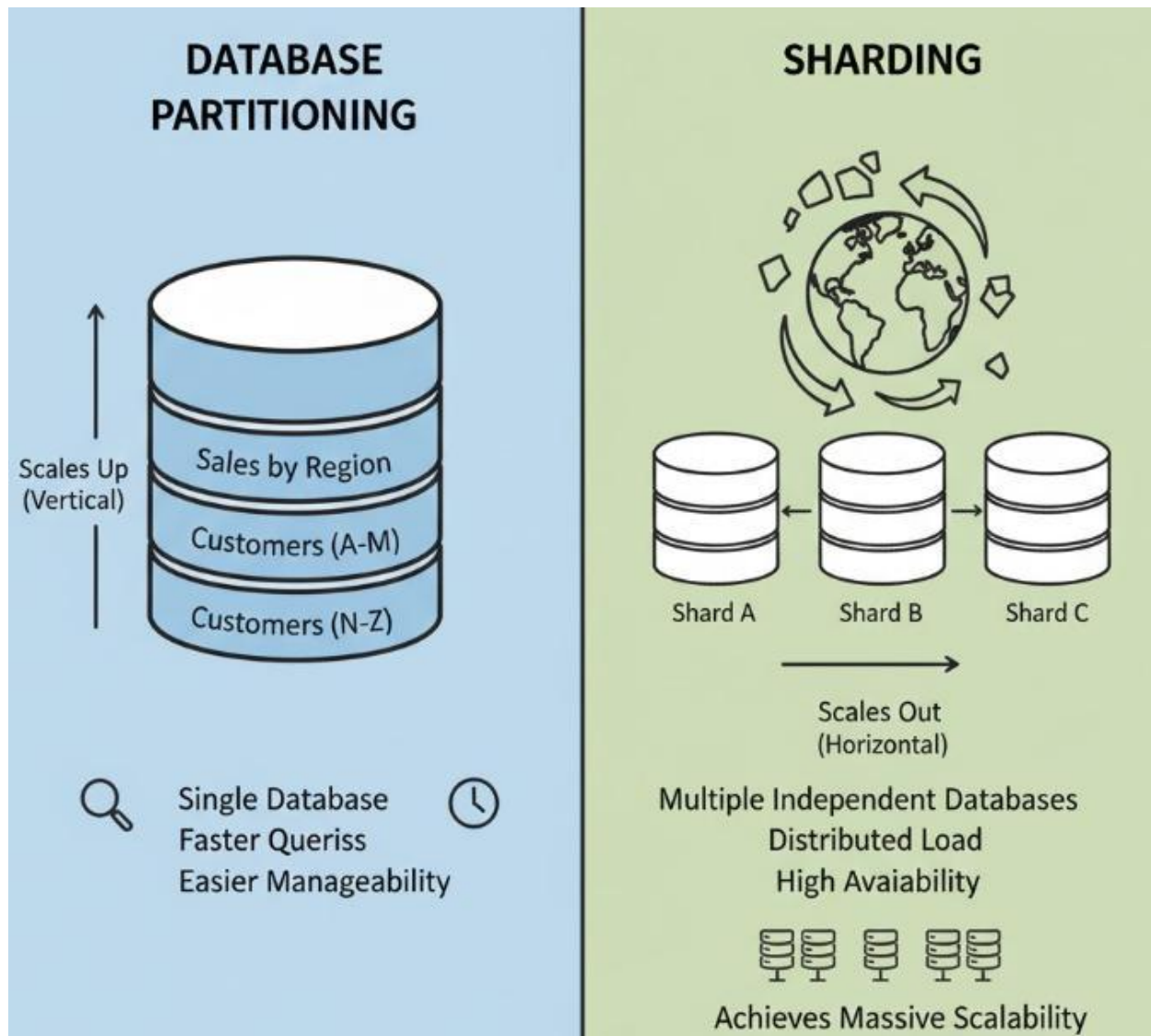
**AWS:**

EKS + MediaConvert + S3 + CloudFront + DynamoDB + Kinesis + SageMaker

This is a strong **YouTube / Media Platform / Cloud / FAANG Architect answer.**

## Design 20 → 10B+ Record Database Design (Sharding + Partitioning + Hot Partitions)





This answers:

**“How do you design a database layer storing 10B+ records with high write/read throughput, geo-scale, and minimal hotspots?”**

---

**Q1. Why normal DB becomes problem at 10B+ records?**

**Answer:**

At huge scale:

- large indexes
- slow scans
- lock contention

- storage growth
- replication lag
- expensive joins
- backup/restore pain
- hotspot issues

Single DB becomes bottleneck.

Huge architect topic.

---

## **Q2. Core requirements?**

**Answer:**

### **Functional Requirements**

- huge reads
- huge writes
- filtering
- range queries
- aggregation
- secondary indexes
- replication
- backup
- archival
- tenant isolation

### **Non-Functional Requirements**

- 10B+ records
- HA
- horizontal scale
- fault tolerance
- geo-distribution
- high write throughput
- low latency



- durability
- 

## 1. Capacity Estimation

Assume:

- 10B–100B rows
- TB/PB scale
- 5–50M writes/day
- millions reads/sec

Challenges:

- shard imbalance
- rebalancing
- hot partitions
- index size
- backups
- replication lag

Huge DB architect topic.

---

## 2. High-Level Architecture

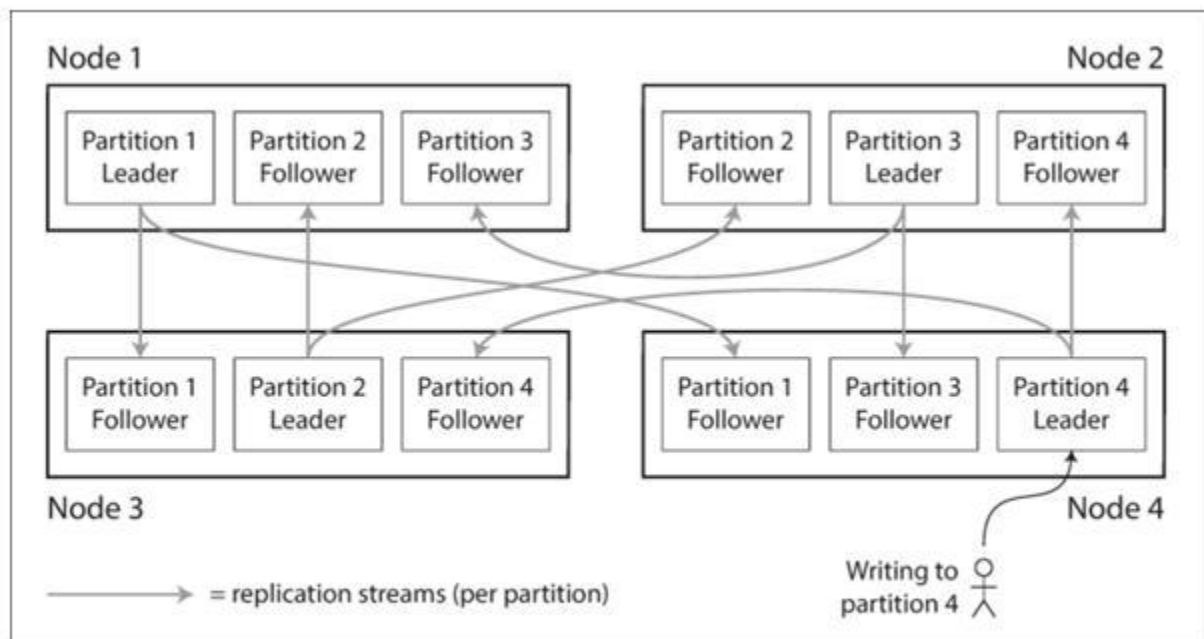
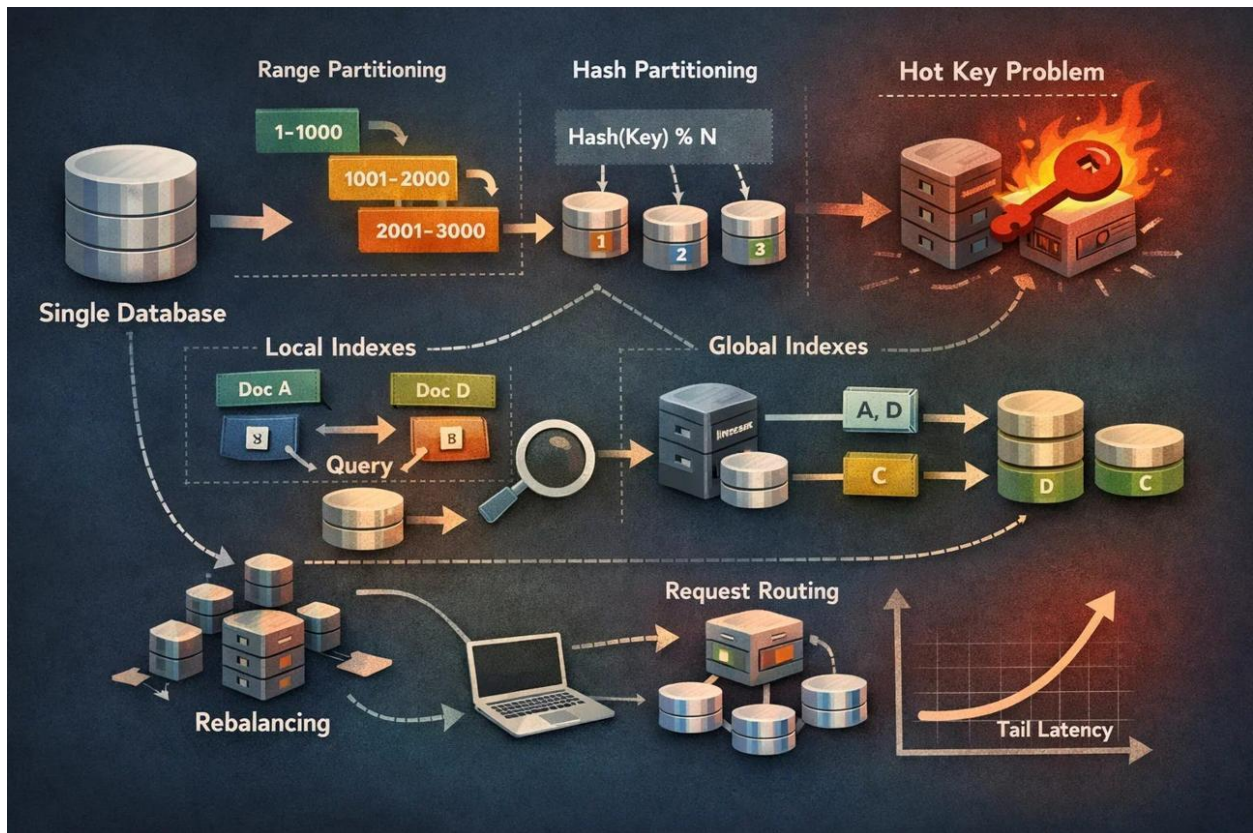


Figure 6-1. Combining replication and partitioning: each node acts as leader for some partitions and follower for other partitions.

Q3. Core components?

**Answer:**

Usually:

- App/API Layer
  - Query Router
  - Shard Map / Metadata Service
  - Primary Shards
  - Replica Shards
  - Secondary Index Layer
  - Cache Layer
  - Archival Store
  - Backup/Restore Service
  - Rebalancer
  - Observability
- 

### **3. Partitioning vs Sharding (Critical)**

**Q4. What is partitioning?**

**Answer:**

Splitting large data into smaller partitions.

Can be logical or physical.

---

**Q5. What is sharding?**

**Answer:**

Distributing partitions across multiple independent DB nodes.

Huge FAANG favorite.

---

**Q6. Partitioning vs Sharding?**

**Answer:**

**Partitioning****Sharding**

split data

distribute across nodes

can stay same DB multi-node scale

Must know.

---

**4. Sharding Strategies (Very Important)****Q7. Common sharding methods?**

**Answer:**

- hash-based
- range-based
- geo-based
- tenant-based
- time-based
- composite

Critical architect topic.

---

**Q8. Hash-based sharding?**

**Answer:**

$\text{hash}(\text{userId}) \% N$

Good:

- balanced load

Weak:

- poor range queries
- 

**Q9. Range-based sharding?**

**Answer:**

Examples:

Jan–Mar → shard1

Apr–Jun → shard2

Good:

- range scans

Weak:

- hotspots
- 

#### **Q10. Geo-based sharding?**

**Answer:**

Shard by region.

Examples:

India, EU, US.

Used for residency + latency.

---

#### **Q11. Tenant-based sharding?**

**Answer:**

Multi-tenant SaaS.

tenantId routes shard.

---

#### **Q12. Which best?**

**Answer:**

Usually hybrid/composite.

Very strong architect answer.

---

#### **5. Hot Partition Problem (Critical)**

##### **Q13. What is hot partition?**

**Answer:**

One shard/partition receives disproportionate load.

Huge distributed DB topic.

---

##### **Q14. Example?**

**Answer:**

Latest timestamp shard,  
viral customer,  
single hot tenant.

---

**Q15. Why dangerous?**

**Answer:**

Causes:

- high latency
  - CPU spikes
  - queue buildup
  - throttling
- 

**Q16. How solve hot partitions?**

**Answer:**

Use:

- better shard key
  - key salting
  - shard splitting
  - load rebalance
  - caching
  - write spreading
- 

Very common architect question.

---

**6. Query Routing**

**Q17. Why Query Router?**

**Answer:**

Routes queries to correct shard.

Avoids full fan-out.

---

**Q18. Shard map service?**

**Answer:**

Stores:

- shard ownership
  - topology
  - split state
  - region
- 

**7. Replication****Q19. Why replication?**

**Answer:**

Supports:

- HA
  - read scale
  - DR
  - failover
- 

**Q20. Primary-replica pattern?**

**Answer:**

Writes → primary

Reads → replicas.

Very common.

---

**Q21. Risks?**

**Answer:**

- replication lag
- stale reads
- split brain

---

## 8. Read/Write Split

### Q22. Why read replicas?

**Answer:**

Reads usually dominate.

Helps scale.

---

### Q23. When read-after-write issue?

**Answer:**

Replica lag.

Huge trap.

---

## 9. Secondary Indexes

### Q24. Why secondary indexes hard?

**Answer:**

At 10B scale:

- huge storage
- cross-shard complexity
- maintenance cost

Very important.

---

### Q25. Global vs local index?

**Answer:**

**Global Index   Local Index**

across shards   per shard

complex   simpler

---

## 10. Rebalancing



### Q26. Why rebalance?

**Answer:**

Traffic/data changes over time.

Need even distribution.

---

### Q27. How rebalance?

**Answer:**

- move shard
  - split shard
  - merge shard
  - update routing
- 

## 11. Archival / Tiering

### Q28. Why archive?

**Answer:**

Old data grows huge.

Move cheaper storage.

---

### Q29. Common retention?

**Answer:**

Hot/Warm/Cold:

- SSD hot
- cheaper warm
- archive/cold

Huge cloud topic.

---

## 12. Data Model Choices

### Q30. SQL vs NoSQL?

**Answer:**

## SQL

## NoSQL

strong transactions flexible scale

joins

denormalized scale

Usually polyglot.

---

### Q31. When polyglot DB?

**Answer:**

Different workloads:

- transactions
- analytics
- cache
- search

Very strong answer.

---

## 13. Consistency

### Q32. Strong vs eventual consistency?

**Answer:**

Strong:

- payments
- banking
- inventory

Eventual:

- analytics
  - feed
  - recommendations
  - search index
- 

## 14. Caching

### Q33. What to cache?

Answer:

- hot rows
  - aggregates
  - indexes
  - tenant configs
  - lookup tables
- 

### 15. Backup / Restore

#### Q34. Why difficult at 10B+?

Answer:

Huge size + long restore.

Must design incremental strategy.

---

#### Q35. Strategy?

Answer:

- snapshots
- incremental backup
- PITR
- geo copies

Must know.

---

### 16. Geo-Distribution

#### Q36. Why geo-shard?

Answer:

- latency
- compliance
- DR
- regional traffic

---

**Q37. Risks?**

**Answer:**

- cross-region consistency
  - conflicts
  - latency
- 

**17. Scale****Q38. How scale to 10B+ records?**

**Answer:**

Use:

- shard routing
  - partitioning
  - replication
  - tiered storage
  - caches
  - denormalization
  - async pipelines
- 

**Q39. Common bottlenecks?**

**Answer:**

- hot shards
  - index rebuild
  - rebalance
  - replica lag
  - large scans
  - cross-shard joins
- 

**18. Security**

**Q40. Security risks?****Answer:**

- tenant leak
  - weak backups
  - exposed replicas
  - key theft
  - data corruption
- 

**Q41. Controls?****Answer:**

Need:

- encryption
  - RBAC
  - row-level security
  - tenant isolation
  - audit
  - backup encryption
  - secret rotation
- 

**19. HA / DR****Q42. HA design?****Answer:**

Use:

- multi-AZ
  - replicas
  - quorum
  - failover
  - autoscale
-

#### Q43. DR strategy?

Answer:

- geo replica
- snapshot restore
- tested failover
- PITR

---

#### 20. Azure Mapping (.NET-focused)

##### Q44. Azure services?

Answer:

| Need             | Azure                |
|------------------|----------------------|
| Relational scale | Azure SQL Hyperscale |
| NoSQL            | Cosmos DB            |
| Cache            | Azure Redis          |
| Archive          | Blob / ADLS          |
| Search           | Azure AI Search      |
| Analytics        | Synapse              |
| Messaging        | Event Hubs           |
| Monitoring       | App Insights         |
| Security         | Key Vault            |
| Containers       | AKS                  |

Very strong Azure topic.

---

#### 21. AWS Mapping (.NET-focused)

##### Q45. AWS services?

Answer:

| Need             | AWS               |
|------------------|-------------------|
| Relational scale | Aurora / RDS      |
| NoSQL            | DynamoDB          |
| Cache            | ElastiCache       |
| Archive          | S3 Glacier        |
| Search           | OpenSearch        |
| Analytics        | Athena / Redshift |
| Messaging        | Kinesis           |
| Monitoring       | CloudWatch        |
| Security         | Secrets Manager   |
| Containers       | EKS               |

---

## 22. Trade-offs

### Q46. Hash vs Range sharding?

Answer:

| Hash               | Range        |
|--------------------|--------------|
| balanced           | better scans |
| weak range queries | hotspot risk |

---

### Q47. Normalize vs denormalize?

Answer:

| Normalize   | Denormalize    |
|-------------|----------------|
| consistency | performance    |
| joins       | duplicate data |

---

## 23. Anti-Patterns

#### Q48. Dangerous anti-patterns?

**Answer:**

- sequential shard key
  - no shard map
  - cross-shard joins everywhere
  - no rebalancing
  - global transactions everywhere
  - huge unbounded indexes
- 

#### 24. FAANG / Architect Trap Questions

##### Q49. Can one DB scale to 10B+?

**Answer:**

Sometimes yes (powerful systems), but distributed design usually safer.

---

##### Q50. Is sharding enough?

**Answer:**

No.

Need:

- indexing
  - replication
  - rebalancing
  - cache
  - DR
- 

##### Q51. Can replicas eliminate hotspot?

**Answer:**

Not always.

Write hotspots remain.

---



## Final Architect Summary

Best design:

**Query Router + Shard Map + Hash/Range/Geo Shards + Replicas + Secondary Index Strategy + Cache + Rebalancer + Archive + Geo DR + Multi-region HA**

**Azure:**

Cosmos + Azure SQL Hyperscale + Redis + Blob + Synapse + AKS

**AWS:**

DynamoDB + Aurora + ElastiCache + S3 + Redshift + EKS

This is a strong **Database / Sharding / Scale / Cloud / FAANG Architect answer.**